

Экзамен СиФО ЭВМ

- 1.Что относится к понятию архитектура команд?**
- 2.В чем состоит проблема семантического разрыва?**
- 3. Аккумуляторная архитектура системы команд, достоинства и недостатки.**
- 4. Стековая архитектура системы команд, достоинства и недостатки.**
- 5. Регистровая архитектура системы команд, достоинства и недостатки.**
- 6. Архитектура системы команд с выделенным доступом к памяти.**
- 7. Принципы CISC архитектуры**
- 8. Принципы RISC архитектуры.**
- 9. Типы команд.**
- 10.Способы адресации. Непосредственная и регистровая адресация.**
- 11.Способы адресации. Прямая и косвенная регистровая адресация.**
- 12.Способы адресации. Косвенная, автоинкрементная и автодекрементная адресация**
- 13.Способы адресации. Адресация со смещением (3 типа).**
- 14.Способы адресации. Страничная и блочная адресации.**
- 15.Способы адресации. Стековая адресация.**
- 16.Транзакция. Целевое назначение шин.**
- 17.Иерархия шин.**
- 18.Распределение линий шины управления.**
- 19.Алгоритмы смены приоритетов при арбитраже шин.**
- 20.Разновидности централизованного арбитража.**
- 21.Разновидности децентрализованного арбитража.**
- 22.Понятие протокола шины.**
- 23.Последовательность действий в процедуре квитирования установления связи.**
- 24.Синхронные и асинхронные шины, достоинства и недостатки.**
- 25.Методы повышения эффективности шин.**
- 26.Чем характеризуются ЗУ?**
- 27. Основная проблема памяти и способ её решения.**
- 28. Система ввода-вывода, понятие и задачи.**
- 29.Совмещенное адресное пространство СВВ, достоинства и недостатки.**
- 30.Конфликты в конвейере команд. Типы рисков.**
- 31.Риск по данным, типы конфликтов по данным.**
- 32.Предсказание переходов — статический и динамический.**
- 33.Стратегии статического перехода.**
- 34.Классификация архитектур системы команды (АСК)**
- 35.Принципы концепции ВМ фон Неймана. Какой из принципов можно назвать наиболее существенным?**
- 36.Принципы EPIC архитектуры (IA-64).**
- 37.Какие факторы влияют на длину команды? Возможные пути сокращения длины команды.**

38. В чём суть и достоинства конвейеризации транзакций?
39. Способы ускорения транзакций.
40. Методы доступа к ЗУ?
41. Иерархия запоминающих устройств.
42. Статические и динамические ОЗУ?
43. Блочная схема расслоения памяти.
44. Циклическая схема расслоения памяти.
45. Блочно-циклическая схема расслоения памяти.
46. Последовательный и конвейерный режим доступа к микросхеме ЗУ.
47. Регистровый и страничный режим доступа к микросхеме ЗУ.
48. Микросхемы SDRAM, отличия от асинхронных DRAM.
49. Энергонезависимые ОЗУ.
50. Специализированные ОЗУ.
51. Типы микросхем ПЗУ.
52. «Flash» ПЗУ.
53. Производительность иерархии памяти.
54. Ассоциативная память.
55. Кэш с полностью ассоциативным отображением.
56. Кэш с прямым отображением.
57. Кэш с множественно-ассоциативным отображением.
58. Эффективность множественно-ассоциативного отображения. Размеры строки.
- 59-60. Синхронизация данных. Метод сквозной/обратной записи.
61. Гарвардская архитектура организации. Преимущества.
62. Выделенное (изолированное) адресное пространство СВВ, достоинства и недостатки.
63. Структура ПУ. Модули ввода-вывода.
64. Структура и функции модуля ввода/вывода.
65. Программно управляемый ВВ.
66. Ввод/вывод по прерываниям.
67. Динамические схемы предсказания: А1.
68. Динамические схемы предсказания: А2.
69. Динамические схемы предсказания: А3.
70. Адрес команды условного перехода в качестве шаблона доступа к таблице предыстории переходов (РНТ).
71. Регистр глобальной истории в качестве шаблона доступа к таблице предыстории переходов (РНТ).
72. Регистр локальной истории в качестве шаблона доступа к таблице предыстории переходов (РНТ).
73. Комбинационный шаблон доступа к таблице предыстории переходов (РНТ).
74. Параллелизм. Уровни параллелизма.
75. Классификация Флинна.
76. Конвейеризация вычислений. Линейные и нелинейные конвейеры. Основные ступени конвейеров команд.

77. Программное и аппаратное устранение конфликтов по данным.
78. Риски по управлению. Потери производительности на переходах. Способы сокращения издержек на переходы.
79. Способы решения проблемы условного перехода
80. Переименование регистров.
81. Идентификация устройств при вводе/выводе по прерываниям.
82. Прямой доступ к памяти. Понятие и задачи КПП.
83. Переупорядочивание команд с использованием технологии «шелвинга»
84. Типы прерываний. Контроллеры прерываний ПК.
85. Типы регенерации динамических ЗУ.
86. Микросхемы DDR SDRAM.
87. Суперконвейерные процессоры.
88. Суперскалярные процессоры.
89. Операционные автоматы. Структура М- автомата и S- автоматов.
90. Операционные автоматы. Структура I-автомата.
91. Управляющие автоматы с жёсткой логикой. Автоматы Мили и Мура.
92. Управляющие автоматы с программируемой логикой.
93. Кодирование микрокоманд. Горизонтальный, вертикальный способы кодирования.
94. Кодирование микрокоманд. Смешанный способ кодирования команд.
95. Последовательность выполнения микропрограмм.
96. Организация памяти микропрограмм.
97. Закон Амдала.
98. Закон Густафсона.
99. Организация памяти вычислительных систем.
100. Модели архитектур распределённой памяти.
101. Когерентность -памяти.
102. Протокол MESI.
103. Организация виртуальной памяти.
104. Построение управляющих автоматов.
105. Способы кодирования микропрограмм.
106. Команды арифметической и логической обработки.
107. Команды управления потоком команд.
108. В чём состоит особенность SIMD-команд и в каком формате должны быть представлены операнды? Что такое «арифметика с насыщением» и где она применяется?
109. Особенности архитектуры ARM процессоров.
110. Организация виртуальной памяти. Многоуровневые таблицы трансляции адресов.
111. Архитектура ARM Cortex процессоров

1. Что относится к понятию архитектура команд?

Архитектура команд – интерфейс между всем выполняемым на машине ПО и аппаратным обеспечением.

Перечень видимых для программиста составных частей архитектуры системы команд:

- коды команд
- число и типы регистров
- форматы команд
- способы адресации и доступа к памяти
- условия возникновения исключений

АСК (ISA – Instruction Set Architecture) – очень важный уровень абстракции:

- интерфейс между аппаратурой и низко-уровневым ПО
- стандартизирует команды, шаблоны битов машинного языка и т.д.
- преимущества: позволяет реализовывать различные аппаратные варианты одной программной архитектуры
- недостатки: периодически препятствует внедрению новых технологий и прочих инноваций

Современные АСК: – 80x86/Pentium, PowerPC, MIPS, HP

Классификация архитектур системы команд (по месту хранения операндов):

Аккумуляторная Архитектура (EDSAC, 1950)

Регистровая Архитектура (IBM 360, 1964)

Архитектура с выделенным доступом к памяти Load/Store (CDC 6600, Cray I, 1963-76)

2. В чем состоит проблема семантического разрыва?

Современная технология программирования ориентирована на языки высокого уровня, главная цель которых — облегчить процесс программирования. Такой переход породил серьезную проблему: сложные операторы, характерные для ЯВУ, существенно отличаются от простых машинных операций, реализуемых в большинстве вычислительных машин. Следствием становится недостаточно эффективное выполнение программ на ВМ.

Факт различия принципов, лежащих в основе языков программирования высокого уровня и тех принципов, которые определяют архитектуру ЭВМ, называется *семантическим разрывом*.

Одной из дополнительных причин семантического разрыва является использование двоичной арифметики: современные вычислительные системы вынуждены затрачивать много времени на десятично-двоичные преобразования.

Таблица 2. 1. Сравнительная оценка CISC-, RISC- и VLIW-архитектур

Характеристика	CISC	RISC	VLIW
Длина команды	Варьируется	Единая	Единая
Расположение полей в команде	Варьируется	Неизменное	Неизменное
Количество регистров	Несколько (часто специализированных)	Много регистров общего назначения	Много регистров общего назначения
Доступ к памяти	Может выполняться как часть команд различных типов	Выполняется только специальными командами	Выполняется только специальными командами -

Таблица 2.1 позволяет оценить наиболее существенные различия в архитектурах типа CISC, RISC и VLIW.

3. Аккумуляторная архитектура системы команд, достоинства и недостатки.

Архитектура на базе аккумулятора исторически возникла одной из первых. В ней для хранения одного из операндов арифметической или логической операции в процессоре имеется выделенный регистр — аккумулятор. В этот же регистр заносится и

результат операции. Поскольку адрес одного из операндов предопределен, в командах обработки достаточно явно указать местоположение только второго операнда. Изначально оба операнда хранятся в основной памяти, и до выполнения операции один из них нужно загрузить в аккумулятор. После выполнения команды обработки результат находится в аккумуляторе и, если он не является операндом для последующей команды, его требуется сохранить в ячейке памяти.

Для загрузки в аккумулятор содержимого ячейки x предусмотрена команда загрузки $\text{load } x$. По этой команде информация считывается из ячейки памяти x , выход памяти подключается к входам аккумулятора и происходит занесение считанных данных в аккумулятор. Запись содержимого аккумулятора в ячейку x осуществляется командой сохранения $\text{store } x$, при выполнении которой выходы аккумулятора подключаются к шине, после чего информация с шины записывается в память.

Для выполнения операции в АЛУ производится считывание одного из операндов из памяти в регистр данных. Второй операнд находится в аккумуляторе. Выходы регистра данных и аккумулятора подключаются к соответствующим входам АЛУ. По окончании предписанной операции результат с выхода АЛУ заносится в аккумулятор.

Достоинствами аккумуляторной АСК можно считать короткие команды и простоту декодирования команд. Однако наличие всего одного регистра порождает многократные обращения к основной памяти.

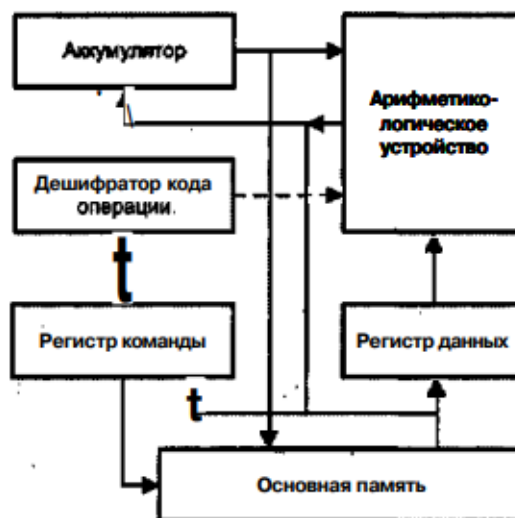


Рис. 2.7. Архитектура вычислительной машины на базе аккумулятора

4. Стековая архитектура системы команд, достоинства и недостатки.

Стек – множество логически взаимосвязанных по принципу LIFO ячеек.

Две операции – push и pop . Сохранение из стека в память x по команде ($\text{pop } x$).

Запись и чтение только в/из вершины стека.

Операнды помещаются в стек, в процессе операции они выталкиваются из стека.

Результат операции – в стек.

Для стековой архитектуры лучше всего подходит обратная польская нотация (Чарльз Хэмблин, 1957), разработанная на основе «польской нотации».

Информация в стек – из памяти или АЛУ. Результат – в АЛУ.

Достоинства:

- стек может быть неоднороден (верхние ячейки быстрые, нижние медленные).
- сокращение адр. части команд.
- компактный код
- простой компилятор
- простое декодирование команд

Недостатки:

- Компилятор не может создать эф. код (нет произвольного обращения к памяти).
- Стек – узкое место. Низкая производительность.

Последние годы – повышение интереса к стековой архитектуре, т.к. Java и Forth.
ROSC (Removed Operand Set Computer) – новая реинкарнация этой архитектуры.



Рис. 2.6. Архитектура вычислительной машины на базе стека

5. Регистровая архитектура системы команд, достоинства и недостатки.

Массив регистров (РОН) – явно управляемый для хранения недавно использовавшихся данных. Размер регистров – обычно совпадает с машинным словом.

К любому регистру можно обратиться по номеру.

Меньшее количество регистров – меньше разрядов на кодирование номера регистра – короче команда. Операнды – либо в памяти, либо в регистрах.

Три подвида команд: Регистр-регистр, Регистр-память, Память-память.

Таблица 2.4. Сравнительная оценка вариантов размещения операндов

Вариант	Достоинства	Недостатки
Регистр-регистр (0,3)	Простота реализации, фиксированная длина команд, простая модель формирования объектного кода при компиляции программ, возможность выполнения всех команд за одинаковое количество тактов	Большая длина объектного кода, из-за фиксированной длины команд часть разрядов в коротких командах не используется
Регистр-память (1,2)	Данные могут быть доступны без загрузки в регистры процессора, простота кодирования команд, объектный код получается достаточно компактным	Потеря одного из операндов при записи результата, длинное поле адреса памяти в коде команды сокращает место под номер регистра, что ограничивает общее число РОН. CPI зависит от места размещения операнда
Память-память (3,3)	Компактность объектного кода, малая потребность в регистрах для хранения промежуточных данных	Разнообразие форматов команд и времени их исполнения, низкое быстродействие из-за обращения к памяти

Операции загрузки операндов в РОН – идентичны работе с аккумулятором. Отличие – выбор нужного регистра.

Выполнение операции АЛУ включает в себя:

- Выбор регистра первого операнда
- Определение местоположения второго операнда (память или регистр)
- Подача на вход АЛУ операндов и выполнение операции
- Выбор регистра результата и занесение в него результата операции из АЛУ.

Между АЛУ и массивом регистров должно быть минимум три шины.

Достоинства – компактность кода, высокая скорость.

Недостатки – более длинные инструкции

Регистровая АСК – преобладающая в настоящее время.

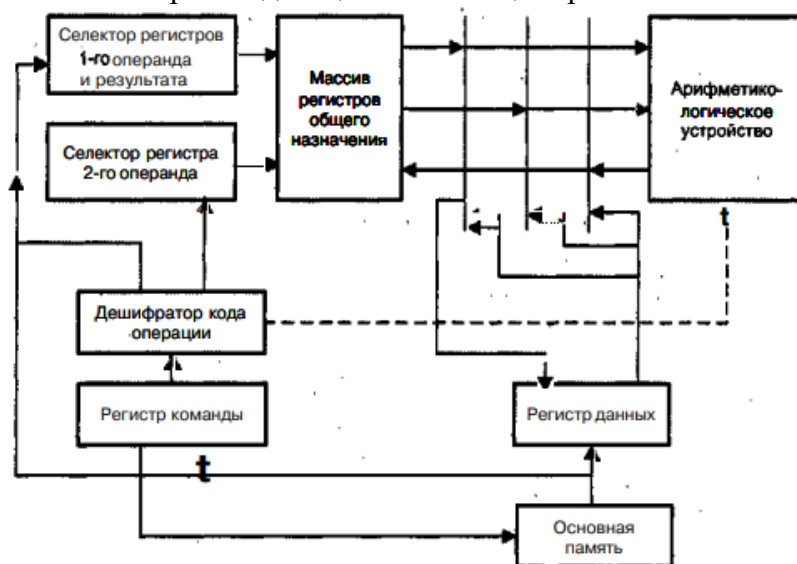


Рис. 2.8. Архитектура вычислительной машины на базе регистров общего назначения

6. Архитектура системы команд с выделенным доступом к памяти.

В архитектуре с выделенным доступом к памяти обращение к основной памяти возможно только с помощью двух специальных команд: load и store. Команда load (загрузка) обеспечивает считывание значения из основной памяти и занесение его в регистр процессора (в команде обычно указывается адрес ячейки памяти и номер регистра). Пересылка информации в противоположном направлении производится командой store (сохранение). Операнды во всех командах обработки информации могут находиться только в регистрах процессора (чаще всего в регистрах общего назначения). Результат операции также заносится в регистр. В архитектуре отсутствуют команды обработки, допускающие прямое обращение к основной памяти. Допускается наличие в АСК ограниченного числа команд, где операнд является частью кода команды.

Две из трех шин, расположенных между массивом РОН и АЛУ обеспечивают передачу в арифметико-логическое устройство операндов, хранящихся в двух регистрах общего назначения. Третья служит для занесения результата в выделенный для этого регистр. Эти же шины позволяют загрузить в регистры содержимое ячеек основной памяти и сохранить в ОП информацию, находящуюся в РОН.

АСК с выделенным доступом к памяти характерна для всех вычислительных машин с RISC-архитектурой. Команды в таких ВМ, как правило, имеют длину 32 бита и трехадресный формат. К достоинствам АСК следует отнести простоту декодирования и исполнения команды.

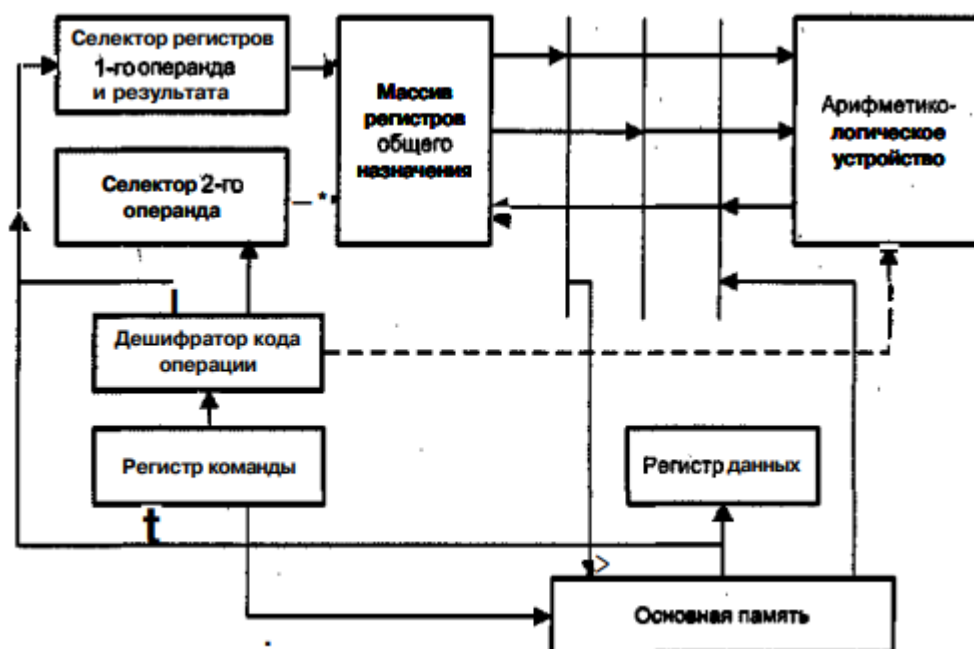


Рис. 2.9. Архитектура вычислительной машины с выделенным доступом к памяти

7. Принципы CISC архитектуры.

В вычислительных машинах типа CISC проблема семантического разрыва решается счет расширения системы команд, дополнения ее сложными командами, семантически аналогичными операторам ЯВУ. Основоположником CISC-архитектуры считается компания IBM.

Для CISC-архитектуры типичны:

- наличие в процессоре сравнительно небольшого числа регистров общего назначения;
- большое количество машинных команд, некоторые из них аппаратно реализуют сложные операторы ЯВУ;
- разнообразие способов адресации операндов;
- множество форматов команд различной разрядности;
- наличие команд, где обработка совмещается с обращением к памяти.

Недостатки CISC:

- высокая стоимость аппаратной части (особенно УУ)
- сложности с распараллеливанием вычислений

В CISC более сложные команды могут разбиваться на несколько частей, которые затем выполняются как последовательность микрокоманд. Эта операция снижает скорость работы, но она полезна для редко встречающихся команд.

8. Принципы RISC архитектуры.

RISC (Reduced Instruction Set Computer) - архитектура с сокращённым набором команд.

Детальный результатов упомянутых исследований привел к серьезному пересмотру традиционных решений, следствием чего стало появление RISC-архитектуры. Идея заключается в ограничении списка команд ВМ наиболее часто используемыми простейшими командами, оперирующими данными, размещенными только в регистрах процессорах. Обращение к памяти допускается лишь с помощью специальных команд чтения и записи. Резко уменьшено количество форматов команд и способов указания адресов операндов. Сокращение числа форматов команд и их простота, использование ограниченного количества способов адресации, отделение операций обработки данных от

операций обращения к памяти позволяет существенно упростить аппаратные средства ВМ и повысить их быстродействие. RISC-архитектура разрабатывалась таким образом, чтобы уменьшить время выч. за счет сокращения CPI и время пр. Как следствие, реализация сложных команд за счет последовательности из простых, но быстрых RISC-команд оказывается не менее эффективной, чем аппаратный вариант сложных команд в CISC-архитектуре.

Характерно:

- все команды выполняются непосредственно аппаратным обеспечением
- компьютер должен начинать выполнение большого количества команд (предполагается повышение производительности с помощью параллелизма)
- команды должны легко декодироваться (опять же для параллелизма, необходимо определять какие ресурсы нужны для выполнения команды)
- к памяти должны обращаться только команды загрузки и сохранения (чтобы не мешать параллелизму, ибо доступ к памяти - долгая история)
- должно быть большое количество регистров (ну чтобы к памяти слишком часто не обращаться)

9. Типы команд.

Команда – совокупность сведений, представленных в виде двоичных кодов, необходимых процессору для выполнения очередного шага.

Команды пересылки данных.

Примеры: передача данных между ЦП и ОП, внутри процессора, внутри памяти.

Содержит: - адреса источника и получателя операндов, размер данных, способ адресации каждого из операндов.

Команды арифметической и логической обработки.

Обеспечивают арифметическую и логическую обработку информации в различных формах ее представления.

Команды SIMD — Single Instruction Multiple Data.

Эти команды обрабатывают сразу две группы чисел. Операнды обычно представлены в упакованном формате.

Команды работы со строками.

Перемещение, сравнение и поиск строк.

Команды преобразования.

Изменение формата представления данных.

Команды ввода/вывода.

- Команды управления периферийными устройствами (ПУ). - Команды проверки состояния ПУ. - Команды ввода /вывода.

Команды управления системой.

Эти команды являются привилегированными командами, выполняются, когда ЦП находится в привилегированном состоянии либо выполняет привилегированную программу (обычно ОС). С помощью данных команд могут быть изменено состояние ряда регистров устройства управления.

Команды управления потоком команд.

Вместо инкрементации счётчика команд команда загружает в счётчик адрес следующей команды.

Три типа таких команд: - безусловные переходы; - условные переходы (ветвления); - вызовы процедур и возврат из процедур.

10.Способы адресации. Непосредственная и регистровая адресация.

При непосредственной адресации (НА) в адресном поле команды вместо адреса содержится непосредственно сам операнд. Этот способ применяется при выполнении арифметических операций, операций сравнения, для загрузки констант в регистры.

Когда операндом является число, оно обычно представляется в дополнительном коде. При записи в регистр, имеющий разрядность, превышающую длину непосредственного операнда, операнд размещается в младшей части регистра, а оставшиеся свободными позиции заполняются значением знакового бита операнда.

В адресном поле могут быть указаны только константы, доп. недостатком данного способа адресации – размер непосредственного операнда ограничен длиной адресного поля команды, которое в большинстве случаев меньше длины машинного слова.

Длина операнда редко превышает 16 бит. Т.о, в подавляющем числе случаев 16 разрядов вполне достаточно, хотя для вычисления адресов const может быть длиннее.

Основная проблема – не каждый операнд можно передать непосредственно – адр. часть команды как правило меньше машинного слова. 50-60% - до 8 бит, 70-80% - до 16 бит.

Плюсы – малое время выполнения команды, экономия памяти.

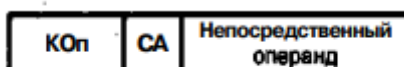


Рис. 2.45. Непосредственная адресация

Регистровая адресация (РА) напоминает прямую адресацию. Адресное поле инструкции указывает на регистр процессора. Обычно размер адресного поля в данном случае составляет три или четыре бита, что позволяет указать соответственно на один из 8 или 16 регистров общего назначения (РОН). Двумя основными преимуществами регистровой адресации являются: короткое адресное поле в команде и исключение обращений к памяти.

Плюсы – размер адресного поля 3-4 бита для 8-16 РОН. Исключение обращений к памяти.

Минусы – мало РОН.

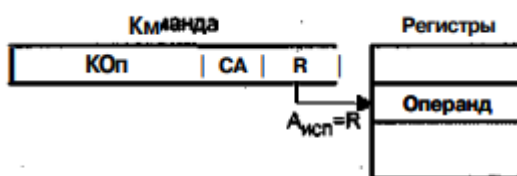


Рис. 2.50. Регистровая адресация

Способ адресации – это способ формирования исполнительного адреса по адресному коду команды.

11.Способы адресации. Прямая и косвенная регистровая адресация.

При прямой (абсолютной) адресации (ПА) адресный код указывает номер ячейки памяти, к которой производится обращение, то есть адресный код = исполнит. адрес.

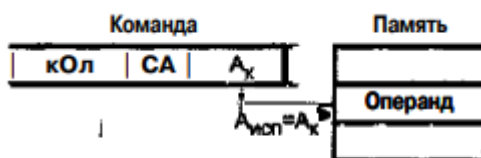


Рис. 2.48. Прямая адресация

При всей простоте использования способ имеет существенный недостаток - ограниченный размер адресного пространства, так как для адресации к памяти большой емкости нужно «длинное» адресное поле. Однако более существенным несовершенством можно считать то, что адрес, указанный в команде, не может быть изменен в процессе вычислений (во всяком случае, такое изменение не рекомендуется). Это ограничивает возможности по произвольному размещению программы в памяти.

Плюсы – простота.

Существенный недостаток – ограниченный размер адресного пространства. Адрес в команде не может быть изменён в процессе вычислений -> ограничение на размещение программы в ОЗУ.

Косвенная регистровая адресация (КРА) представляет собой косвенную адресацию, где исполнительный адрес операнда хранится не в ячейке основной памяти, а в регистре процессора. Соответственно, адресное поле команды указывает не на ячейку памяти, а на регистр.

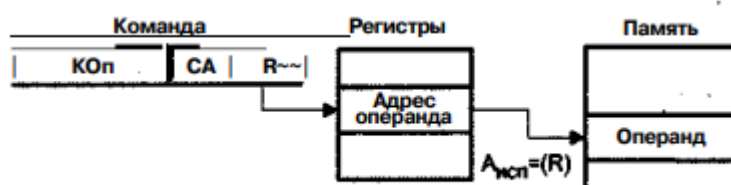


Рис. 2.51. Косвенная регистровая адресация

Достоинства и ограничения косвенной регистровой адресации те же, что и у обычной косвенной адресации (косвенный адрес в процессе выполнения программы можно изменять, упрощение обработки массивов и списков, а также передачи параметров подпрограмм), но благодаря тому, что косвенный адрес хранится не в памяти, а в регистре, для доступа к операнду требуется на одно обращение к памяти меньше.

Плюсы аналогичны обычной косвенной адресации – можно изменять адреса операндов в процессе вычислений. На одно обращение к памяти меньше.

Способ адресации – это способ формирования исполнительного адреса по адресу коду команды.

12. Способы адресации. Косвенная, автоинкрементная и автодекрементная адресация

Одним из путей преодоления проблем, свойственных прямой адресации, может служить прием, когда с помощью ограниченного адресного поля команды указывается адрес ячейки, в свою очередь, содержащей полноразрядный адрес операнда. Этот способ известен как косвенная адресация (КА).

При косвенной адресации содержимое адресного поля команды остается неизменным в то время как косвенный адрес в процессе выполнения программы можно изменять. Это позволяет проводить вычисления, когда адреса операндов заранее неизвестны и появляются лишь в процессе решения задачи. Дополнительно такой прием упрощает обработку массивов и списков, а также передачу параметров подпрограмм.



Рис. 2.49. Косвенная адресация

Плюсы – можно изменять адреса операндов в процессе вычислений.

Недостатком косвенной адресации является необходимость в двукратном обращении к памяти: сначала для извлечения адреса операнда, а затем для обращения к операнду. + задействуется лишняя ячейка памяти для хранения исполнительного адреса операнда.

Автоиндексирование с увеличением содержимого индексного регистра носит название автоинкрементной адресации. Автоиндексирование с уменьшением содержимого индексного регистра носит название автодекрементной адресации.

В большинстве ВМ увеличение или уменьшение содержимого индексного регистра до или после обращения к нему осуществляется автоматически как часть машинного цикла. Такой прием называется автоиндексированием.

При индексной адресации (ИА) подполе содержит адрес ячейки памяти, а регистр (указанный явно или неявно) — смещение относительно этого адреса. Как видно, этот способ адресации похож на базовую регистровую адресацию. Поскольку при индексной адресации в поле полноразрядный адрес ячейки памяти, играющий роль базы, длина этого поля больше, чем при базовой регистровой адресации.

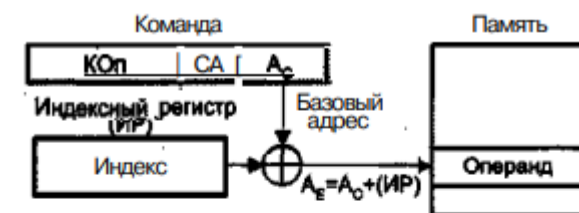


Рис. 2.87. Индексная адресация с индексным регистром

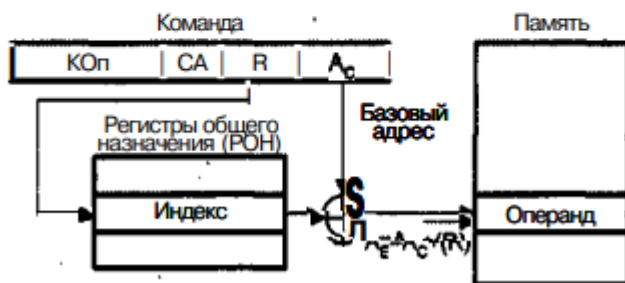


Рис. 2.58. Индексная адресация с использованием одного из РОП

Способ адресации – это способ формирования исполнительного адреса по адресному коду команды.

13.Способы адресации. Адресация со смещением (3 типа).

В адресации со смещением исполнительный адрес формируется в результате суммирования содержимого адресного поля содержимым одного или нескольких регистров процессора. Адресация со смещением предполагает, что адресная часть команды включает в себя как минимум одно поле. В нем содержится константа, смысл которой в разных вариантах может меняться. Константа может представлять собой некий базовый адрес, к которому добавляется хранящееся в регистре смещение.

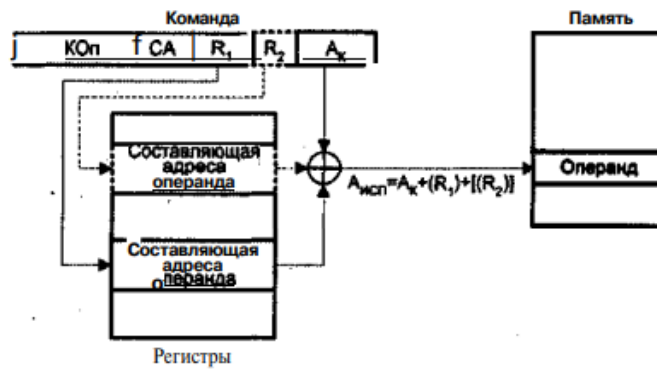


Рис. 2.52. Адресация со смещением

При относительной адресации (ОА) для получения исполнительного адреса операнда содержимое подполя A_x складывается с содержимым счетчика команд. Т.о, адресный код в команде представляет собой смещение относительно адреса текущей команды. Следует отметить, что в момент вычисления исполнительного адреса операнда в счетчике команд может уже быть сформирован адрес следующей команды, что нужно учитывать при выборе величины смещения. Обычно подполе как двоичное число в дополнительном коде.

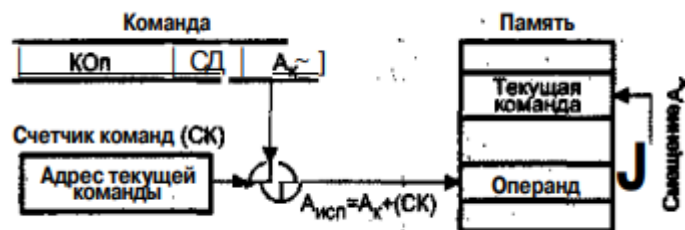


Рис. 2.53. Относительная адресация

В случае базовой регистровой адресации (БРА) регистр, называемый базовым, содержит полноразрядный адрес, а подполе A_c смещение относительно адреса. Ссылка на базовый регистр может быть явной или неявной. В некоторых ВМ имеется специальный базовый регистр и его использование является неявным. Базовую регистровую адресацию обычно используют для доступа к элементам массива, положение которого в памяти в процессе вычислений может меняться.

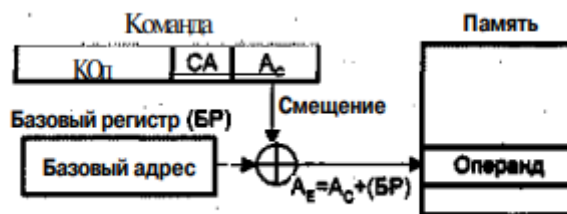


Рис. 2.54. Базовая регистровая адресация с базовым регистром

При индексной адресации (ИА) подполе содержит адрес ячейки памяти, а регистр (указанный явно или неявно) — смещение относительно этого адреса. Как видно, этот способ адресации похож на базовую регистровую адресацию. Поскольку при индексной адресации в поле полноразрядный адрес ячейки памяти, играющий роль базы, длина этого поля больше, чем при базовой регистровой адресации.

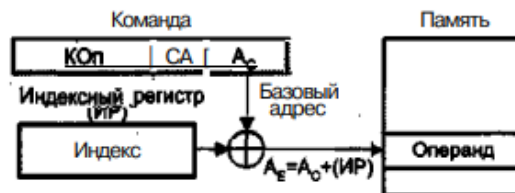


Рис. 2.87. Индексная адресация с индексным регистром

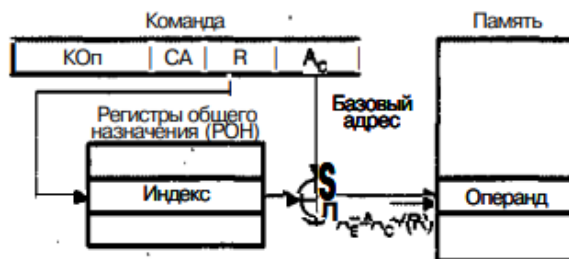


Рис. 2.58. Индексная адресация с использованием одного из РОН

Способ адресации – это способ формирования исполнительного адреса по адресному коду команды.

14. Способы адресации. Страничная и блочная адресации.

Страничная адресация (СТА) предполагает разбиение адресного пространства на страницы. Страница определяется своим начальным адресом, выступающим в качестве базы. Старшая часть этого адреса хранится в специальном регистре — регистре адреса страницы (РАС). В адресном коде команды указывается смещение внутри страницы, рассматриваемое как младшая часть исполнительного адреса.

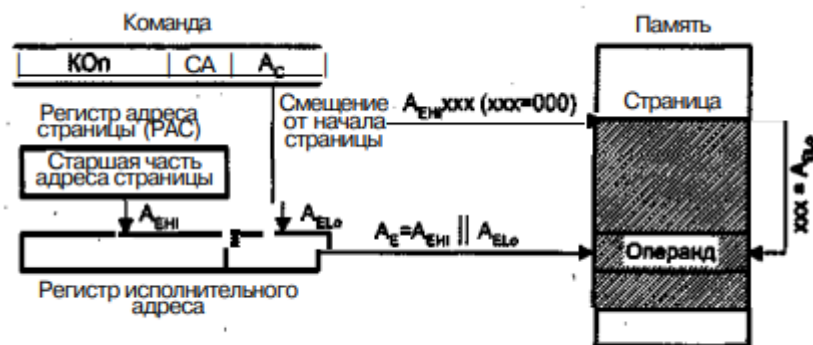


Рис. 2.59. Страничная адресация

Блочная адресация используется в командах, для которых единицей обработки служит блок данных, расположенных в последовательных ячейках памяти. Этот способ очень удобен при работе с внешними запоминающими устройствами и в операциях с векторами. Для описания блока обычно берется адрес ячейки, где хранится первый или последний элемент блока, и общее количество элементов блока, заданное числом байтов или ячеек. Вместо длины блока может использоваться специальный признак «конец блока», помещаемый за последним элементом блока. Удобен для работы с внешними ЗУ, операциях с векторами.

Способ адресации – это способ формирования исполнительного адреса по адресному коду команды.

15. Способы адресации. Стековая адресация

Стек представляет собой группу последовательно пронумерованных регистров (*аппаратурный стек*) или ячеек памяти, снабженных указателем стека (обычно регистром) (УС), в котором автоматически при записи и считывании устанавливается номер (адрес) последней занятой ячейки стека (вершины стека). При операции записи

заносимое в стек слово помещается в следующую по порядку свободную ячейку стека, а при считывании из стека извлекается последнее поступившее в него слово. Таким образом, в стеке реализуется правило «*последний пришел - первый ушел*».

Указанное правило при обращении к стеку реализуется автоматически, и поэтому при операциях со стеком возможно безадресное задание операнда — команда не содержит адреса ячейки стека, но содержит адрес (или он подразумевается) ячейки памяти или регистра, откуда слово передается в стек или куда помещается из стека.

При выполнении команды передачи в стек слова из регистра или ячейки ОП сначала указатель стека увеличивается на 1 (в *перевернутом стеке* уменьшается на 1), а затем слово помещается в ячейку стека, указываемую УС. При команде загрузки из стека регистра или ячейки памяти сначала слово извлекается из вершины стека, а затем указатель стека уменьшается на 1 (в перевернутом стеке увеличивается на 1). Как это ни кажется на первый взгляд удивительным, но при соответствующем расположении операндов в стеке можно вычислять выражения полностью безадресными командами, указывающими только вид операции. Такая команда извлекает из стека в соответствии с кодом операции один или два операнда, выполняет над ними предписанную операцию и заносит результат в стек.

Способ адресации – это способ формирования исполнительного адреса по адресному коду команды.

Используется в ВМ со стековой архитектурой.

Стековый доступ к памяти происходит в большинстве процессоров в следующих ситуациях:

1) При выполнении команд стекового доступа: "поместить в стек" push или "извлечь из стека" pop.

2) При выполнении команды вызова подпрограммы и при возврате из нее.

3) При входе в прерывание и возврате из него.

Положение стека в адресуемой памяти определяется содержимым указателя стека.

Содержимое указателя стека используется как адрес операнда-приемника при записи или как адрес операнда-источника при считывании из стека.

При обращении к стеку автоматически модифицируется и содержимое указателя стека, чтобы обеспечить следующее обращение к очередной ячейке стека.

Обращение к стеку - косвенно-регистровая адресация через указатель стека с автоиндексацией.

16. Транзакция. Целевое назначение шин.

Взаимосвязь частей ВМ и ее «общение» с внешним миром обеспечиваются системой шин. Большинство машин содержат несколько различных шин, каждая из которых оптимизирована под определенный вид коммуникаций. Часть шин скрыта внутри интегральных микросхем или доступна только в пределах печатной платы. Некоторые шины имеют доступные извне точки, с тем чтобы к ним легко можно было подключить дополнительные устройства, причем большинство таких шин не просто доступны, но и отвечают определенным стандартам, что позволяет подсоединять к шине устройства различных производителей.

Шину образует набор коммуникационных линий, каждая из которых способна передавать сигналы, представляющие двоичные цифры 1 и 0. По линии может пересылаться развернутая во времени последовательность таких сигналов. При совместном использовании несколько линий могут обеспечить одновременную (параллельную) передачу двоичных чисел.

Этим стремятся подчеркнуть, что шина ведет себя как магистраль, способная обеспечить всевозможные виды трафика

Операции на шине называют транзакциями. Основные виды транзакций - транзакции чтения и транзакции записи. Если в обмене участвует устройство

ввода/вывода, можно говорить о транзакциях ввода и вывода, по сути эквивалентных транзакциям чтения и записи соответственно. Шинная транзакция включает в себя две части: посылку адреса и прием (или посылку) данных.

Целевое назначение шин:

- Шины «процессор-память»
- Шины ввода/вывода
- Системные шины

Шина «процессор-память»

Связь между ЦП и ОП (основной памятью)- Front-Side Bus (FSB).

Шина для связи процессора с -памятью второго уровня – Back Side Bus (BSB).

Максимизация пропускной способности и минимизация длины шины.

Шины ввода/вывода

Соединение между ЦП и устройствами В/В-да.

Многообразие устройств -> стандартизация и унификация шин.

За исключением видеосистем – шины в/в не особо быстры.

Минимизация конструктива шины (разъёмов и т.п.). Меньше линий, но больше длина (SCSI).

Системная шина

Физ. и лог. объединение всех устр-в ВМ (backplane bus). Обычно несколько сот линий.

17. Иерархия шин.

Если к шине подключено большое число устройств, ее пропускная способность падает, поскольку слишком частая передача прав управления шиной от одного устройства к другому приводит к ощутимым задержкам. => во многих ВМ предпочтение отдается использованию нескольких шин, образующих определенную иерархию.

Вычислительная машина с одной шиной

В структурах взаимосвязей с одной шиной имеется одна системная шина, обеспечивающая обмен информацией между процессором и памятью, а также между УВВ, с одной стороны, и процессором либо памятью - с другой (рис. 4.5).



Рис. 4.5. Структура взаимосвязей с одной шиной

Для такого подхода характерны простота и низкая стоимость. Однако одношинная организация не в состоянии обеспечить высокие интенсивность и скорость транзакций, причем «узким местом» становится именно шина.

Вычислительная машина с двумя видами шин

Хотя контроллеры устройств ввода/вывода (УВВ) могут быть подсоединены непосредственно к системной шине, больший эффект достигается применением одной или нескольких шин ввода/вывода (рис. 4.6). УВВ подключаются к шинам ввода/вывода, которые берут на себя основной трафик, не связанный с выходом на процессор или память. *Адаптеры шин* обеспечивают буферизацию данных при их пересылке между системной шиной и контроллерами УВВ. Это позволяет ВМ поддерживать работу множества устройств ввода/вывода и одновременно «развязать» обмен информацией по тракту процессор-память и обмен информацией с УВВ.



Рис. 4.6. Структура взаимосвязей с двумя видами шин

Вычислительная машина с тремя видами шин

Для подключения быстродействующих периферийных устройств в систему шин может быть добавлена высокоскоростная шина расширения (рис. 4.7).

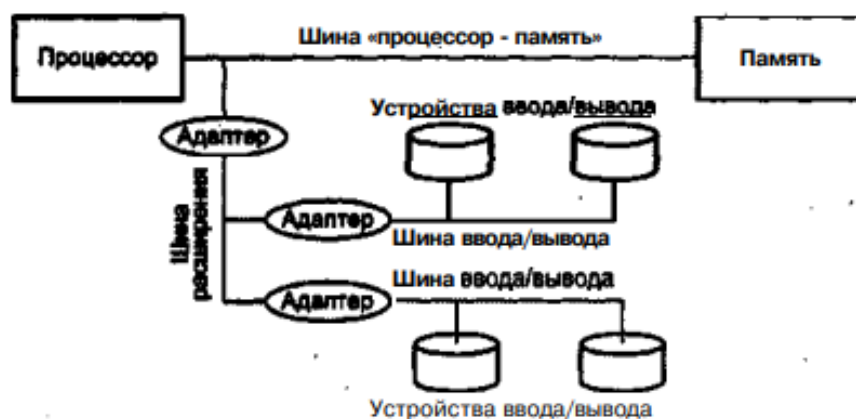
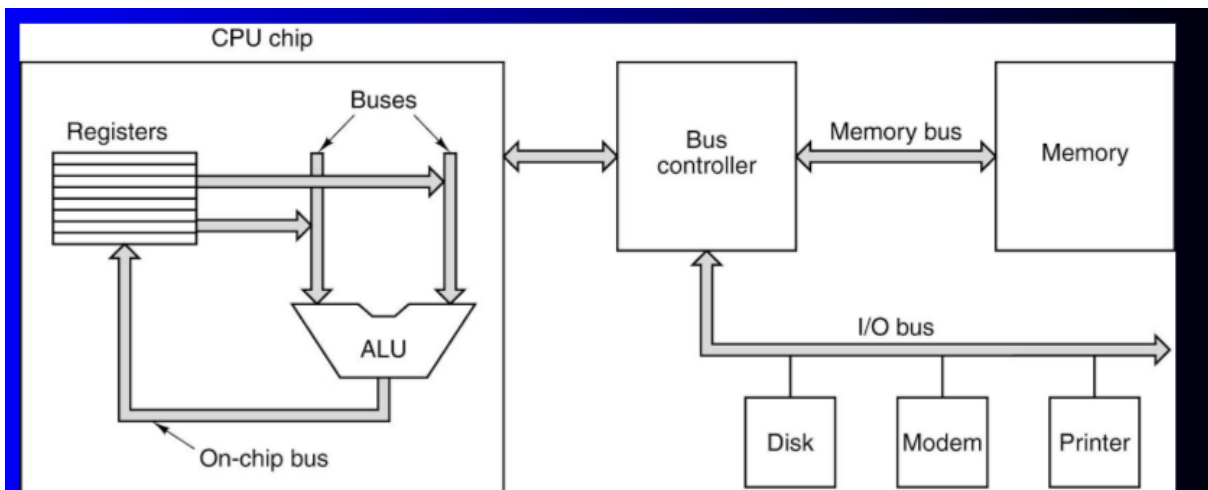


Рис. 4.7. Структура взаимосвязей с тремя видами шин

Шины ввода/вывода подключаются к шине расширения, а уже с нее через адаптер к шине «процессор-память». Схема еще более снижает нагрузку на шину «процессор-память». Такую организацию шин называют *архитектурой с «пристройкой»* (mezzanine architecture).



A computer system with multiple buses.

Master	Slave	Example
CPU	Memory	Fetching instructions and data
CPU	I/O device	Initiating data transfer
CPU	Coprocessor	CPU handing instruction off to coprocessor
I/O	Memory	DMA (Direct Memory Access)
Coprocessor	CPU	Coprocessor fetching operands from CPU

Examples of bus masters and slaves.

18. Распределение линий шины управления.

Шина управления (ШУ):

Линии сигналов управления транзакциями—

- тип выполняемой транзакции (чт. или запись),
- количество байтов, передаваемых по ШД, если часть слова – то какие байты.
- какой тип адреса на ША
- какой протокол передачи должен использоваться

Всего на данную группу - от 2 до 8 линий.

Линии информации состояния (статуса) –

от одной до четырёх линий для передачи ведомым информации ведущему.

Линии арбитража – от 3-х до 11 линий.

Линии прерывания – запросы от ведомых к ведущему на обслуживание (обычно одна две-линии) + доп. Арбитраж (если не используются линии пред. группы).

Линии для организации последовательных локальных сетей – обычно от 1 до 4 линий (последовательная передача значительно медленнее параллельной – выгоднее добавить пару линий чем загружать основные), иногда могут заменить ША и ШД, служить для реализации спец. функций – обработка прерываний или сортировка приоритетов задач.

Линии позиционного кода – (от 4 до 5 линий) – для передачи уникального позиционного кода дочерних плат.

Линии тактирования и синхронизации – от 2 до 6 линий – в зависимости от протокола (асинхронный, синхронный).

Линии питания и заземления – от 2 до 20.

19. Алгоритмы смены приоритетов при арбитраже шин.

Несколько ведущих на одной шине -> конфликт интересов. Значит, необходимо присвоение приоритетов.

Статический приоритет - устройства с высшим приоритетом могут полностью блокировать шину.

Динамический - шанс есть у каждого.

Наибольшее распространение получили следующие алгоритмы динамического изменения приоритетов:

- простая циклическая смена приоритетов;
- циклическая смена приоритетов с учетом последнего запроса;
- смена приоритетов по случайному закону;
- схема равных приоритетов;
- алгоритм наиболее давнего использования;
- алгоритм очереди (первым пришел — первым обслужен);
- алгоритм фиксированного кванта времени.

В алгоритме простой циклической смены приоритетов после каждого цикла арбитража все приоритеты понижаются на один уровень, при этом устройство, имевшее ранее низший уровень приоритета, получает наивысший приоритет.

В схеме циклической смены приоритетов с учетом последнего запроса все возможные запросы упорядочиваются в виде циклического списка. После обработки очередного запроса обслуженному ведущему назначается низший уровень приоритета.

Следующее в списке устройство получает наивысший приоритет, а остальным устройствам приоритеты назначаются в убывающем порядке, согласно их следованию в циклическом списке.

В обеих схемах циклической смены приоритетов каждому ведущему обеспечивается шанс получить шину в свое распоряжение, однако большее распространение получил второй алгоритм.

При смене приоритетов по случайному закону после очередного цикла арбитража с помощью генератора псевдослучайных чисел каждому ведущему присваивается случайное значение уровня приоритета.

В схеме равных приоритетов при поступлении к арбитру нескольких запросов каждый из них имеет равные шансы на обслуживание. Возможный конфликт разрешается арбитром. Такая схема принята в асинхронных системах.

В алгоритме наиболее давнего использования (LRU, Least Recently Used) после каждого цикла арбитража наивысший приоритет присваивается ведущему, который дольше чем другие не использовал шину.

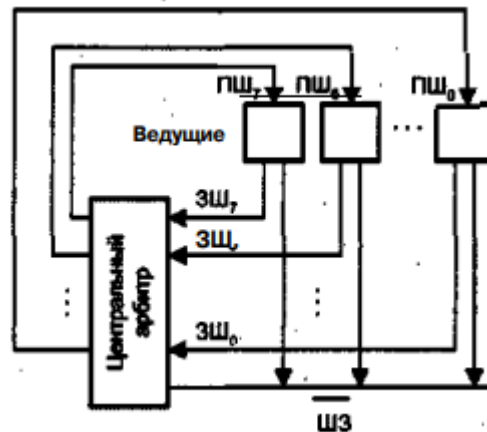
В алгоритме очереди запросы обслуживаются в порядке очереди, образовавшейся к моменту начала цикла арбитража. Сначала обслуживается первый запрос в очереди, то есть запрос, поступивший раньше остальных. Аппаратурная реализация алгоритма связана с определенными сложностями, поэтому используется он редко.

В алгоритме фиксированного кванта времени каждому ведущему для захвата шины в течение цикла арбитража выделяется определенный квант времени. Если ведущий в этот момент не нуждается в шине, выделенный ему квант остается не использованным. Такой метод наиболее подходит для шин с синхронным протоколом.

20. Разновидности централизованного арбитража

При централизованном арбитраже в системе имеется специальное устройство — центральный арбитр, - ответственное за предоставление доступа к шине только одному из запросивших ведущих. Это устройство, называемое иногда центральным контроллером шины, может быть самостоятельным модулем или частью ЦП. Наличие на шине только одного арбитра означает, что в централизованной схеме имеется единственная точка отказа. В зависимости от того, каким образом ведущие устройства подключены к центральному арбитру возможные схемы централизованного арбитража можно подразделить на параллельные и последовательные.

В параллельном варианте центральный арбитр связан с каждым потенциальным ведущим индивидуальными двухпроводными трактами. Поскольку запросы к центральному арбитру могут поступать независимо и параллельно, данный вид арбитража называют централизованным параллельным арбитражем или централизованным арбитражем независимых запросов.



Второй вид централизованного арбитража известен как централизованный последовательный арбитраж. В последовательных схемах для выделения запроса с наивысшим приоритетом используется один из сигналов, поочередно проходящий через цепочку ведущих, чем и объясняется другое название - цепочечный или гирляндный арбитраж.

В зависимости от того, какой из сигналов используется для целей арбитража, различают три основных типа схем цепочечного арбитража: с цепочкой для сигнала предоставления шины, с цепочкой для сигнала запроса шины (ЗШ) и с цепочкой для дополнительного сигнала разрешения (ПШ). Наиболее распространена схема цепочки для сигнала ПШ.

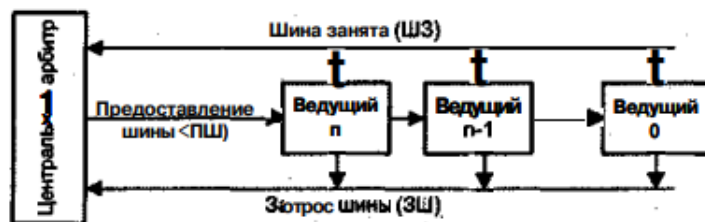


Рис. 4.12. Централизованный последовательный арбитраж с цепочкой для сигнала предоставления шины

Еще раз отметим, что очередной ведущий не может приступить к управлению шиной до момента ее освобождения. Центральный арбитр не должен формировать сигнал ПШ вплоть до этого момента.

Цепочечная реализация предполагает статическое распределение приоритетов. Наивысший приоритет имеет ближайшее к арбитру ведущее устройство (устройство, на которое арбитр дал сигнал ПШ). Далее приоритеты ведущих в цепочке последовательно понижаются.

Основное достоинство цепочечного арбитража заключается в простоте реализации и в малом количестве используемых линий. Последовательные схемы арбитража позволяют легко наращивать число устройств, подключаемых к шине.

Схеме тем не менее присущи существенные недостатки. Прежде всего, последовательное прохождение сигнала по цепочке замедляет арбитраж, причем время арбитража растет пропорционально длине цепочки. Статическое распределение приоритетов может привести к полному блокированию устройств с низким уровнем приоритета (расположенных в конце цепочки). Наконец, как и параллельный вариант, централизованный последовательный арбитраж не очень удобен в плане диагностики работы шины

21. Разновидности децентрализованного арбитража.

Кольцевая схема с циклической сменой приоритетов.

Переход к следующему ведущему – со сменой приоритетов. Текущий ведущий в след. цикле – наименьший приоритет, его сосед справа – наивысший, остальные – на 1 меньше, чем у соседа слева.

Циклическая смена с учётом последнего запроса.



Слайд 36

Текущий ведущий генерирует ПШ, который проходит через все схемы ЛА. Если имеется ещё один ведущий, который просит шину (ЗШ), то его ЛА не пропускает ПШ к другим устройствам. Текущий ведущий, «потеряв» ПШ на своём входе, должен при первой возможности освободить шину. Запрашивающий – её займет.

Чаще встречается иная схема децентр-го арбитража. Арбитражные линии и устройства по схеме «монтажное ИЛИ». Каждому ведущему – уникальный номер (уровень приоритета). Запрашивающие шину выставляют на арб. линии свой номер. Каждый из запрашивающих, обнаружив на линиях номер с более высоким приоритетом снимает младшие биты своего номера. В итоге (может понадобится несколько итераций) на линии остаётся только номер с наиболее высоким приоритетом. Ведущий, распознавший на линиях свой номер – захватывает шину. Способ называется – *распределённый арбитраж с самостоятельным выбором*. Модуль равнодоступности – последний ведущий не может участвовать в след. цикле арбитража. Шины – FutureBus, NuBus, MMultiBus, FastBus.

22. Понятие протокола шины

Протокол шины

Сигналы на шине – перекося – прежде чем реагировать, все ведомые должны знать с какого момента поступивший адрес можно считать достоверным.

С данными ещё сложнее – могут передаваться в двух направлениях. При чтении – доп. задержка на выдачу ведомым необходимых ведущему данных на шину.

Метод информирования о достоверности адреса, данных, управляющей информации и информации состояния – протокол шины.

Два класса протоколов – **синхронный** (все сигналы привязаны к импульсам ГТИ) и **асинхронный** – для каждой группы сигналов свой сигнал подтверждения достоверности.

Протокол шины

Синхронные шины: (доп. неудобства) ведущий не знает, что ответил ведомый, ведущий должен работать со скоростью самого медленного из устр-в, участвующих в пересылке данных.

Асинхронные шины: позволяют избегать указанных выше недостатков.

Начало очередного события определяется не ТИ, а окончанием предыдущего. Каждая совокупность сигналов, помещаемых на шину, сопровождается синхронизирующим сигналом (стробом).

Синхросигналы от ведомого – *квитирующие сигналы* (handshakes) или *подтверждения сообщений* (acknowledges).

23. Последовательность действий в процедуре квитирования установления связи.

Квитирование установления связи

Начальный обмен служебной информацией между двумя системами перед началом и во время передачи данных, выполняемый для обеспечения правильности передачи данных.

Метод квитирования установления связи (например, XON/XOF) является частью полного протокола передачи данных.

Последовательный (асинхронный) протокол передачи данных может включать в себя метод квитирования установления связи, установку скорости передачи, контроля четности, количества битов данных и числа стоповых битов.

Концепция квитирования

В рамках соединения правильность передачи каждого сегмента должна подтверждаться квитанцией получателя. *Квитирование* - это один из традиционных методов обеспечения надежной связи. Идея квитирования состоит в следующем.

Для того, чтобы можно было организовать повторную передачу искаженных данных отправитель нумерует отправляемые единицы передаваемых данных (далее для простоты называемые кадрами). Для каждого кадра отправитель ожидает от приемника так называемую положительную квитанцию - служебное сообщение, извещающее о том, что исходный кадр был получен и данные в нем оказались корректными. Время этого ожидания ограничено - при отправке каждого кадра передатчик запускает таймер, и если по его истечению положительная квитанция не получена, то кадр считается утерянным. В некоторых протоколах приемник, в случае получения кадра с искаженными данными должен отправить отрицательную квитанцию - явное указание того, что данный кадр нужно передать повторно.

Существуют два подхода к организации процесса обмена положительными и отрицательными квитанциями: с простоями и с организацией "окна".

http://citforum.ru/nets/ip/glava_6.shtml

ссылка - сайт со второго скрина(описываются два подхода, если посчитаешь, что не хватит).

Квитирование широко применяется в параллельных интерфейсах (в том же LPT-порте, шинах расширения), для чего используются специальные интерфейсные линии.

24. Синхронные и асинхронные шины, достоинства и недостатки.

Синхронные шины

Синхронные шины: следует добиваться, чтобы ТИ доставлялись к каждому разъёму шины практически одновременно.

ТИ должен выбираться так, чтобы любой сигнал на шине мог быть доставлен в любую её точку до завершения тактового периода -> чем короче шина, тем м.б. выше частота ТИ.

Синхронные протоколы – проще, требуют меньше сигн. линий, но менее гибки – привязаны к конкретной тактовой частоте (уровню технологий). Из-за проблемы перекоса синхросигналов **синхронн**ые шины не могут быть длинными.

Обычно синхр. шины – «процессор-память».

Асинхронные шины

Скорость асинхронной пересылки определяется ведомым -> шины самосинхронизирующиеся, поэтому успешно могут применяться и новые и старые устройства. Плата – некоторое увеличение сложности аппаратуры.

Квитирование не всегда производится в полном объёме. Иногда транзакция не может быть завершена стандартным образом (например обращение по несуществующему адресу памяти) – ведомое не ответит подтверждающим сигналом. Выход из ситуации – схема тайм-аута.

Тайм-ауты из-за отказа оборудования – редки, но тайм-ауты по адресу – часты (например, проверка присутствия устройств на шине) -> малые значения тайм-аута и очень быстрые схемы декод-я адреса.

Асинхронные обычно шины ввода/вывода.

Слайд 48

Синхронные vs асинхронные

Ранее в ВМ преобладали асинхронные шины, в последних разработках – всё чаще синхронные.

В настоящее время синхронные чуть быстрее асинхронных и близки к макс. значениям распространения сигналов. Возможное ускорение асинхронных даст лишь незначительный выигрыш по сравн. с синхр-ми.

Перекося сигналов – в синхронных уже заложен в такт. частоту, в асинхронных должен учитываться в каждой транзакции и каждым устройством.

Проблема *метастабильного состояния* триггеров ведомых в асинхронных решается легче – спец. Схемы определяют подобные ситуации и асинхр. система ожидает пока состояние не станет стабильным. В синхронном протоколе – синхронизация триггеров ТИ и/или двухтактные триггеры.

Слайд 49

Методы повышения эффективности шин

Приёмы повышения производительности шин:

- Пакетный режим
- Конвейеризация транзакций
- Расщепление транзакций

Пакетный или блочный режим (burst mode) – как для асинхр., так и для синхр-х. Один адресный цикл сопровождается множественными циклами данных (либо чтения либо записи, но не смешанных!). Пакет данных передаётся без указания адресов данных внутри пакета -> увеличение адреса производится автоматически как у ведущего, так и у ведомого. Длина пакета может достигать до 1024 байт.

Методы повышения эффективности шин

Пакетный или блочный режим (burst mode) – наиболее частый вариант – передача пакетов из четырёх байт.



В асинхронных режимах – дополнительный выигрыш: разрешение отправителю начинать следующий цикл данных, не ожидая подтверждения от получателя -> экономия времени прохождения сигнала от отправителя к приёмнику, время подтверждения + внутренние задержки в ведущем и ведомом устр-вах и восстановления шины после квитирования.

Пример шины с пакетным режимом - FutureBus+.

Слайд 4

Методы повышения эффективности шин

Конвейеризация транзакций: очередной элемент данных может быть послан устройством А, до того, как устройство В завершит считывание предыдущего элемента.



Данные должны оставаться стабильными в течение времени $t_{ст}$ (стабилизации) + $t_{уд}$ (удержания), только после этого – смена элемента данных.

Максимальная скорость передачи – $1/(t_{ст} + t_{уд})$

Методы повышения эффективности шин

Протокол с расщеплением транзакций (соединения/разъединения) или протокол с коммутацией пакетов (packet-switched): обеспечивает преимущество на транзакциях чтения.

В классическом варианте – любая транзакция непрерывна.

В протоколе с расщеплением – две шины ША и ШД обязательно разделены. Ведущий выставляет адрес и требование данных. После чего шина свободна. Когда память подготовит данные она становится ведущим и запрашивает доступ к шине, после чего пересылает данные запрашивающему. Соответственно, пока память готовит данные к отправке шина свободна и может использоваться другими ведущими...

Методы повышения эффективности шин

Протокол с расщеплением транзакций – на такой шине имеет место поток запросов и поток откликов.

Контроллер памяти обычно проектируется так, чтобы обеспечивать буферизацию множественных потоков.

Возможен вариант организации шины с ответами на запросы в произвольном порядке (требуется вводить теги на данные).

Плюс – более эффективное использование шины.

Минусы – дополнительные подтверждения при запросе и отклике, дополнительные затраты на аппаратуру.

Для любой шины с расщеплением, существует предельное число одновременно обслуживаемых запросов.

26. Чем характеризуются ЗУ?

Память предназначена для фиксации, хранения и выдачи информации в процессе работы ЭВМ. Процессы чтения и записи информации определяются как процессы обращения к запоминающему устройству (ЗУ).

Перечень основных характеристик, которые необходимо учитывать, рассматривая конкретный вид ЗУ, включает в себя:

- месторасположения (процессорные, внутренние и внешние);
- емкость (число бит либо байт, которое может храниться в ЗУ);
- единицу пересылки (основной памяти (ОП) единица пересылки определяется шириной шины данных, т.е. количеством битов, передаваемых по линиям шины параллельно. Обычно единица пересылки равна длине слова, но не обязательно. Применительно к внешней памяти данные часто передаются единицами, превышающими размер слова, и такие единицы называются блоками);
- метод доступа (• Последовательный доступ. ЗУ с последовательным доступом ориентировано на хранение информации в виде последовательности блоков данных, называемых записями. Для доступа к нужному элементу (слову или байту) необходимо прочитать все предшествующие ему данные. • Прямой доступ. Каждая запись имеет уникальный адрес, отражающий ее физическое размещение на носителе информации. Обращение осуществляется как адресный доступ к началу записи, с последующим последовательным доступом к определенной единице информации внутри записи. • Произвольный доступ. Каждая ячейка памяти имеет уникальный физический адрес. Обращение к любой ячейке занимает одно и то же время и может вестись в произвольной очередности. • Ассоциативный доступ. Этот вид доступа позволяет выполнять поиск ячеек, содержащих такую информацию, в которой значение отдельных битов совпадает с состоянием одноименных битов в заданном образце);
- быстродействие (• Время доступа. Для памяти с произвольным доступом оно соответствует интервалу времени от момента поступления адреса до момента, когда данные заносятся в память или становятся доступными. В ЗУ с подвижным носителем информации - это время, затрачиваемое на установку головки записи/считывания (или носителя) в нужную позицию. • Длительность цикла памяти или период обращения. Понятие применяется к памяти с произвольным доступом, для которой оно означает минимальное время между двумя последовательными обращениями к памяти. Период обращения включает в себя время доступа плюс некоторое дополнительное время. • Скорость передачи - скорость, с которой данные могут передаваться в память или из нее);
- физический тип (полупроводниковая память, память с магнитным носителем информации, память с оптическим носителем);
- физические особенности (энергозависимость, приводит ли считывание информации к ее разрушению);
- стоимость (отношением общей стоимости ЗУ к его емкости в битах, т.е. стоимостью хранения одного бита информации.).

27. Основная проблема памяти и способ её решения.

Память часто называют «узким местом» фон-неймановских ВМ из-за ее серьезного отставания по быстродействию от процессоров, причем разрыв этот неуклонно увеличивается. Так, если производительность процессоров ежегодно возрастает вдвое примерно каждые 1,5 года, то для микросхем памяти прирост быстродействия не превышает 9% в год (удвоение за 10 лет), что выражается в увеличении разрыва в быстродействии между процессором и памятью приблизительно на 50% в год.

Если проанализировать используемые в настоящее время типы ЗУ, выявляется следующая закономерность:

- чем меньше время доступа, тем выше стоимость хранения бита;

- чем больше емкость, тем ниже стоимость хранения бита, но больше время доступа.

При создании системы памяти постоянно приходится решать задачу обеспечения требуемой емкости и высокого быстродействия за приемлемую цену. Наиболее распространенным подходом здесь является построение системы памяти ВМ по иерархическому принципу. Иерархическая память состоит из ЗУ различных типов, которые, в зависимости от характеристик, относят к определенному уровню иерархии. Более высокий уровень меньше по емкости, быстрее и имеет большую стоимость в пересчете на бит, чем более низкий уровень. Уровни иерархии взаимосвязаны: все данные на одном уровне могут быть также найдены на более низком уровне, и все данные на этом более низком уровне могут быть найдены на следующем нижележащем уровне и т. д.

Четыре верхних уровня иерархии образуют внутреннюю память ВМ, а все нижние уровни - это внешняя или вторичная память.

По мере движения вниз по иерархической структуре:

1. Уменьшается соотношение «стоимость/бит».
2. Возрастает емкость.
3. Растет время доступа.
4. Уменьшается частота обращения к памяти со стороны центрального процессора

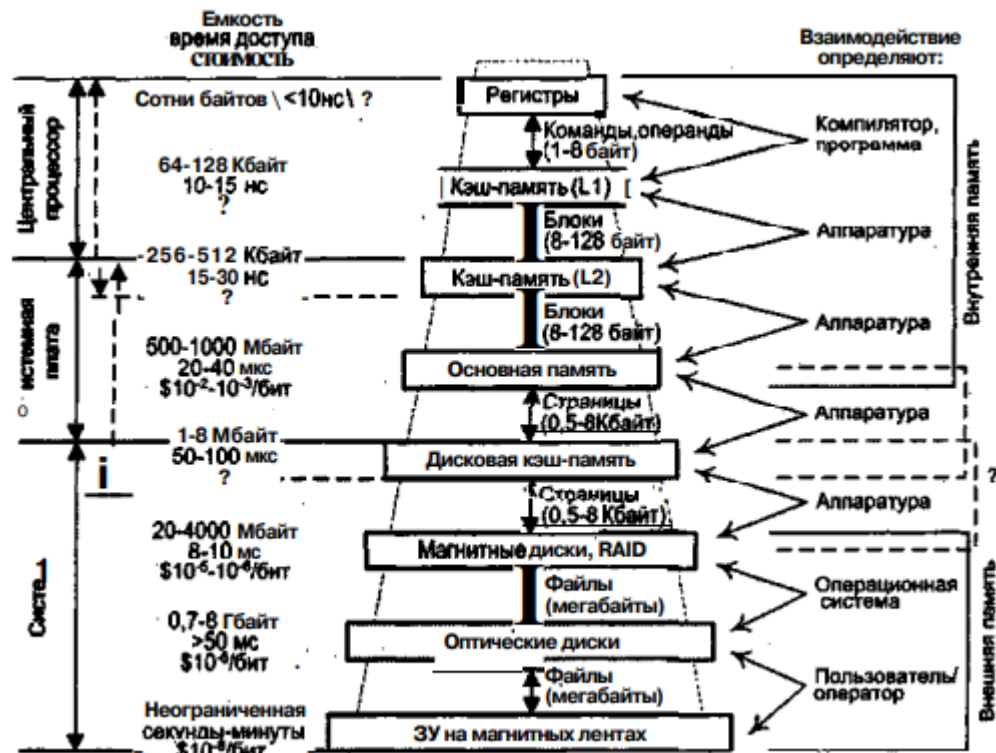


Рис. 5.1. Иерархия запоминающих устройств

28. Система ввода-вывода, понятие и задачи.

Система ввода/вывода должна обеспечивать:

- построение вычислительной системы с переменным составом оборудования
- реализацию параллельной работы процессора над программой и выполнение им процедур ввода/вывода
- простоту и стандартность операций ввода/вывода, обеспечивая независимость программ от особенностей ПУ
- автоматическое распознавание и обслуживание ПУ

Способы обмена:

- программный (не форсированный)

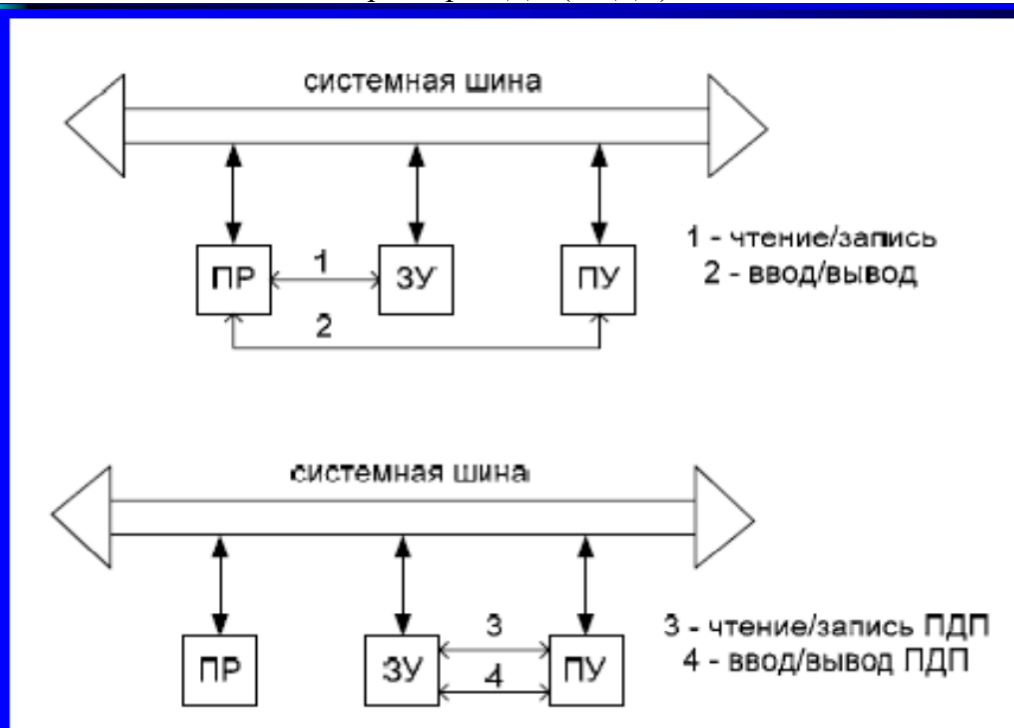
- по прерыванию (форсированный)
- при прямом доступе к памяти (аппаратный)

Программно управляемый обмен данными осуществляется по инициативе процессора и под его управлением. Данные между процессором и памятью, памятью и внешним устройством пересылаются через процессор (регистр АХ). При таком обмене процессор на всё время его выполнения отвлекается от других команд – снижается производительность ЭВМ. Пересылая блок данных ЦП выполняет много вспомогательных действий:

- буферизация данных
- преобразование форматов данных
- подсчёт количества передач
- учёт ограничений по счётчику
- формирование адреса памяти
- обнаружение конечных символов при передаче цепочек

В результате скорость передачи данных снижается в десятки раз

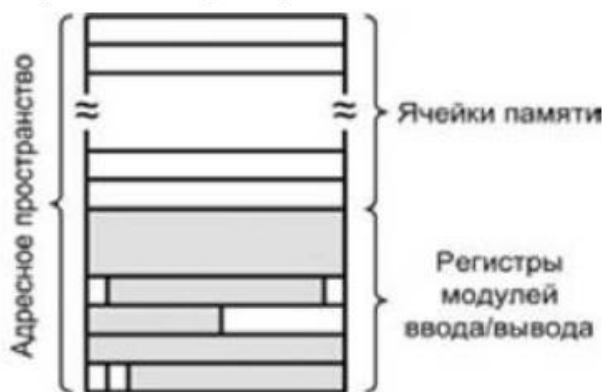
Форсировать скорость передачи данных с ПУ можно используя обмен по прерываниям. В этом случае обмен осуществляется по требованию ПУ, когда оно готово к нему, а идентификацию устройства производить по вектору прерывания, переходя соответственно к программе обслуживания ПУ. Это всё повышает скорость передачи, но принцип программно управляемого обмена остаётся неизменным. Максимально быстрый режим обмена данными осуществляется с помощью ПДП (прямой доступ к памяти). ПДП происходит без участия процессора, данные перекачиваются в ПУ непосредственно из памяти (минуя процессор) и наоборот (аппаратная передача). Этот режим реализуется с помощью дополнительного контроллера ПДП (КПДП)



Прямой доступ к памяти (мимо процессора)

29. Совмещенное адресное пространство СВВ, достоинства и недостатки.

При совмещении адресного пространства для адресации модулей ввода/вывода отводится определенная область адресов. Обычно все операции с модулем ввода/вывода осуществляются с использованием входящих в него внутренних регистров: управления, состояния, данных. Фактически процедура ввода/вывода сводится к записи информации в одни регистры МВВ и считыванию ее из других регистров. Это позволяет рассматривать регистры МВВ как ячейки основной памяти и работать с ними с помощью обычных команд обращения к памяти, при этом в системе команд ВМ вообще могут отсутствовать специальные команды ввода и вывода. Так, модификацию регистров МВВ можно производить непосредственно с помощью арифметических и логических команд. Адреса регистрам МВВ назначаются в области адресного пространства памяти, отведенной под систему ввода/вывода.



Достоинства совмещенного адресного пространства:

- расширение набора команд для обращения к внешним устройствам, что позволяет сократить длину программы и повысить быстродействие;
- значительное увеличение количества подключаемых внешних устройств;
- возможность внепроцессорного обмена данными между внешними устройствами, если в системе команд есть команды пересылки между ячейками памяти;
- возможность обмена информацией не только с аккумулятором, но и с любым регистром центрального процессора.

Недостатки совмещенного адресного пространства:

- сокращение области адресного пространства памяти;
- усложнение декодирующих схем адресов в СВВ;
- трудности распознавания операций передачи информации при вводе/выводе среди других операций. Сложности в чтении и отладке программы, в которой простые команды вызывают выполнение сложных операций ввода/вывода;
- трудности при построении СВВ на простых модулях ввода/вывода: сигналы управления не смогут координировать сложную процедуру ввода/вывода. Поэтому МВВ часто должны генерировать дополнительные сигналы под управлением программы.

30. Конфликты в конвейере команд. Типы рисков.

Идея конвейера команд была предложена в 1956 году академиком С. А. Лебедевым. Как известно, цикл команды представляет собой последовательность этапов. Возложив реализацию каждого из них на самостоятельное устройство и последовательно соединив такие устройства, мы получим классическую схему конвейера команд.

Для иллюстрации воспользуемся примером. Выделим в цикле команды шесть этапов:

1. Выборка команды. Чтение команды из памяти и занесение ее в регистр.

2. Декодирование команды. Определение кода операции и способов адресации операндов.

3. Вычисление адресов операндов (ВА). Вычисление исполнительных адресов каждого из операндов в соответствии с указанным в команде способом их адресаций.

4. Выборка операндов (ВО). Извлечение операндов из памяти. Эта операция не нужна для операндов, находящихся в регистрах.

5. Исполнение команды (ИК). Исполнение указанной операции.

6. Запись результата (ЗР), Занесение результата в память.

Конфликтные ситуации в конвейере принято обозначать термином риск (hazard), а обусловлены они могут быть тремя причинами:

- попыткой нескольких команд одновременно обратиться к одному и тому же ресурсу ВМ, чаще всего память (структурный риск) (подобных конфликтов частично удается избежать за счет модульного построения памяти и использования кэш-памяти - имеется вероятность того, что команды будут обращаться либо к разным модулям ОП, либо одна из них станет обращаться к основной памяти, а другая - к кэш-памяти. С этих позиций выгоднее разделять кэш-память команд и кэш-память данных.);

- взаимосвязью команд по данным (риск по данным) (- «Чтение после записи» (ЧПЗ): команда j читает x до того, как команда i успела записать новое значение x, т.е. j ошибочно получит старое значение x вместо нового. - «Запись после чтения» (ЗПЧ): команда j записывает новое значение x до того, как команда i успела прочитать x, т.е. команда i ошибочно получит новое значение x вместо старого. - «Запись после записи» (ЗПЗ): команда j записывает новое значение x прежде, чем команда i успела записать в качестве x свое значение, т.е. x ошибочно содержит i-е значение x вместо j- го);

- неоднозначностью при выборке следующей команды в случае команд перехода (риск по управлению) (любые переходы кроме jmp).

Конфликты в конвейере команд

Устранение конфликтов – программно и аппаратно:

Программно – компиляция оптимизирующим компилятором (в спорных случаях ставится NOP).

Аппаратно – пузырьёк и ускоренное продвижение информации (Forwarding).

Пузырёк – приостановка на несколько тактов команды j, чтобы команда i успела завершиться или пройти конфликтную ступень.

Задерживаются и все команды, следующие за командой j. Иногда приостанавливают j, но не задерживают остальные команды – эффективно, но требует сложной реализации конвейера.

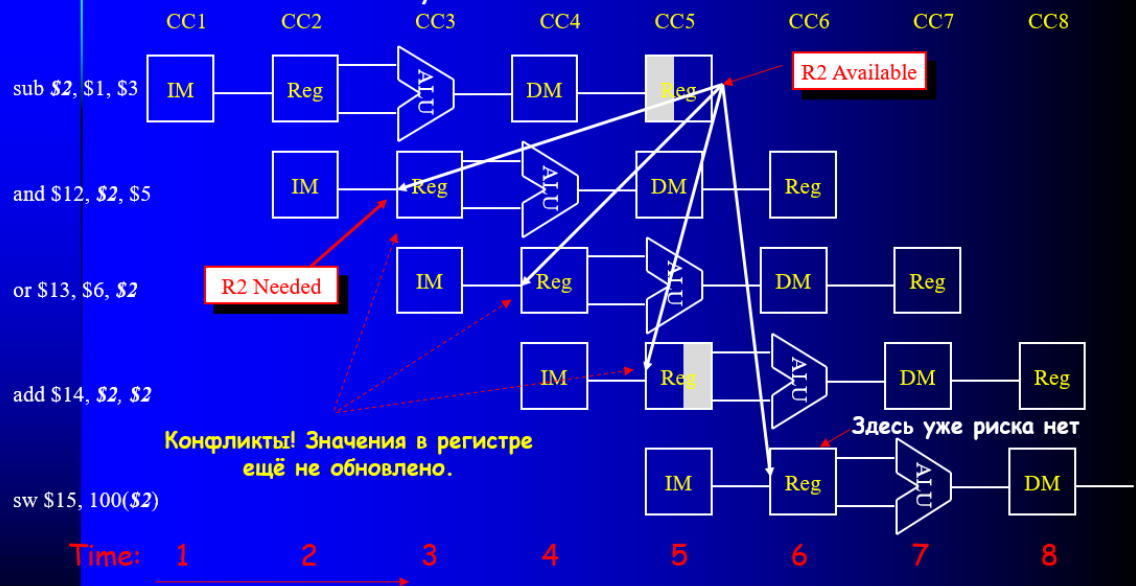
Forwarding - обычно буферный регистр между двумя ступенями конвейера, передача информации возможна только между соседними ступенями конвейера. При forwarding, операнд уже вычисленный предыдущей командой может быть получен непосредственно из соотв. буфера, минуя все промежуточные ступени конвейера -> доп. тракты данных (опережения, перехода) + соответствующее усложнение УУ.

Слайд 4

31. Риск по данным, типы конфликтов по данным.

Data Hazards

Риск по данным ("data hazard") – если результат требуется до того как он получен.



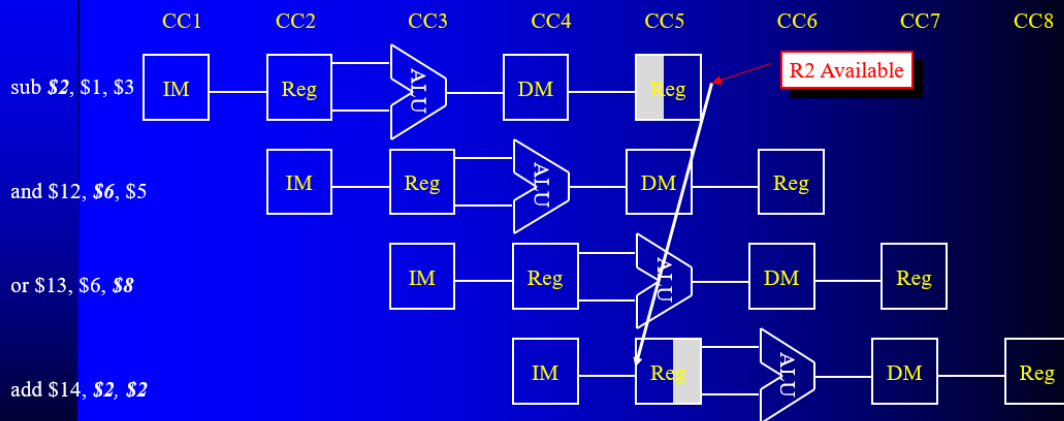
Слайд 8

Устранение риска по данным

Введение «прозрачного» набора регистров:

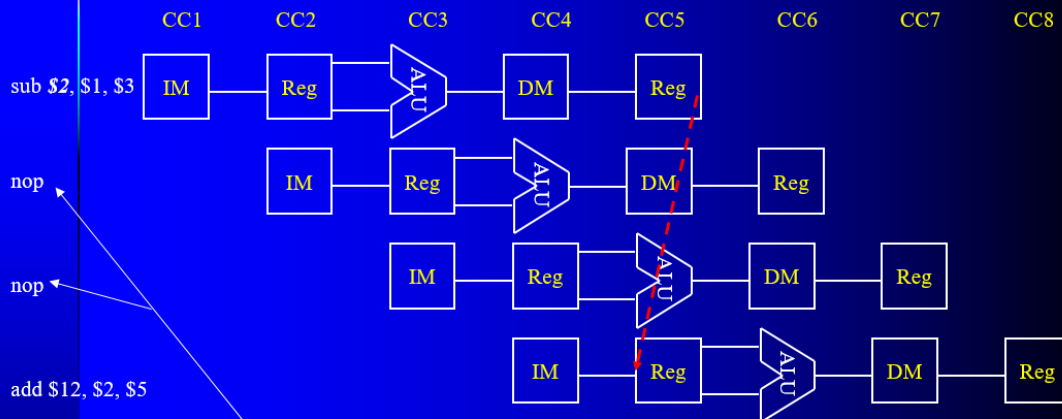
Использование защёлок (latches) вместо триггеров (flip-flops) в регистрах

- Первую половину цикла 5: регистр 2 загружается
- Вторую половину цикла: новое значение считывается



Слайд 9

Программный подход устранения риска по данным



Вставка достаточного количества «no-ops» (или других команд, не обращающихся к регистру 2) устраняет риск по данным

Слайд 10

За и против (Pros/Cons)

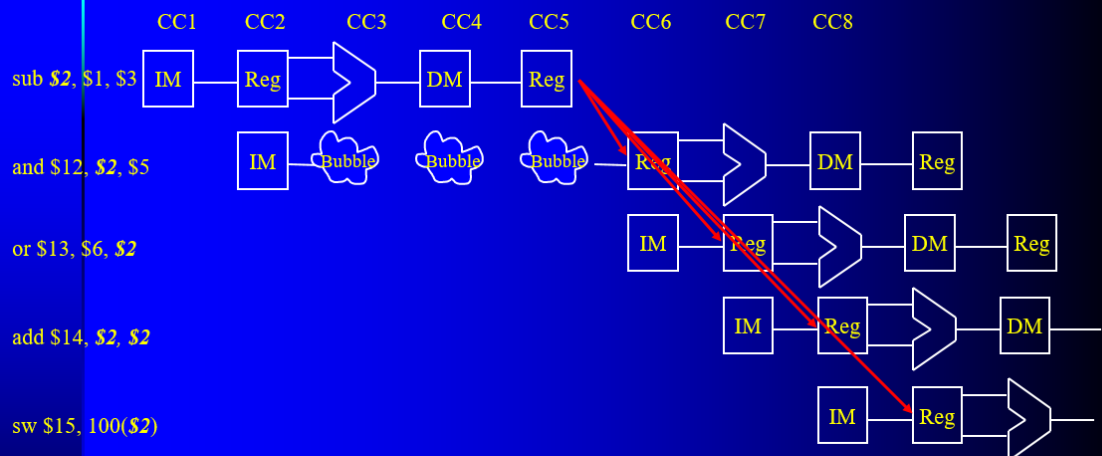
- Нет дополнительной аппаратуры – проблема передаётся на откуп компилятору
- В худшем случае требует «разбавления» кода «водой»
- Увеличение размера кода
- Сложности с поддержкой совместимости
 - Правила для отслеживания в командах риска изменяются при изменении конфигурации конвейера

Последний минус – самый главный «гвоздь в крышку гроба»

- Требование производителей полной смены программ при апгрейде системы очень не понравится покупателям.

Слайд 11

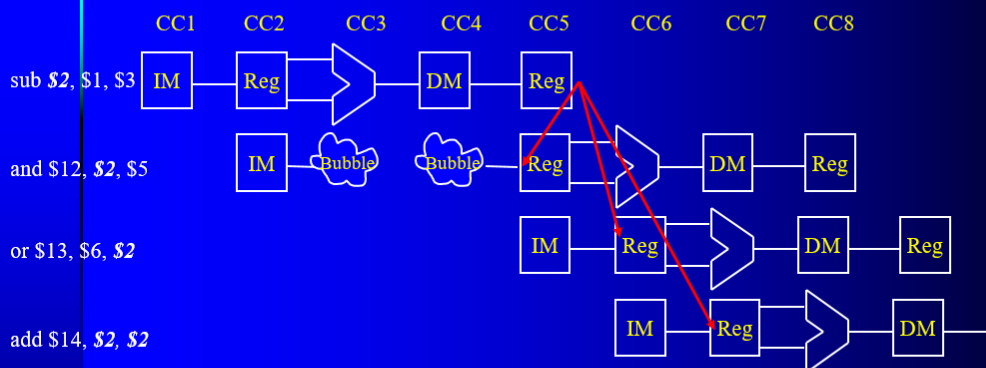
Пузырёк – приостановка конвейера Pipeline Stalls



Приостановка в случае реализации блока регистров на триггерах.

Слайд 12

Latches instead of flip-flops

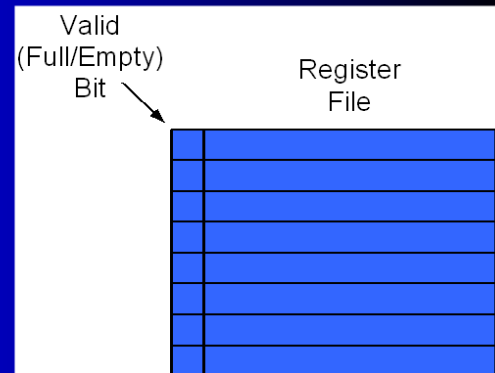


Приостановка в случае реализации блока регистров на защёлках.

Слайд 13

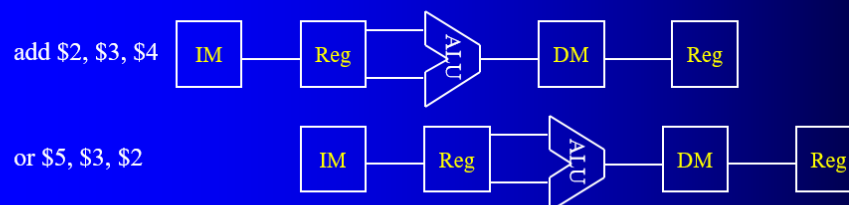
Флаг доступности данных. Register Scoreboard – Tracking Operand Availability

- К каждому регистру из пула регистров добавляется по 1 бит.
- Аппаратура сбрасывает флаг для соотв. регистра, если в конвейере появляется команда, записывающая результат в регистр.
- Флаг устанавливается в 1, если команда завершилась и записала результат в соотв. регистр.
- Команды не допускаются к выполнению до тех пор, пока их регистры – источники операндов не станут доступными

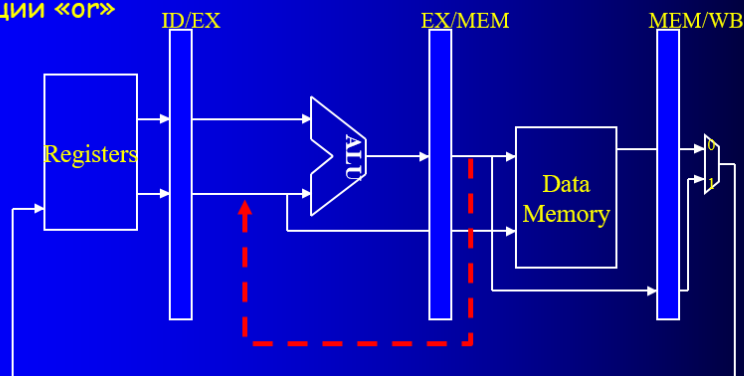


Слайд 14

Устранение риска по данным с помощью Forwarding



Можно избежать остановки конвейера, если предусмотреть возможность передачи результата операции «add» с выхода ALU прямо на его вход для операции «or»



Слайд 15

Ускоренное продвижение информации Data Forwarding

- Forwarding устраняет риск по данным на стадиях:
 - Выполнения команды (EX hazard)
 - Записи/чтения в/из памяти (MEM hazard)
- Использование «прозрачного» пула регистров позволяет бороться с риском по данным на третьей стадии – записи результата (WB)
 - в случае, если пул регистров запрашивается на одновременное чтение и запись из одного регистра в один цикл, то «прозрачный» пул регистров позволяет перенаправить записываемые данные прямо на выход.

Слайд 16

Типы конфликтов по данным:

1) Конфликты типа RAW (**Read After Write**): команда j пытается прочитать операнд прежде, чем команда i запишет на это место свой результат. При этом команда j может получить некорректное старое значение операнда.

Конфликты типа RAW обусловлены именно конвейерной организацией обработки команд.

Главной причиной двух других типов конфликтов по данным является возможность неупорядоченного выполнения команд в современных микропроцессорах, то есть выполнение команд не в том порядке, в котором они записаны в программе.

2) Конфликты типа WAR (**Write After Read**): команда j пытается записать результат в приемник, прежде чем он считается оттуда командой i . При этом команда i может получить некорректное новое значение операнда:

- 3) i) ADD R1,R2
 $i+1 = j$) SUB R2,R3

Этот конфликт возникнет в случае, если команда j вследствие неупорядоченного выполнения завершится раньше, чем команда i прочтает старое содержимое регистра R2.

4) Конфликты типа WAW (**Write After Write**): команда j пытается записать результат в приемник, прежде чем в этот же приемник будет записан результат выполнения команды i , то есть запись заканчивается в неверном порядке, оставляя в приемнике результата значение, записанное командой i .

32. Предсказание переходов — статический и динамический.

Предсказание переходов на сегодняшний день рассматривается как один из наиболее эффективных способов борьбы с конфликтами по управлению. Идея заключается в том, что еще до момента выполнения команды условного перехода или сразу же после ее поступления на конвейер делается предположение о наиболее вероятном исходе такой команды (переход произойдет или не произойдет). Последующие команды подаются на конвейер в соответствии с предсказанием. Если предсказан переход, то после команды x на конвейер подаются команды $y, y+1, \dots$. При ошибочном предсказании конвейер необходимо вернуть к состоянию, с которого началась выборка

«ненужных» команд (очистить начальные ступени конвейера), и приступить к загрузке, начиная с «правильной» точки, что по эффекту эквивалентно приостановке конвейера.

Цена ошибки может оказаться достаточно высокой, но при правильных предсказаниях крупен и выигрыш — конвейер функционирует ритмично без остановок и задержек, причем выигрыш тем больше, чем выше точность предсказания. Термин «точность предсказания» в дальнейшем будем трактовать как процентное отношение числа правильных предсказаний к их общему количеству. В работе [70] дается следующая оценка: чтобы снижение производительности конвейера из-за его остановок по причине конфликтов по управлению не превысило 10%, точность предсказания переходов должна быть выше 97,7%.

Статическое предсказание переходов осуществляется на основе некоторой априорной информации о подлежащей выполнению программе. Предсказание делается на этапе компиляции программы и в процессе вычислений уже не меняется. Главное различие между известными механизмами статического прогнозирования заключается в виде информации, используемой для предсказания, и ее трактовке. Исходная информация может быть получена двумя путями: на основе анализа кода программы или в результате ее профилирования. Известные стратегии статического предсказания для команд УП можно классифицировать следующим образом:

- переход происходит всегда (ПВ);
- переход не происходит никогда (ПН);
- предсказание определяется по результатам профилирования;
- предсказание определяется кодом операции команды перехода;
- предсказание зависит от направления перехода;

В динамических стратегиях решение о наиболее вероятном исходе команды УП принимается в ходе вычислений, исходя из информации о предшествующих переходах (истории переходов), собираемой в процессе выполнения программы. В целом, динамические стратегии по сравнению со статическими обеспечивают более высокую точность предсказания. Идея динамического предсказания переходов предполагает накопление информации об исходе предшествующих команд УП. История переходов фиксируется в форме таблицы, каждый элемент которой состоит из t битов. Нужный элемент таблицы указывается с помощью k -разрядной двоичной комбинации - шаблона (pattern). Этим объясняется общепринятое название таблицы предыстории переходов - таблица истории для шаблонов (PHT, Pattern History Table).

В качестве шаблонов для доступа PHT могут быть: адрес команды условного перехода, регистр глобальной истории, регистр локальной истории, комбинация этих вариантов.

При описании динамических схем предсказания переходов их часто расценивают как один из видов автоматов Мура, при этом содержимое элементов PHT трактуется как информация, отображающая текущее состояние автомата. Существуют различные варианты подобных автоматов, однако реально осмысленно говорить лишь о трех вариантах: A1, A2 и A3. Автоматы могут принимать 4 состояния (00 – перехода нет сильн., 01 – перехода нет слаб., 10 – переход есть слаб., 11 – переход есть сильн.).

33. Стратегии статического перехода.

Статическое предсказание переходов осуществляется на основе некоторой априорной информации о подлежащей выполнению программе. Предсказание делается на этапе компиляции программы и в процессе вычислений уже не меняется. Главное различие между известными механизмами статического прогнозирования заключается в виде информации, используемой для предсказания, и ее трактовке. Исходная информация может быть получена двумя путями: на основе анализа кода программы или в результате ее профилирования. Известные стратегии статического предсказания для команд УП можно классифицировать следующим образом:

- переход происходит всегда (ПВ);
- переход не происходит никогда (ПН);
- предсказание определяется по результатам профилирования;
- предсказание определяется кодом операции команды перехода;
- предсказание зависит от направления перехода;

В первом из перечисленных вариантов предполагается, что каждая команда условного перехода в программе обязательно завершится переходом, и, с учетом такого предсказания, дальнейшая выборка команд производится, начиная с адреса перехода. В основе второй стратегии лежит прямо противоположный подход: ни одна из команд условного перехода в программе никогда не завершается переходом, поэтому выборка команд продолжается в естественной последовательности.

Цифры свидетельствуют, что успешность стратегии ПВ (в среднем >50%) существенно зависит от характера программы и методов программирования, что, естественно, можно рассматривать как недостаток.

Ни ПВ, ни ПН не обладает явным преимуществом над другой. Тем не менее анализ большого количества программ показывает, что условные переходы имеют место более чем в 50% случаев, поэтому если стоимость реализации двух рассмотренных вариантов одинакова, то предпочтение следует отдать стратегии ПВ.

В третьем из перечисленных способов статического предсказания назначение командам УП наиболее вероятного исхода производится по результатам профилирования подлежащих выполнению программ, т.е. выполнялась программа при некотором эталонном наборе исходных данных, сопровождающееся сбором информации об исходах каждой команды условного перехода. Тем командам, которые чаще завершались переходом, назначается стратегия ПВ, а всем остальным - ПН. Выбор фиксируется в специальном бите кода операции. Некоторые компиляторы самостоятельно проводят профилирование программы и по его результатам устанавливают этот бит в формируемом объектном коде. При выполнении программы поведение конвейера команд определяется после выборки команды по состоянию упомянутого бита в коде операции. Основным недостатком этого образа действий связан с тем, что изменение набора исходных данных для профилирования может существенно менять поведение одних и тех же команд условного перехода.

При предсказании на основе кода операции команды перехода для одних команд предполагается, что переход произойдет, для других - его не случится. Например, для команды перехода по переполнению логично предположить, что перехода не будет. На практике назначение разным видам команд УП наиболее вероятного исхода чаще производится по результатам профилирования программ.

Достаточно логичным представляется предсказание исхода из направления перехода. Если указанный в команде адрес перехода меньше содержимого счетчика команд, говорят о переходе «назад», и для такой команды назначается стратегия ПВ. Переход к адресу, превышающему адрес команды перехода, считается переходом «вперед», и такой команде назначается стратегия ПН. В основе рассматриваемого подхода лежит статистика по множеству программ, согласно которой большинство команд УП в программах используются для организации циклов, причем, как правило, переходы происходят к началу цикла (переходы «назад»).

В последней из рассматриваемых статических стратегий при первом выполнении любой команды условного перехода делается предсказание, что переход обязательно будет. Предсказания на последующее выполнение команды зависят от правильности начального предсказания. Стратегию можно считать статической только частично, поскольку она однозначно определяет исход команды лишь при первом ее выполнении. Точность прогноза в соответствии с этой стратегией выше, чем у всех предшествующих.

34. Классификация архитектур системы команды (АСК)

Архитектура системы команд – интерфейс между всем выполняемым на машине ПО и аппаратным обеспечением.

АСК(ISA) характеризуется ответами на вопросы:

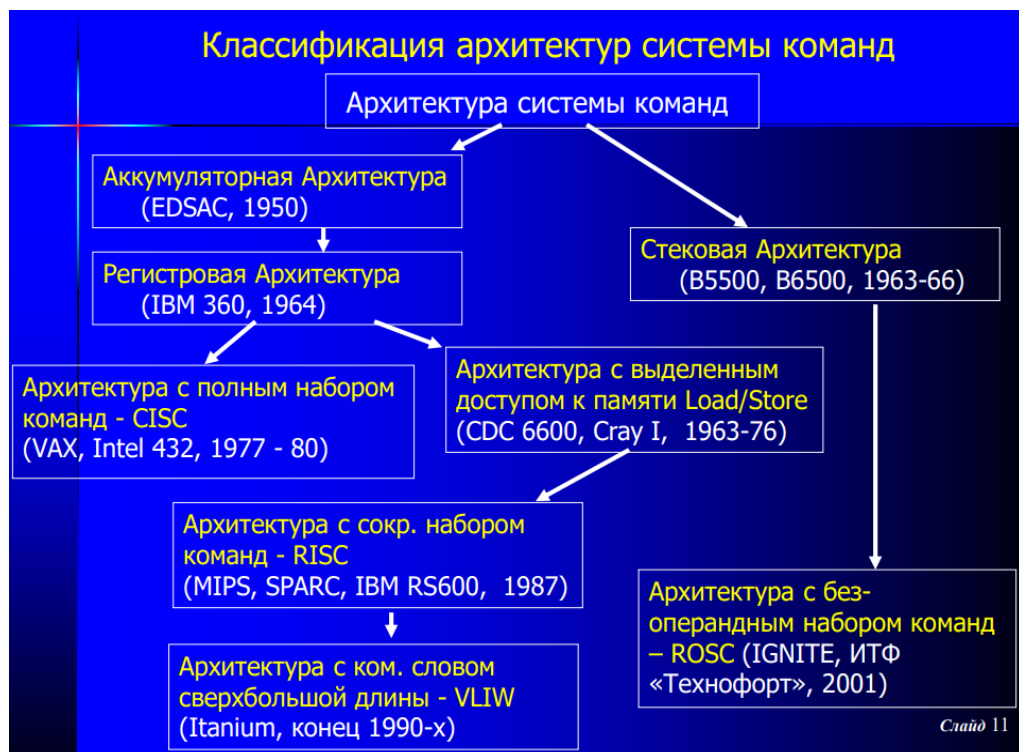
- Какого вида данные будут представлены в ВМ и в какой форме?
- Где эти данные могут храниться помимо основной памяти?
- Каким образом будет осуществляться доступ к данным?
- Какие операции могут быть выполнены над данными?
- Сколько операндов может присутствовать в команде?
- Как будет определяться адрес очередной команды?
- Каким образом будут закодированы команды?

АСК (ISA) – очень важный уровень абстракции:

- интерфейс между аппаратурой и низко-уровневым ПО
- стандартизирует команды, шаблоны битов машинного языка и т.д.
- преимущества: позволяет реализовывать различные аппаратные варианты одной программной архитектуры
- недостатки: периодически препятствует внедрению новых технологий и прочих инноваций

Современные АСК: – 80x86/Pentium/K6, PowerPC, DEC Alpha, MIPS, SPARC,

HP



Классификация по месту хранения команд:

- **Аккумуляторная архитектура (EDSAC, 1950)**

Исторически первая архитектура. В процессоре выделенный регистр – аккумулятор. В этот регистр – заносится результат операции. Изначально оба операнда в памяти. Перед операцией один из них – в аккумулятор. После выполнения результат в аккумуляторе (либо в памяти - если он не является операндом для следующей команды).

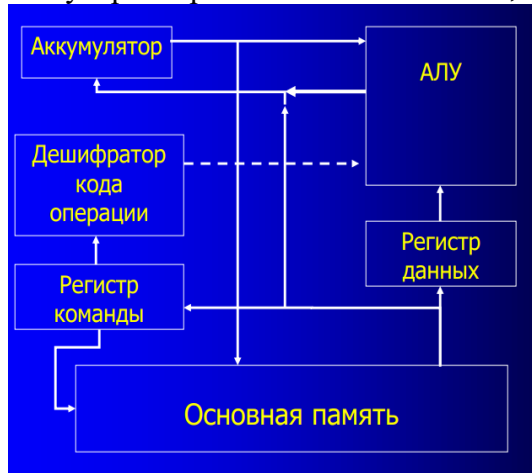
Достоинства:

- Короткие команды
- Простота декодирования команд

Недостатки:

- многократные обращения к памяти

Популярна в ранних ВМ – IBM 7090, DEC PDP-8, MOS 6502



- **Стековая архитектура (B5500, B6500, 1963-66)**

Стек – множество логически взаимосвязанных по принципу LIFO ячеек. Две операции – push и pop. Запись и чтение только в/из вершины стека. Операнды помещаются в стек, в процессе операции они выталкиваются из стека. Результат операции – в стек. Для стековой архитектуры лучше всего подходит обратная польская нотация (Чарльз Хэмблин, 1957), разработанная на основе «польской нотации» (Я. Лукасевич, 1920)

Достоинства:

- стек может быть неоднороден (верхние ячейки быстрые, нижние медленные).
- сокращение адр. части команд.
- компактный код
- простой компилятор
- простое декодирование команд

Недостатки:

Компилятор не может создать эф. код (нет произвольного обращения к памяти).

Стек – узкое место.

Низкая производительность.

Расцвет архитектуры – 60е годы: HP 2116B, HP 3000/70. Последние годы – повышение интереса к стековой архитектуре. Удобен для Java и Forth. Машины – JEM 1 и JEM 2 от aJile Systems, Clip от Imsys. ROSC (Removed Operand Set Computer) – новая реинкарнация стековой архитектуры. IGNITE от Patriot Scientist



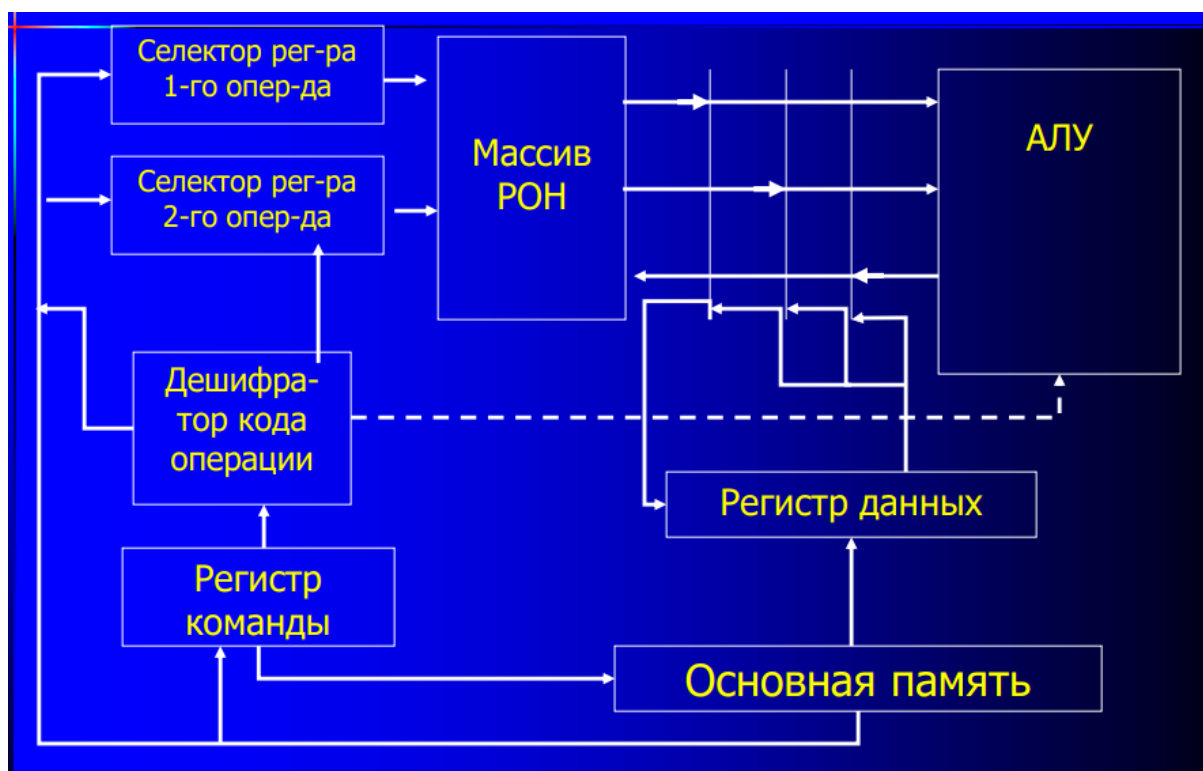
- **Регистровая Архитектура (IBM 360, 1964)**

Массив регистров (РОН) – явно управляемый для хранения недавно использовавшихся данных. Размер регистров – обычно совпадает с маш. словом. К люб. регистру можно обратиться по номеру. Меньшее количество регистров – меньше разрядов на кодирование

номера регистра – короче команда. Операнды – либо в памяти либо в регистрах.

Три подвида команд: Регистр-регистр, Регистр-память, Память-память

Вариант	Достоинства	Недостатки
Регистр-регистр	Простота реализации, фикс. длина команд, простота кода при компиляции, возможность выполнения всех команд за одинак. кол-во тактов	Большая длина кода, из-за фикс. длины команд – часть разрядов в команде не используется.
Регистр-память	Данные могут быть доступны и без загрузки в рег., простота кодирования команд, компактный код	Потеря одного из опер. при записи результата, длинное поле адреса в команде – сокращает кол-во РОН, CPI зависит от места размещ. операнда.
Память-память	Компактность кода, малая потребность в регистрах для хранения промежут. результатов	Разнообразие форматов команд и времени их исполнения, низкое быстродействие.



Операции загрузки операндов в РОН – идентичны работе с аккумулятором. Отличие – выбор нужного регистра. Выполнение операции АЛУ включает в себя: Выбор регистра первого операнда Определение местоположения второго оперда (память или регистр) Подача на вход АЛУ операндов и выполнение операции Выбор регистра результата и занесение в него результата операции из АЛУ.

Достоинства – компактность кода, высокая скорость.

Недостатки – более длинные инструкции (по сравнению с аккумуляторной архитектурой)

Регистровая АСК – преобладающая в настоящее время (все современные персоналки).

Первые машины – IBM 360/370, PDP-11

• **Архитектура с выделенным доступом к памяти Load/Store (CDC 6600, Cray I, 1963-76)**

Загрузка в регистр N ячейки x – load x N

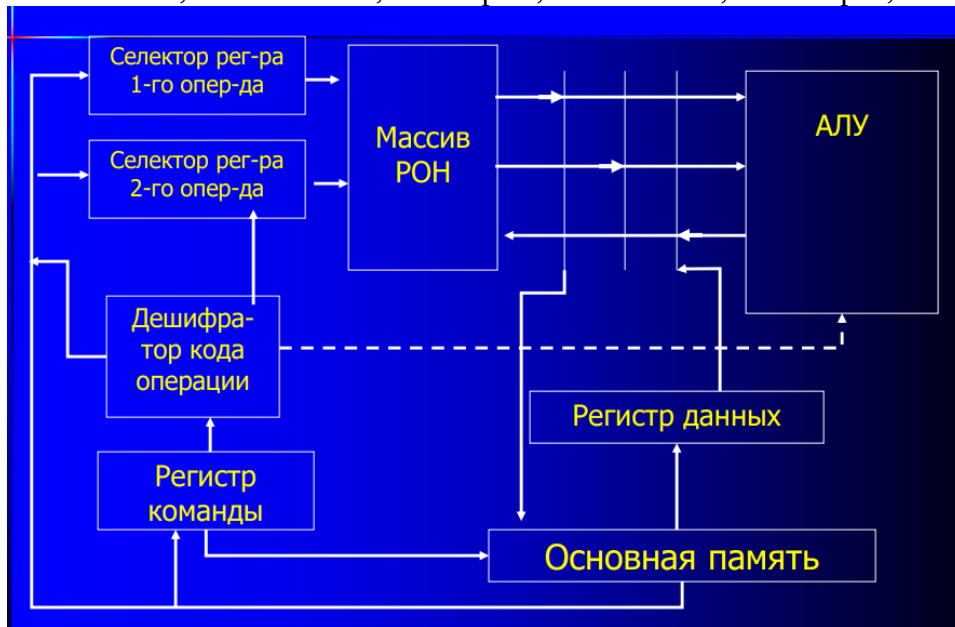
Запись из регистра N в ячейку x – store N x

Данная АСК характерна для всех ВМ с RISC архитектурой. Команды как правило – трёх

адресный формат и 32 бита длина.

Достоинства Load/Store ACK – простота декодирования и исполнения команд.

HP PA-RISC, IBM RS/6000, SUN Sparc, MIPS R4000, DEC Alpha, etc.



35. Принципы концепции ВМ фон Неймана. Какой из принципов можно назвать наиболее существенным?

Принципы ВМ Фон Неймана:

- **принцип хранимой в памяти программы**
 - ENIAC – (1946 -1955), программа задавалась схемой коммутации триггеров на 40 наборных полях.
 - EDVAC – (1949-...) первая машина с хранимой в памяти программой
- **двоичное кодирование**

Вся информация (команды, данные) – в бинарном виде

 - Каждый тип информации имеет свой формат
 - Последовательность битов, связанных одним смыслом, называется полем:
 - для данных
 - поле знака и поле значащих разрядов
 - для команды
 - поле кода операции и поле адресов
- **линейное пространство памяти (адресность)**
 - Память – набор пронумерованных ячеек
 - В каждый момент доступна любая ячейка
 - Двоичные коды данных и команд хранятся в единицах информации – словах
 - Доступ к данным по номерам ячеек – адресам
- **однородность памяти (безразличие к целевому назначению данных) Принстон**
 - Команды и данные хранятся в одной памяти и внешне не различимы
 - Возможность производить над командами те же операции, что и над данными (модификация команд)
 - Модификация команд современным программированием не приветствуется
 - Самый полезный эффект данного принципа – команды одной программы могут быть получены как результат работы другой программы. Возможность трансляции – главное следствие.
 - Команды и данные хранятся в одной памяти и внешне не различимы – Принстонская архитектура
 - Гарвардская архитектура – отдельная память команд и отдельная память данных.

- Узкое место принстонской архитектуры – пропускная способность тракта «процессор-память»
- В последнее время благодаря -памяти всё чаще начинает использоваться гарвардская архитектура.
- **программное управление (последовательное выполнение команд)**
 - Все вычисления алгоритма должны быть представлены в виде программы
 - Программа состоит из последовательности управляющих слов - команд
 - Команды хранятся в последовательных ячейках памяти и выполняются в естественной последовательности
 - Последовательность может быть изменена – условно или безусловно



Рис. 1.3. Структура фон-неймановской вычислительной машины

36. Принципы EPIC архитектуры (IA-64)

Использование архитектуры EPIC - главное отличие Itanium от RISC-процессоров. Она позволяет разбивать различные приложения и процессы вычислений на выполняемые параллельно фрагменты, обеспечивая в результате существенный скачок производительности. В архитектуре EPIC сочетаются многие технологические решения, которые в результате сочетания дают значительное повышение скорости обработки и решение некоторых проблем трансляции программ. Рассмотрим эти решения:

- Поддержка явно выделенного компилятором параллелизма.
- Большой регистровый файл.
- Подобно архитектуре PowerPC, в EPIC присутствуют предикатные регистры.

Использование предикатных регистров – важнейшая особенность, кардинально отличающая IA-64 от всех других процессоров.

- Спекулятивная загрузка данных, позволяющая избежать простоев конвейера при загрузке данных из оперативной памяти

- Поддержка предикатно выполняемых команд, которая позволяет избежать лишних инструкций ветвления, если количество команд в ветвях условного оператора невелико и уменьшить нагрузку на устройство предсказания ветвлений.

- Аппаратная поддержка программной конвейеризации с помощью механизма переименования регистров.

- Используется стек регистров (и регистровые окна).
- Для предсказания инструкций используется поддержка компилятора.
- Введена также поддержка инструкций циклического выполнения команд без потерь времени на инструкции циклического выполнения.

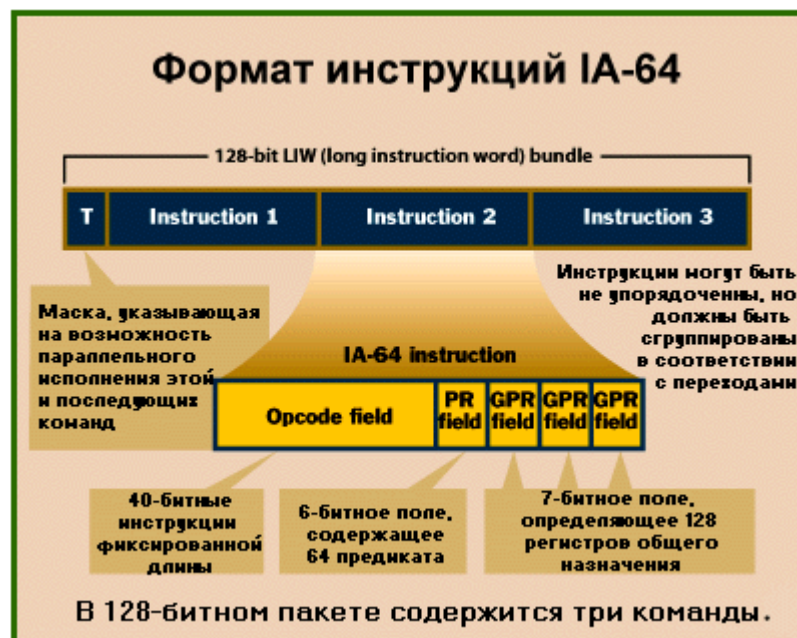
В EPIC, в отличие от RISC, задача распараллеливания инструкций возложена на компилятор. Это позволяет упростить структуру процессора, и как следствие, добиться более высокой его производительности при меньшей степени сложности. Кроме

того, компилятор оптимизирует выполнение программы целиком, а не отдельного ее фрагмента, который доступен процессору в случае аппаратного распараллеливания.

Наиболее интересной в архитектуре EPIC является поддержка спекулятивного выполнения команд и загрузка данных. Рассмотрим их подробнее.

Большинство команд загрузки данных из памяти выполняется длительное время. Выполнение команды за 1-2 такта возможно только в том случае, если значение содержится в -памяти второго уровня. Время несколько увеличивается, если значение находится в -памяти второго уровня. В случае чтения же данных из микросхем динамической памяти даже при попадании на активную страницу происходит значительная задержка при загрузке, а при смене страницы задержка имеет огромную величину, причём конвейер быстро блокируется, так команд, которые можно выполнить без нарушения зависимости по данным, обычно оказывается крайне мало. При спекулятивном выполнении используется вынесение команд загрузки далеко вперёд инструкций, использующих эти данные, в основном вверх за инструкции условного перехода. При достижении потоком команд места, где необходимо использовать загружаемые данные, вставляется инструкция, проверяющая, не произошло ли исключения в процессе загрузки (например, какая-то инструкция произвела запись по этому адресу), если исключение произошло, то вызывается специально написанный восстановительный код, который попросту перезагружает значение в регистр. Также при спекулятивных операциях по данным оптимизируется часто встречающийся участок во многих программах





В двух пространственных интервью журналу BYTE они раскрыли некоторые детали архитектуры IA-64.

- Команды в формате IA-64 упакованы по три в 128-битный пакет для быстрой обработки. Обычно это называют "LIW encoding". (Русский аналог подобрать сложно. Наиболее адекватно, на мой взгляд, перевести как "кодирование в длинные слова команд".) Однако компания Intel избегает такого названия, заявляя, что с ним связаны "негативные ассоциации" (negative connotation). По той же причине Intel не любит называть сами команды RISC-подобными (RISC-like), даже несмотря на то, что они имеют фиксированную длину и предположительно оптимизированы для исполнения за один такт в ядре, не нуждающемся в микрокоде. Intel предпочитает называть свою новую LIW-технология Explicitly Parallel Instruction Computing или EPIC (Вычисления с Явной Параллельностью Инструкций, где "явной" означает явно указанной при трансляции). В любом случае формат команд IA-64 не имеет ничего общего с x86. Команды x86 могут иметь длину от 8 до 108 бит, и процессор должен последовательно декодировать каждую команду после определения её границ.

- Каждый 128-битный пакет содержит шаблон (template) длиной в несколько бит, помещаемый в него компилятором, который указывает процессору, какие из команд могут выполняться параллельно. Теперь процессору не нужно будет анализировать поток команд в процессе выполнения для выявления "скрытого параллелизма". Вместо этого наличие параллелизма определяет компилятор и помещает информацию в код программы. Каждая команда (как для целочисленных вычислений, так и для вычислений с плавающей точкой) содержит три 7-битных поля регистра общего назначения (РОН). Из этого следует, что процессоры архитектуры IA-64 содержат 128 целочисленных РОН и 128 регистров для вычислений с плавающей точкой. Все они доступны программисту и являются регистрами с произвольным доступом (programmer-visible random-access registers). По сравнению с процессорами x86, у которых всего восемь целочисленных РОН и стек глубины 8 для вычислений с плавающей точкой, IA-64 намного "шире" и, соответственно, будет намного реже простаивать из-за "нехватки регистров".

- Компиляторы для IA-64 будут использовать технологию "отмеченных команд" (predication) для устранения потерь производительности из-за неправильно предсказанных переходов и необходимости пропуска участков кода после ветвлений. Когда процессор встречает "отмеченное" ветвление в процессе выполнения программы, он начинает одновременно выполнять все ветви. После того, как будет определена "истинная" ветвь, процессор сохраняет необходимые результаты и сбрасывает остальные.

- **Компиляторы для IA-64 будут также просматривать исходный код с целью поиска команд, использующих данные из памяти. Найдя такую команду, они будут добавлять пару команд - команду предварительной загрузки (speculative loading) и проверки загрузки (speculative check).** Во время выполнения программы первая из команд загружает данные в память до того, как они понадобятся программе. Вторая команда проверяет, успешно ли произошла загрузка, перед тем, как разрешить программе использовать эти данные. Предварительная загрузка позволяет уменьшить потери производительности из-за задержек при доступе к памяти, а также повысить параллелизм.

37. Какие факторы влияют на длину команды? Возможные пути сокращения длины команды.

Каждая команда должна содержать сведения, необходимые для ее выполнения. Сведения кодируются. Для кодировки каждой группы сведений выделяется свое поле. Совокупность полей, содержащих необходимые сведения для выполнения требуемой операции, называют **форматом команды**.

В формате команды должны быть определены:

- **функциональное назначение операции в виде кода операции;**
- **адреса источников данных.** В общем случае должны быть указаны адреса двух операндов;
- **адрес места расположения результата;**
- **адрес следующей команды.**

Рассмотрим гипотетическую ситуацию. Допустим, что в формате команды:

- поле кода операций (КО) занимает 4 разряда, что позволяет закодировать $2^4 = 16$ операций;
- под адреса двух операндов-источников, адрес места расположения результата, адрес следующей команды выделены поля АО 1, АО 2, АР, АСК (рис. 2.7.1) по 12 разрядов, что позволяет в каждом случае адресовать $2^{12} = 4К$ ячеек памяти. Как видно из рис. 2.7.1, несмотря на скромные возможности команды, ее общая длина составляет достаточно большое число (52) бит.

КО	АО1	АО2	АР	АСК
4 бита	12 бит	12 бит	12 бит	12 бит

Рис. 2.7.1. Формат команды

Для сокращения количества разрядов команды часть информации должна быть задана неявно и не должна зависеть от особенностей конкретной команды.

Наиболее употребительными являются следующие способы сокращения длины кода команды:

- **использование специально предусмотренных для адресации регистров: программного счетчика, указателя стека и др.** Например, при выполнении команд с последовательно возрастающими адресами программный счетчик автоматически считывает из памяти следующую команду. В этом случае в формате команды отсутствует поле адреса следующей команды;

- **использование неявных способов адресации.** Например, при использовании неявной регистровой адресации адрес следующей команды или операнда хранится в регистре, код которого содержит значительно меньшее число разрядов, чем код исполнительного адреса;

- **совмещение источника одного из операндов с приемником результата.** В этом случае в формате команды исключается поле адреса результата;

- использование команд с укороченной адресацией, т. е. части адресного пространства памяти;

- использование для некоторой группы операций одного регистра. Наиболее часто для этой цели используется аккумулятор. В операциях с аккумулятором не требуется его кодировка. Например, команда ADD выполняет операцию сложение содержимого регистра В с аккумулятором А и в него же помещает результат. Такая команда является одноадресной. Ее формат состоит из двух полей: кода операции и адреса операнда. Однако одноадресные команды требуют дополнительных команд для предварительной загрузки операндов в аккумулятор и последующего размещения результатов в памяти;

- использование нескольких аккумуляторов (Motorola 6800, National PACE, Signetic 2650). В этом случае каждая команда имеет два адреса, однако адрес источника и места назначения может быть задан другим аккумулятором. Следует отметить, что в одних процессорах все команды имеют одинаковую длину, в других — разную длину. Одинаковая длина всех команд упрощает декодирование, однако требует большего пространства, поскольку все команды должны быть такой же длины, как самая длинная. Команды могут быть короче слова, равными слову или длиннее слова. В процессорах с неймановской архитектурой команды и данные имеют одинаковую длину и поступают в процессор по шине данных. Поэтому для отличия команд, отданных в процессоре предусмотрены средства, обеспечивающие:

- засылку команд (первого байта) в регистр команд с дальнейшей их дешифрацией для активизации устройства управления;

- поступление данных (последующих байт) в аккумулятор или другие регистры для обработки в АЛУ.

38. В чём суть и достоинства конвейеризации транзакций?

Одним из способов повышения скорости передачи данных по шине является *конвейеризация транзакций*. Очередной элемент данных может быть отправлен устройством А до того, как устройство В завершит считывание предыдущего элемента.



Данные должны оставаться стабильными в течение времени $t_{ст}$ (стабилизации) + $t_{уд}$ (удержания), только после этого — смена элемента данных.

Максимальная скорость передачи — $1/(t_{ст} + t_{уд})$

39. Способы ускорения транзакций.

Для сокращения времени транзакций проектировщики обычно прибегают к следующим приемам:

- арбитражу с перекрытием;
- арбитражу с удержанием шины;

- расщеплению транзакций

В шине с расщеплением транзакций линии адреса и данных обязаны быть независимыми. Каждая транзакция чтения разделяется на две части: адресную транзакцию и транзакцию данных. Считывание данных из памяти начинается с адресной транзакции: выставления ведущим на адресную шину адреса ячейки. С приходом адреса память приступает к относительно длительному процессу поиска и извлечения затребованных данных. По завершении чтения память становится ведущим устройством, запрашивает доступ к шине и направляет считанные данные по шине данных. Фактически от момента поступления запроса до момента формирования отклика шина остается незанятой и может быть востребована для выполнения других транзакций. В этом и состоит главная идея протокола расщепления транзакций. Т.о., на шине с расщеплением транзакций имеют место поток запросов и поток откликов. Часто в системах с расщеплением транзакций контроллер памяти проектируется так, чтобы обеспечить буферизацию множественных запросов.

Арбитраж с перекрытием (overlapped arbitration) заключается в том, что одновременно с выполнением текущей транзакции производится арбитраж следующей транзакции.

При арбитраже с удержанием шины (bus parking) ведущий может удерживать шину и выполнять множество транзакций, пока отсутствуют запросы от других потенциальных ведущих. В современных шинах обычно сочетаются все вышеперечисленные способы ускорения транзакций

40. Методы доступа к ЗУ?

При оценке быстродействия необходимо учитывать применяемый в данном типе ЗУ метод доступа к данным. Различают четыре основных метода доступа:

- Последовательный доступ. ЗУ с последовательным доступом ориентировано на хранение информации в виде последовательности блоков данных, называемых записями. Для доступа к нужному элементу (слову или байту) необходимо прочитать все предшествующие ему данные. Время доступа зависит от положения требуемой записи в последовательности записей на носителе информации и позиции элемента внутри данной записи. Примером может служить ЗУ на магнитной ленте.

- Прямой доступ. Каждая запись имеет уникальный адрес, отражающий ее физическое размещение на носителе информации. Обращение осуществляется как адресный доступ к началу записи, с последующим последовательным доступом к определенной единице информации внутри записи. В результате время доступа к определенной позиции является величиной переменной. Такой режим характерен для магнитных дисков.

- Произвольный доступ. Каждая ячейка памяти имеет уникальный физический адрес. Обращение к любой ячейке занимает одно и то же время и может вестись в произвольной очередности. Примером могут служить запоминающие устройства основной памяти.

- Ассоциативный доступ. Этот вид доступа позволяет выполнять поиск ячеек, содержащих такую информацию, в которой значение отдельных битов совпадает с состоянием одноименных битов в заданном образце. Сравнение осуществляется параллельно для всех ячеек памяти, независимо от ее емкости. По ассоциативному принципу построены некоторые блоки кэш-памяти.

41. Иерархия запоминающих устройств.

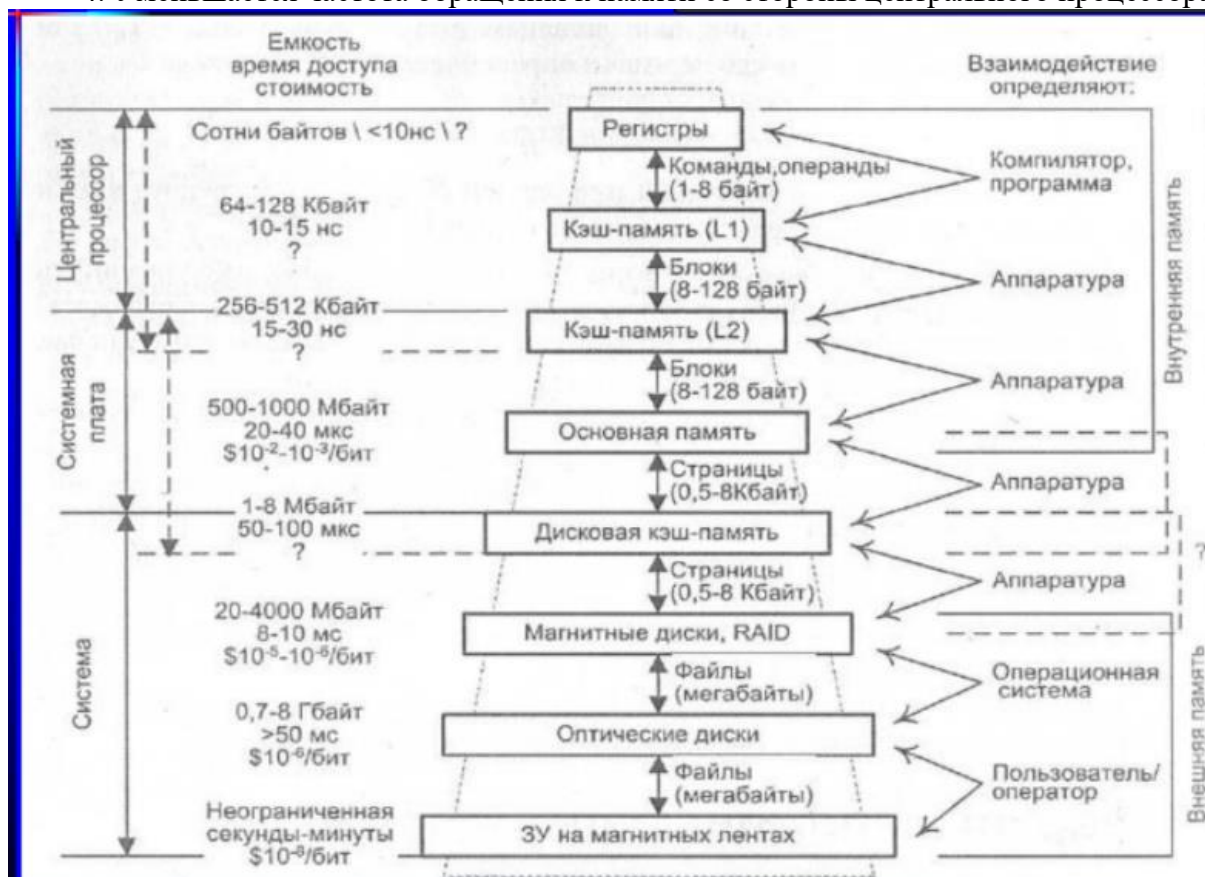
Иерархическая память состоит из ЗУ различных типов, которые, в зависимости от характеристик, относят к определенному уровню иерархии. Более высокий уровень меньше по емкости, быстрее и имеет большую стоимость в пересчете на бит, чем более низкий уровень. Уровни иерархии взаимосвязаны: все данные на одном уровне могут

быть также найдены на более низком уровне, и все данные на этом более низком уровне могут быть найдены на следующем нижележащем уровне и т. д.

Четыре верхних уровня иерархии образуют внутреннюю память ВМ, а все нижние уровни - это внешняя или вторичная память.

По мере движения вниз по иерархической структуре:

1. Уменьшается соотношение «стоимость/бит».
2. Возрастает емкость.
3. Растет время доступа.
4. Уменьшается частота обращения к памяти со стороны центрального процессора



Принцип локальности по обращению. Если рассмотреть процесс выполнения большинства программ, то можно заметить, что с очень высокой вероятностью адрес очередной команды программы либо следует непосредственно за адресом, по которому была считана текущая команда, либо расположен вблизи него. Такое расположение адресов называется пространственной локальностью программы.

Из свойства локальности вытекает, что программу разумно представить в виде последовательно обрабатываемых фрагментов - компактных групп команд и данных. Помещая такие фрагменты в более быструю память, можно существенно снизить общие задержки на обращение, поскольку команды и данные, будучи один раз переданы из медленного ЗУ в быстрое, затем могут использоваться многократно, и среднее время доступа к ним в этом случае определяется уже более быстрым ЗУ.

На каждом уровне иерархии информация разбивается на блоки, выступающие в качестве наименьшей информационной единицы, пересылаемой между двумя соседними уровнями иерархии. Размер блоков может быть фиксированным либо переменным. При фиксированном размере блока емкость памяти обычно кратна его размеру. Размер блоков на каждом уровне иерархии чаще всего различен и увеличивается от верхних уровней к нижним.

При доступе к командам и данным, например, для их считывания, сначала производится поиск в памяти верхнего уровня. Факт обнаружения нужной информации называют попаданием (hit), в противном случае говорят о промахе (miss). При промахе

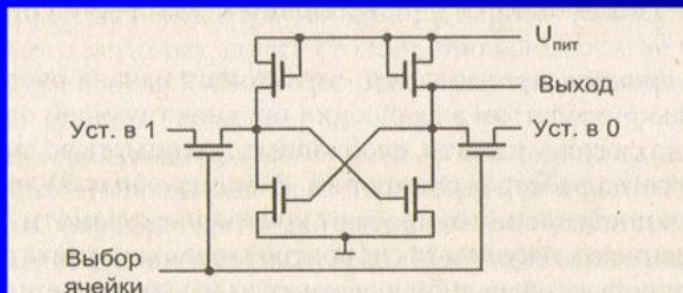
производится поиск в ЗУ следующего более низкого уровня, где также возможны попадание или промах

42. Статические и динамические ОЗУ?

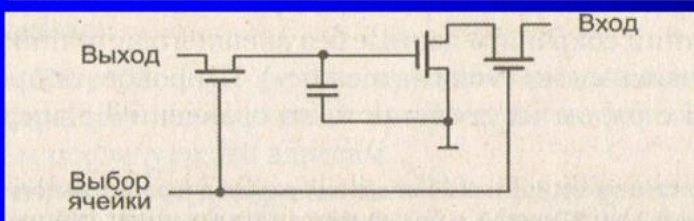
В **статических ОЗУ** запоминающий элемент может хранить записанную информацию неограниченно долго (при наличии питающего напряжения).

Запоминающий элемент **динамического ОЗУ** способен хранить информацию только в течение достаточно короткого промежутка времени, после которого информацию нужно восстанавливать заново, иначе она будет потеряна. Динамические ЗУ, как и статические, энергозависимы.

Роль запоминающего элемента в статическом ОЗУ выполняет триггер (схема с двумя устойчивыми состояниями, обычно состоящую из четырех или шести транзисторов).



SRAM, на 6 транзисторах – быстрее, менее чувствительны к внешним излучениям



DRAM

Простота схемы DRAM позволяет достичь высокой плотности размещения, в итоге, *снизить стоимость*. *Главный недостаток - что накапливаемый на конденсаторе заряд со временем теряется. Среднее время утечки заряда DRAM составляет сотни(десятки) миллисекунд, поэтому, заряд следует регенерировать.*

Слайд 2

Рассмотрим достоинства и недостатки SRAM и DRAM:

1. Память типа DRAM, в силу своей технологии, имеет гораздо большую плотность размещения данных, чем SRAM.
2. DRAM гораздо дешевле SRAM,
3. но последняя производительнее и надежнее, поскольку всегда готова к считыванию.

43. Блочная схема расслоения памяти.

Емкость современных ОЗУ слишком велика для одной ИМС. Также объединение нескольких ИМС ЗУ необходимо если разрядность ИМС меньше разрядности ШД. Если объединить несколько ИМС по адресным входам до нужной разрядности - это модуль памяти (может состоять и из одной ИМС).

Один или несколько модулей - банк памяти.

Для получения ЗУ нужной ёмкости – объединение нескольких банков памяти. Адрес ячейки A памяти из B банков $\rightarrow (b, w)$, где b – номер банка, w – адрес ячейки в банке.

Три схемы распределения адресов A на (b, w) :

Блочная (номер банка b – старшие разряды адреса)

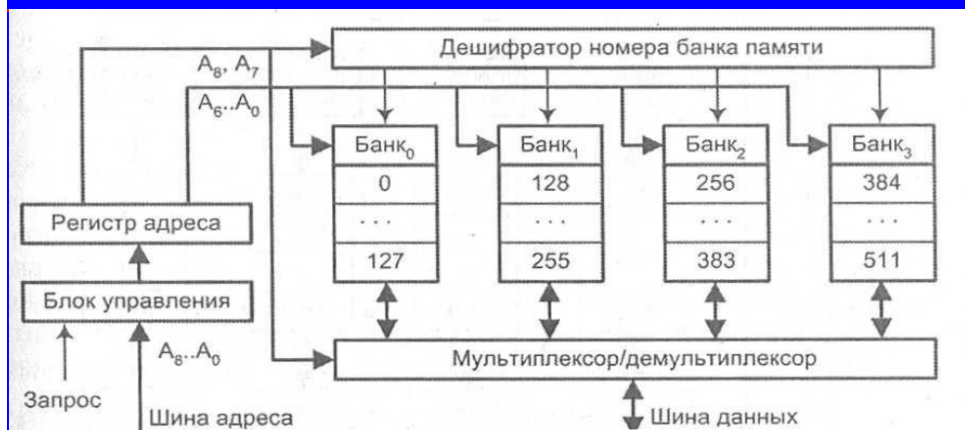
Циклическая ($b = A \bmod B$; $w = A \div B$)

Блочно-циклическая (комбинация двух предыдущих схем)

Пример – память ёмкостью 512 слов – четыре банка по 128 слов в каждом (типовая структура по блочному принципу).

По данному:

Блочная организация памяти



Адресное пространство памяти разбито на группы последовательных адресов. Каждая такая группа обеспечивается отдельным банком памяти. Для обращения используется 9-разрядный адрес, семь младших разрядов которого ($A_6 - A_0$) поступают параллельно на все банки памяти и выбирают в каждом из них одну ячейку.

Два старших разряда адреса (A_8, A_7) содержат номер банка. Выбор банка обеспечивается либо с помощью дешифратора номера банка памяти, либо путем мультиплексирования информации рис. иллюстрирует оба варианта). В функциональном отношении такая ОП может рассматриваться как единое ЗУ, емкость которого равна суммарной емкости отдельных банков, а быстродействие – быстродействию отдельного банка.

44. Циклическая схема расслоения памяти.

Начало 43 вопроса;

В каждом такте на шине адреса может присутствовать адрес только одной ячейки, поэтому параллельное обращение к нескольким банкам невозможно, но оно может быть организовано со сдвигом на один такт. Адрес ячейки запоминается в индивидуальном регистре адреса, и дальнейшие операции по доступу к ячейке в каждом банке протекают независимо. При большом количестве банков среднее время доступа к памяти сокращается почти в B раз (B – количество банков), но при условии, что ячейки относятся к разным банкам. Если же запросы к одному тому же банку следуют друг за другом, каждый следующий запрос должен ожидать завершения обслуживания предыдущего. Такая ситуация называется конфликтом по доступу. При частых конфликтах данный метод становится неэффективным.

45. Блочно-циклическая схема расслоения памяти.

Начало 43 вопроса;

Блочно-циклическая схема расслоения – каждый банк – из нескольких модулей, адресуемых по круговой схеме. Адреса между банками – по блочной схеме. В итоге – адрес ячейки разбивается на три части: *номер банка, с модуле, модуль в банке.*

Банк ₀		Банк ₁		Банк ₂		Банк ₃	
Модуль ₀	Модуль ₁	Модуль ₂	Модуль ₃	Модуль ₄	Модуль ₅	Модуль ₆	Модуль ₇
0	1	128	129	256	257	384	385
2	3	130	131	258	259	386	387
...	...	...	...	...	...	...	...
126	127	254	255	382	383	510	511

Расслоение хорошо работает для обычной задачи – благодаря свойству локальности. В многопроцессорных системах с общей памятью – несколько контроллеров для автономной работы – по отдельным банкам. Чем больше банков – тем меньше вероятность конфликта. В суперкомп-ре NEC SX/3 – 128 банков основной памяти.²

46. Последовательный и конвейерный режим доступа к микросхеме ЗУ.

Немного воды:

Интегральные микросхемы (ИМС) памяти организованы в виде матрицы ячеек, каждая из которых в зависимости от разрядности ИМС состоит из одного или более запоминающих элементов (ЗЭ).

Каждый ЗЭ способен хранить 1 бит информации. При матричной организации ИМС памяти реализуется координатный принцип адресации ячеек.

Адрес ячейки разделяется логикой выбора на две составляющие адрес строки и адрес столбца. Адреса запоминаются в соответствующих регистрах. Дешифраторы регистров выбирают вертикальные и горизонтальные линии, на пересечении которых находятся по одному ЗЭ.

Последовательный режим

При использовании последовательного режима (Flow through Mode) адрес и управляющие сигналы подаются на микросхему до поступления синхроимпульса. В

момент прихода синхроимпульса вся входная информация запоминается во внутренних регистрах — по его переднему фронту, и начинается цикл чтения. Через некоторое время, но в пределах того же цикла данные появляются на внешней шине, причем момент этот определяется только моментом прихода синхронизирующего импульса и скоростью внутренних цепей микросхемы.

Конвейерный режим

Конвейерный режим (pipelined mode) - это такой метод доступа к данным, при котором можно продолжать операцию чтения по предыдущему адресу в процессе запроса по следующему.

При чтении из памяти время, требуемое для извлечения данных из ячейки, можно условно разбить на два интервала. Первый из них — непосредственно доступ к массиву запоминающих элементов и извлечение данных из ячейки. Второй — передача данных на выход (при этом происходит детектирование состояния ячейки, сигнала и другие операции, необходимые для считывания информации).

В отличие от последовательного режима, где следующий цикл чтения начинается только по окончании предыдущего, в конвейерном режиме процесс разбивается на два этапа. Пока данные из предыдущего цикла чтения передаются на внешнюю шину, происходит запрос на следующую операцию чтения. Таким образом, два цикла чтения перекрываются во времени. Из-за усложнения схемы передачи данных на внешнюю шину время считывания увеличивается на один такт, и данные поступают на выход только в следующем такте, но такое запаздывание наблюдается лишь при первом чтении в последовательности операций считывания из памяти. Все последующие данные поступают на выход друг за другом, хотя и с запаздыванием на один такт относительно запроса на чтение. Так как циклы чтения перекрываются, микросхемы с конвейерным режимом могут использоваться при частотах шины, вдвое превышающих допустимую для ИМС с последовательным режимом чтения.

47. Регистровый и страничный режим доступа к микросхеме 3У.

Регистровый режим

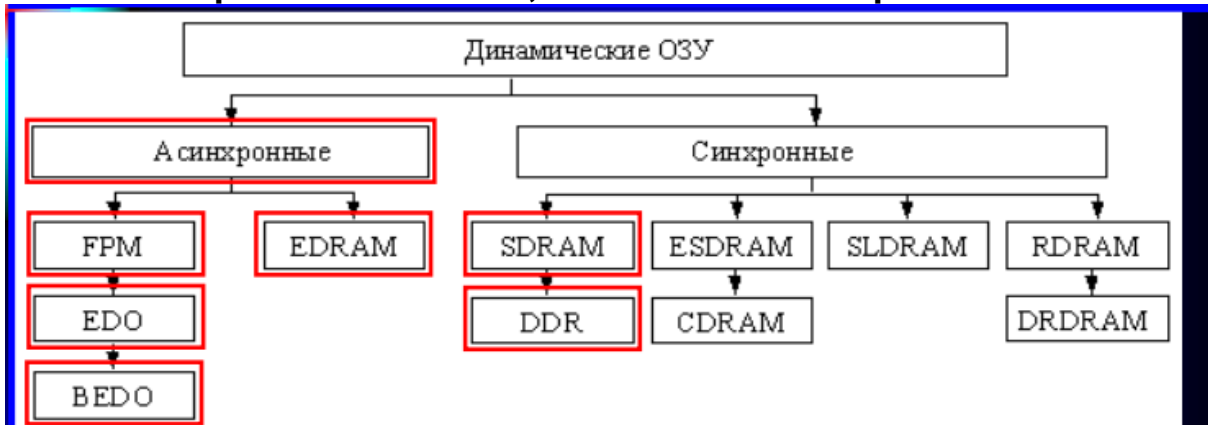
Регистровый режим (Register to Latch) используется относительно редко и отличается наличием регистра на выходе микросхемы. Адрес и управляющие сигналы выдаются на шину до поступления синхронизирующего импульса. С приходом положительного фронта синхроимпульса адрес записывается во внутренний регистр микросхемы, и начинается цикл чтения. Считанные данные заносятся в промежуточный выходной регистр и хранятся там до появления отрицательного фронта (спада) синхроимпульса, а с его поступлением передаются на шину. Метод однозначно определяет момент появления данных на выходе ИМС, причем изменяя ширину импульса синхронизации можно менять время появления данных на шине. Данное свойство часто оказывается весьма полезным при проектировании специализированных ВМ. По быстродействию микросхемы с регистровым режимом идентичны ИМС с последовательным режимом.

Страничный режим

В основе идеи лежит тот факт, что при доступе к ячейкам со смежными адресами (согласно принципу локальности, такая ситуация наиболее вероятна), причем к таким, где все 3Э расположены в одной строке матрицы, доступ ко второй и последующим ячейкам можно производить существенно быстрее. Действительно, если адрес строки при очередном обращении остался прежним, то все временные затраты, связанные с повторным занесением адреса строки в регистр ИМС, дешифровкой, зарядом паразитной емкости горизонтальной линии и т. п., можно исключить. Для доступа к очередной ячейке достаточно подавать на ИМС лишь адрес нового столбца, сопровождая его сигналом CAS. Отметим, что обращение к первой ячейке в последовательности производится

стандартным образом — поочередным заданием адреса строки и адреса столбца, то есть здесь время доступа уменьшить практически невозможно. Рассмотренный режим называется режимом страничного доступа или просто страничным режимом (Page Mode). Под страницей понимается строка матрицы ЗЭ. Микросхемы, где реализуется страничный режим и его модификации, принято характеризовать формулой $x-y-u$. Первое число x представляет количество тактов системной шины, необходимое для доступа к первой ячейке последовательности, а y — к каждой из последующих ячеек. Так, выражение 7-3-3-3 означает, что для обработки первого слова необходимо 7 тактовых периодов системной шины (в течение шести из которых шина простаивает в ожидании), а для обработки последующих слов — по три периода, из которых два системная шина также простаивает.

48. Микросхемы SDRAM, отличия от асинхронных DRAM.



DDR SDRAM – Double Data Rate Synchronous DRAM.
 Главное отличие – данные в **пакетном режиме** выдаются по обоим фронтам импульса синхронизации, за счёт чего пропускная способность возрастает вдвое.

Микросхемы DDR SDRAM (Double Data Rate SDRAM - SDRAM с удвоенной скоростью передачи данных) - дальнейшее развитие технологии SDRAM - выдает данные в пакетном режиме по обоим фронтам импульса синхронизации, увеличивая пропускную способность вдвое.

Сущ. несколько спецификаций DDR SDRAM в зависимости от тактовой частоты шины – DDR266, DDR333, DDR400, DDR533. Пиковая пропускная способность DDR333 – 2,7 Гбайт/с, DDR400 – 3,2 Гбайт в сек.

Микросхемы SDRAM. Аббревиатура SDRAM (Synchronous DRAM - синхронная DRAM) используется для обозначения микросхем «обычных» синхронных динамических ОЗУ. Кардинальные отличия SDRAM от рассмотренных выше асинхронных динамических ОЗУ можно свести к четырем положениям:

- синхронный метод передачи данных на шину;
- конвейерный механизм пересылки пакета;
- применение нескольких (двух или четырех) внутренних банков памяти;

- передача части функций контроллера памяти логике самой микросхемы.

Синхронность памяти позволяет контроллеру памяти «знать» моменты готовности данных, за счет чего снижаются издержки циклов ожидания и поиска данных. Так как данные появляются на выходе ИМС одновременно с тактовыми импульсами, упрощается взаимодействие памяти с другими устройствами ВМ.

Значительный эффект дает разбиение всей совокупности ячеек на независимые внутренние массивы (банки). Это позволяет совмещать доступ к ячейке одного банка с подготовкой к следующей операции в остальных банках (перезарядкой управляющих цепей и восстановлением информации). Возможность держать открытыми одновременно несколько строк памяти (из разных банков) также способствует повышению быстродействия памяти. При поочередном доступе к банкам частота обращения к каждому из них в отдельности уменьшается пропорционально числу банков и SDRAM может работать на более высоких частотах. Благодаря встроенному счетчику адресов SDRAM, позволяет производить чтение и запись в пакетном режиме, причем в SDRAM длина пакета варьируется и в пакетном режиме есть возможность чтения целой строки памяти. ИМС может быть охарактеризована формулой 5-1-1-1.

49. Энергонезависимые ОЗУ.

Под понятие энергонезависимое ОЗУ (NVRAM - Non-Volatile RAM) подпадает несколько типов памяти. От перепрограммируемых постоянных ЗУ их отличает отсутствие этапа стирания, предваряющего запись новой информации, поэтому вместо термина «программирование» для них употребляют стандартный термин «запись».

Микросхемы BBSRAM. К рассматриваемой группе относятся обычные статические ОЗУ со встроенным литиевым аккумулятором и усиленной защитой от искажения информации в момент включения и отключения внешнего питания. Для их обозначения применяют аббревиатуру BBSRAM (Battery-Back SRAM).

Микросхемы NVRAM. Другой подход реализован в микросхеме, разработанной компанией Simtec. Особенность ее в том, что в одном корпусе объединены статическое ОЗУ и перепрограммируемая постоянная память типа EEPROM. При включении питания данные копируются из EEPROM в SRAM, а при выключении — автоматически перезаписываются из SRAM в EEPROM. Благодаря такому приему данный вид памяти можно считать энергонезависимым.

Микросхемы FRAM. FRAM (Ferroelectric RAM - ферроэлектрическая память) разработана компанией Ramtron и представляет собой еще один вариант энергонезависимой памяти. По быстродействию данное ЗУ несколько уступает динамическим ОЗУ и пока рассматривается лишь как альтернатива флэш-памяти. Причисление FRAM к оперативным ЗУ обусловлено отсутствием перед записью явно выраженного цикла стирания информации.

50. Специализированные ОЗУ.

В ряде практических задач более выгодным оказывается использование специализированных архитектур ОЗУ, где стандартные функции (запись, хранение, считывание) сочетаются с некоторыми дополнительными возможностями или учитывают особенности применения памяти. Такие виды ОЗУ называют специализированными и к ним причисляют:

- память для видеоадаптеров;
- память с множественным доступом (многопортовые ОЗУ);
- память типа очереди (ОЗУ типа FIFO).

Использование памяти в видеоадаптерах имеет свою специфику и для реализации дополнительных требований прибегают к несколько иным типам микросхем. Так, при создании динамичных изображений часто достаточно просто изменить расположение уже хранящейся в видеопамяти информации. Вместо того чтобы многократно пересылать по

шине одни и те же данные, лишь несколько изменив их расположение, выгоднее заставить микросхему памяти переместить уже хранящиеся в ней данные из одной области ядра в другую. На ИМС памяти можно также возложить операции по изменению цвета точек изображения (SGRAM, VRAM, WRAM, MDRAM, 3D-RAM)

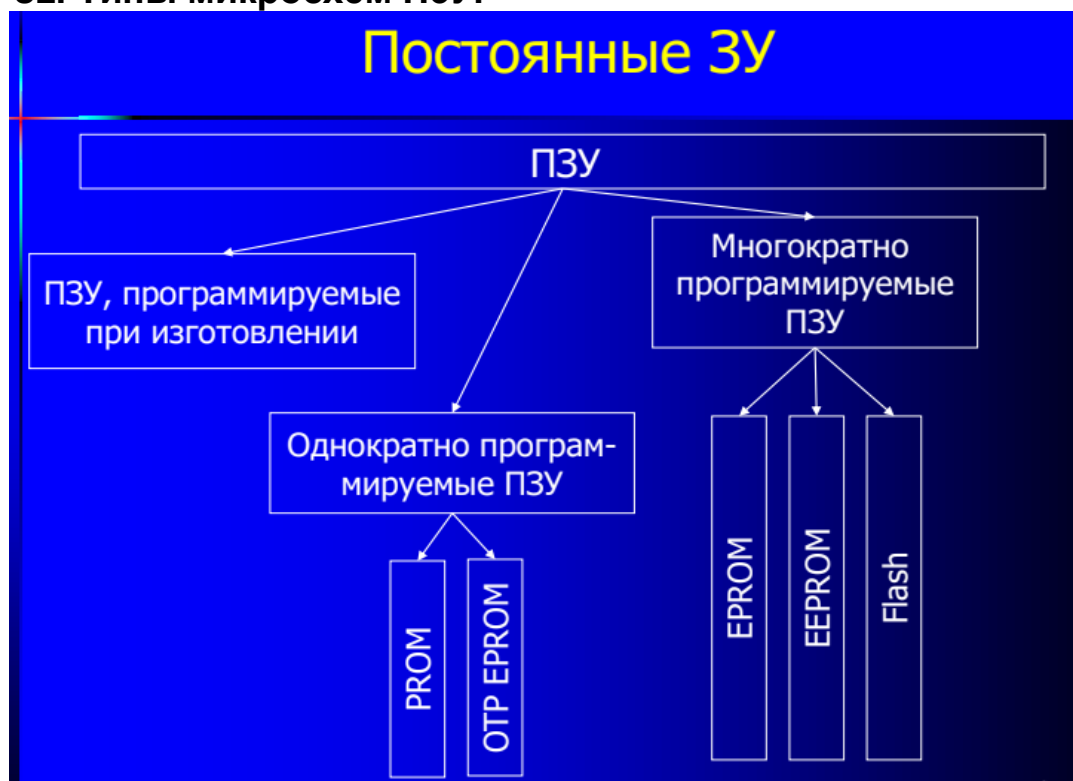
В отличие от стандартного в n-портовом ОЗУ имеется п независимых наборов шин адреса, данных и управления, гарантирующих одновременный и независимый доступ к ОЗУ п устройствам. Данное свойство позволяет существенно упростить создание многопроцессорных и многомашинных вычислительных систем, где многопортовое ОЗУ выступает в роли общей или совместно используемой памяти. В рамках одной ВМ подобное ОЗУ может обеспечивать обмен информацией между ЦП и УВВ (например, контроллером магнитного диска) намного эффективней, чем прямой доступ к памяти.

Во многих случаях ОЗУ применяется для буферизации потока данных, когда данные считываются из памяти в той же последовательности, в которой они туда заносились, но поступление и считывание происходят с различной скоростью. Часто для этой цели применяют обычное ОЗУ, однако здесь одновременная запись и считывание информации невозможны. Более эффективным видом ОЗУ, где оба действия могут вестись одновременно, служит память типа FIFO. Микросхема представляет собой двухпортовое ОЗУ, где один порт предназначен для занесения информации, а второй - для считывания. Для FIFO-памяти характерны все технологические приемы, свойственные двухпортовой памяти, в частности способы арбитража при одномоментном обращении к одной и той же ячейке. В то же время есть и существенные отличия.

Первое состоит в том, что у микросхемы нет входов для указания адреса ячейки, занесение и считывание данных производится в порядке их поступления через одну входную точку и одну выходную.

Второе отличие связано с необходимостью слежения за состоянием очереди. Для этого в микросхеме имеются регистры-указатели адресов начала и конца очереди, а также специальные флаги, которые указывают на две ситуации: отсутствие данных (в этом случае блокируется считывание из микросхемы) и полное заполнение памяти (блокируется запись).

51. Типы микросхем ПЗУ.



ПЗУ – **Read Only Memory (ROM)**. Программируемые при изготовлении – MROM (Mask ROM).

Занесение информации в ПЗУ – часть технологического процесса изготовления микросхемы. Различные технологии, но чаще всего – отсутствие либо наличие перемычки (транзистора) на пересечении адресной и разрядной линии. Наиболее дешёвый вид ПЗУ – высокая плотность упаковки ЗЭ на кристалле и высокая скорость считывания. Пример применения – ПЗУ шрифтов в лазерных принтерах.

Однократно программируемые ПЗУ – полуфабрикат, занесение информации – на этапе изготовления мелкосерийного устройства, использующего ПЗУ.

PROM – **Programmable ROM**. Первые микросхемы – на базе плавких предохранителей. Занесение информации – пережигание отдельных перемычек (пробоя отдельных диодов). Основной недостаток – необходимость спец. программатора, большой процент брака, необходимость термической постобработки схемы (- иначе низкая надёжность хранения).

ОТР EPROM – **One Time Programmable EPROM** – однократно программируемая EPROM – кристалл EPROM в пластиковом корпусе (без кварцевого окна).

Многократно программируемые ПЗУ

В многократно программируемых два этапа – стирание всего + запись новой информации.

EPROM – **Erasable Programmable ROM** – запись электрическими сигналами, но стирание – воздействием ультрафиолетового облучения через кварцевое окно корпуса микросхемы. Для исключения случайного стирания – заклеивание окна непрозрачной плёнкой. Стирание – многократное (20 мин). Иногда называют UV-EPROM - **Ultra Violet EPROM**. Время программирования – несколько сотен миллисекунд. Время считывания как у ROM и DRAM. Дороже, чем PROM, но выгоднее в итоге.

EEPROM – **Electrical Erasable Programmable ROM** – стирание и запись по-байтово, причём стирание – как обязательный этап записи. Запись – намного дольше, чем чтение – несколько сотен микросекунд на байт. Не требует программатора – запись реализуется внутренней логикой микросхемы.

Два варианта – с последовательным и параллельным доступом. 90% всех микросхем – **SEEPROM – Serial EEPROM** – адреса, данные и управление – по одному входу. Преимущества – малые габариты, минимальное число линий ввода/вывода. Недостатки – большое время доступа, цена – EEPROM дороже EPROM.

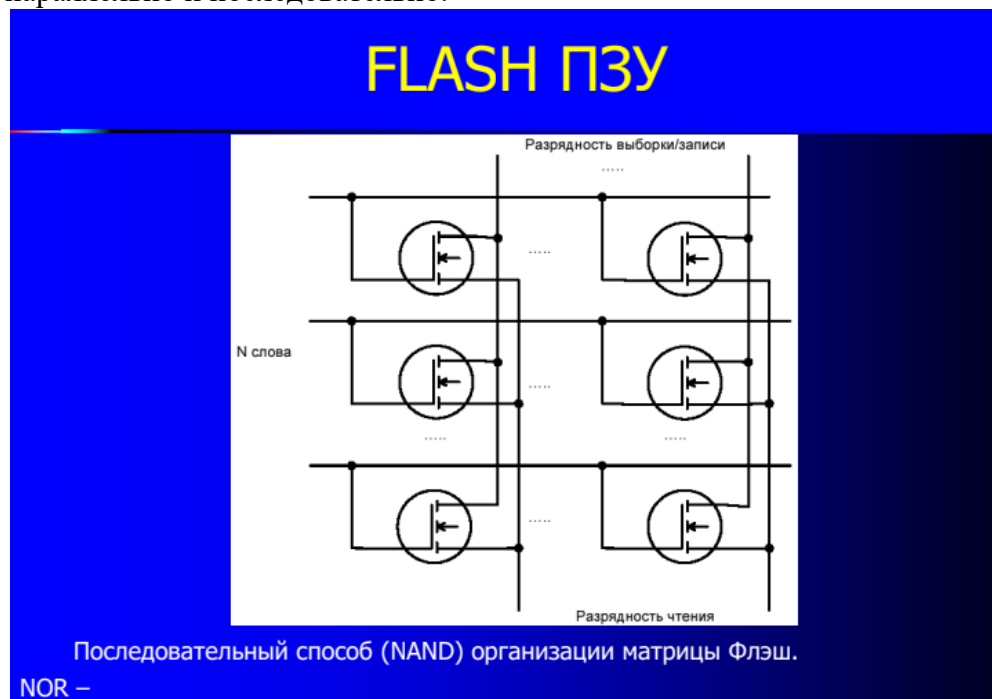
52. «Flash» ПЗУ.

FLASH - Флэш-память появилась в конце 1980-х годов (Intel) – перепрограммируемая ПЗУ с электрическим стиранием. Стирание осуществляется сразу целой областью ячеек : блока или всей микросхемы => более быстрая запись информации (программирование ЗУ).

Для упрощения записи – спец. блоки ИМС, делающие запись “прозрачной” для ПО и внешних устройств.

Флэш-память строится на однотранзисторных элементах памяти (с “плавающим” затвором), => плотность хранения информации чуть выше, чем DRAM.

Различные технологии построения базовых элементов флэш - количество слоев, методы стирания и записи данных, структура. Наиболее широко известны NOR и NAND типы флэш-памяти, запоминающие транзисторы в которых подключены к разрядным шинам - параллельно и последовательно.



Минусы – относительно невысокая скорость передачи данных.

Элементы памяти флэш-ЗУ организованы в матрицы, разрядность данных для микросхем - 1-2 байта.

Операция чтения из флэш - как в обычных ЗУ с произвольным доступом (ОЗУ). Но запись – с некоторыми особенностями, аналогичными - ПЗУ. Перед записью данных в ячейки они должны быть очищены (стерты). Стирание - перевод элементов памяти в состояние единицы и возможно только сразу для целого блока ячеек (в первых микросхемах предусматривалось стирание только для всей матрицы сразу- Bulk Erase). Выборочное - невозможно. В процессе записи информации соответствующие элементы памяти переключаются в нулевое состояние. Также, как и в ПЗУ, без стирания можно дозаписать нули в уже запрограммированные ячейки, однако необходимость в такой операции относительно редка.

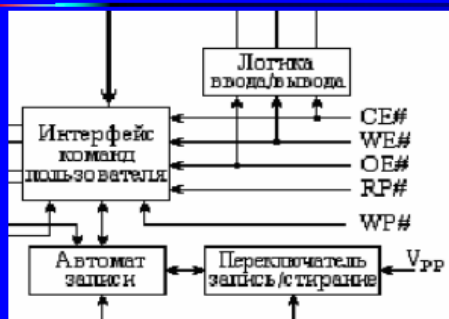
Фактически при операции записи производится два действия: стирание и запись, но управление этими операциями производится внутренним автоматом и “прозрачно” для процессора.

Разбиение адресного пространства микросхемы флэш –памяти на блоки обычно двух видов: симметричное и асимметричное.

Количество блоков зависит от ёмкости микросхемы. Например, в микросхеме 28F128J3 (Intel Strata Flash) емкостью 128 Мбит (16 Мбайт) имеется 128 блоков по 128 Кб.

Пример - 28F640C3 (Intel Advanced+ Boot Block) ёмкостью 64 Мбит: загрузочный (Boot) блок размером 64 Кбайт, разбит на 8 блоков (parameter blocks) по 8 Кбайт, и 127 основных (main) блоков по 64 Кбайт. Загрузочный блок- либо в начале, либо в конце адресного пространства ИМС.

FLASH ПЗУ



WP# (Write Protect) используется для исключения возможности случайной записи по командам программы. RP# (Reset/Deep Power Down) - закрывает все блоки от записи. На вход V_{pp} подается напряжение, необходимое для ускорения операций стирания и записи данных.

Для улучшения характеристик в флэш-памяти предусматриваются:

1)

Для улучшения характеристик в флэш-памяти:

4) использование режимов понижения мощности когда к ЗУ нет обращения (важно для мобильных устройств);

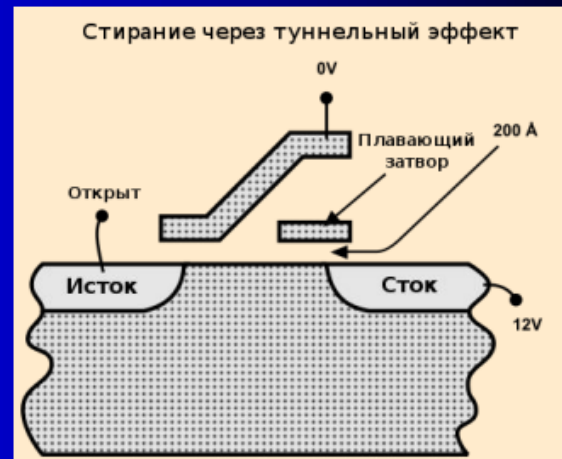
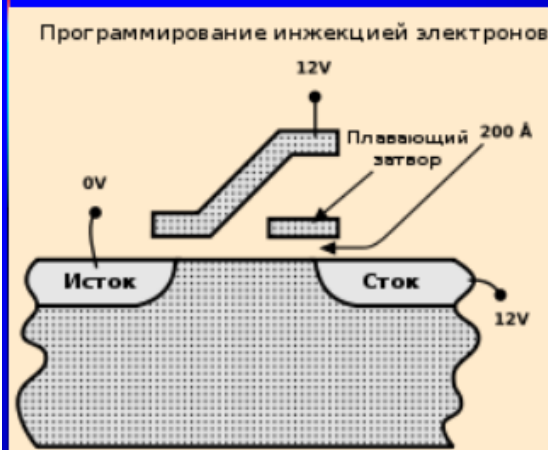
5) приспособленность к работе с различными питающими напряжениями;

6)

Применение – BIOS, «твердотельные диски» (solid-state disks), карты памяти мобильных устройств.

Время записи байта – 10 мкс, время доступа при чтении – 35-200нс, полное стирание 1-2 с, высокая плотность размещения на кристалле – на 30% больше чем у DRAM.

FLASH ПЗУ



53. Производительность иерархии памяти.

При доступе к командам и данным, например, для их считывания, сначала производится поиск в памяти верхнего уровня. Факт обнаружения нужной информации называют попаданием (hit), в противном случае говорят о промахе (miss). При промахе производится поиск в ЗУ следующего более низкого уровня, где также возможны попадание или промах.

Производительность иерархии памяти

Основная формула:

$$T_{avg} = P_{hit} * T_{hit} + P_{miss} * T_{miss}$$

T_{hit} = время выполнения запроса данных из памяти в случае, если данные присутствуют на текущем уровне иерархии

T_{miss} = время выполнения запроса данных в случае если данные на текущем уровне не присутствуют и необходимо обращение к нижестоящему уровню

P_{hit} , P_{miss} = вероятности попадания (hit) или промаха (miss) для текущего уровня

– P_{hit} всегда 100% для самого нижнего уровня иерархии

54. Ассоциативная память.

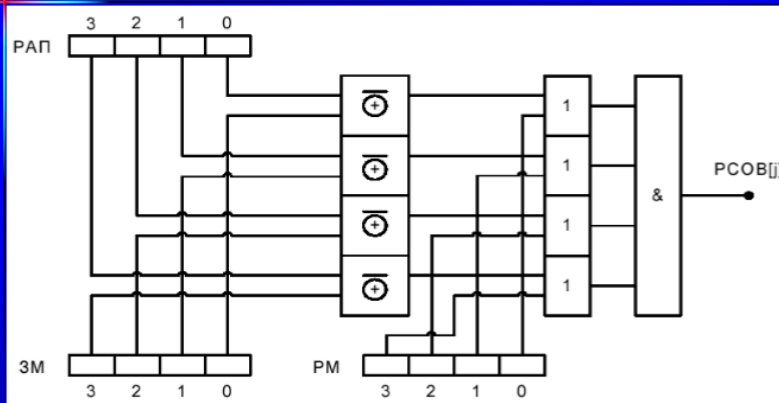
Ассоциативная память



Структура ассоциативного ЗУ

Слайд 24

Ассоциативная память

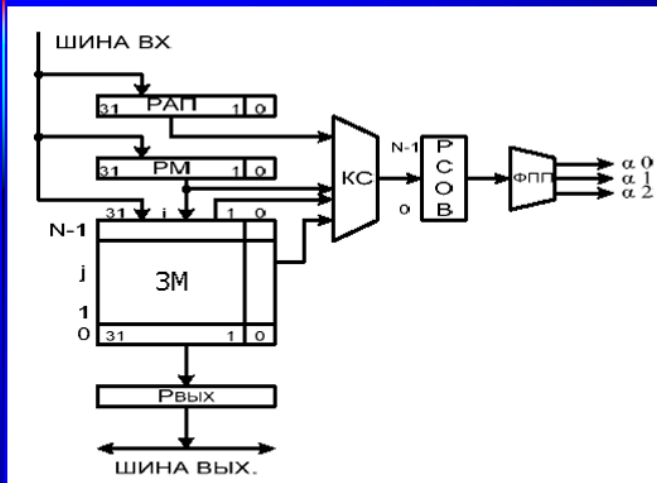


РАП – регистр ассоциативного признака;
РМ – регистр маски;
ЗМ – запоминающий массив;
PCOV [j] – регистр совпадений;

ЗМ содержит N ячеек для указания занятости которых используется нулевой разряд (0-свободна, 1-занята) - необходим для поиска свободных ячеек при записи новой информации.
РМ - в ассоциативном поиске участвует только тот разряд, в котором $PM_i = 1$, если $PM_i = 0$, то i-ый разряд не участвует в поиске.

Слайд 4

Ассоциативная память



РАП – регистр ассоциативного признака;

PM – регистр маски;

ЗМ – запоминающий массив;

РСОВ [j] – регистр совпадений;

КС – комбинационная схема
ФПП – формирование признака поиска

α0- данные не найдены

α1- содержатся в одной ячейке

α2- сод-ся в нескольких ячей-х.

При считывании сначала – поиск по ассоциативному признаку в РАП и по PM, далее анализируются признаки поиска.

При α0 = 1 считывание отменяется из-за отсутствия искомой информации. При α1 = 1 считывается единственное найденное слово. При α2 = 1 – обычно считывается слово из ячейки, имеющей наименьший номер и отмеченной единицей в РСОВ.

Слайд 5

Ассоциативная память

При записи сначала отыскивается свободная ячейка в ЗМ. Для ее поиска выполняется операция контроля ассоциаций для РАП равного 11..110 и PM = 00...001, так как у свободных ячеек последний младший разряд равен нулю. Все найденные свободные ячейки отмечаются единицами в РСОВ. Для записи выбирается свободная ячейка с наименьшим номером, в нее записывается слово с шины входных данных. Если α0 = 0, то есть, свободных ячеек нет, то производится удаление из ЗМ "устаревшей" информации. По некоторому устанавливаемому признаку в РАП производится контроль ассоциации и операция удаления, то есть сброс в нуль нулевого разряда ячеек, отмеченных при контроле.

Для ассоциативной памяти не может использоваться динамическое ЗУ, так как считывание производится одновременно во всем ЗМ. Поэтому используются элементы памяти, не допускающие разрушения информации при считывании.

Слайд 6

55. Кэш с полностью ассоциативным отображением.

Простой кэш

- Полностью ассоциативный: любой блок данных ОЗУ может быть помещён в любое место кэш
- LRU (least recently used) стратегия замещения: при недостатке места выкидывается наиболее давно требовавшиеся данные.

Очень маленький кэш:
4 записи, каждая по 4-х байтному слову, любая запись может содержать любое слово.

Вспомогательное поле,
облегчающее определение
кандидатов на вылет

Тэг (tag)
определяет
адреса
данных кэш

tag

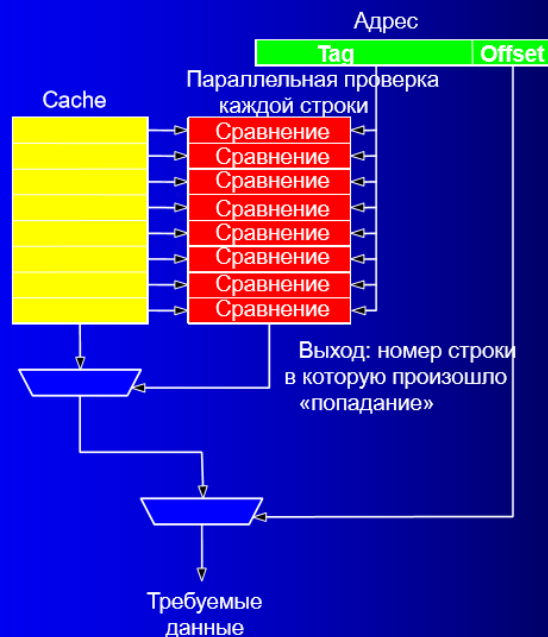
data

time since last
reference

tag	data	time since last reference

Слайд 8

Полностью ассоциативное отображение (Fully-Associative Cache)



Слайд 10

В памяти с полностью ассоциативным отображением любая строка данных из основной памяти может быть продублирована в любом месте (строке). Таким образом, данного типа не может производить замещение хранимых в ней данных до тех пор, пока в наличие имеются свободные строки.

56. Кэш с прямым отображением.

Прямое отображение

В кэш с прямым отображением (direct mapped cache), каждому блоку памяти поставлена в соответствие отдельная строка кэш-памяти.

$i = j \bmod m$, где m – общее число строк в кэш-памяти.

- Обычно* используются несколько бит адреса для «задания» отображения блоков памяти на кэш
- Например, биты 2 и 3 (если считать LSB = "0") адреса – будут индексом.

**В некоторых машинах используется псевдо-случайная хэш-таблица адресов*

Слайд 11

Direct mapped cache in action

Последовательность ячеек: 24, 20, 04, 12, 20, 44, 04, 24, 44

index	tag	data
00	-	-
01	20 - 23	data
10	24 - 27	data
11	-	-
00	-	-
01	04 - 08	data
10	24 - 27	data
11	-	-
00	-	-
01	04 - 08	data
10	24 - 27	data
11	12 - 15	data
00	-	-
01	04 - 08	data
10	24 - 27	data
11	12 - 15	data

24 = 011000₂ ; индекс - 10.

20 = 010100₂ ; индекс - 01.

(* index is bits 2-3 of address)

04 = 000100₂ ; индекс - 01.

(выбрасывает 20-23 из кэш)

12 = 001100₂ ; индекс - 11.

your turn ...

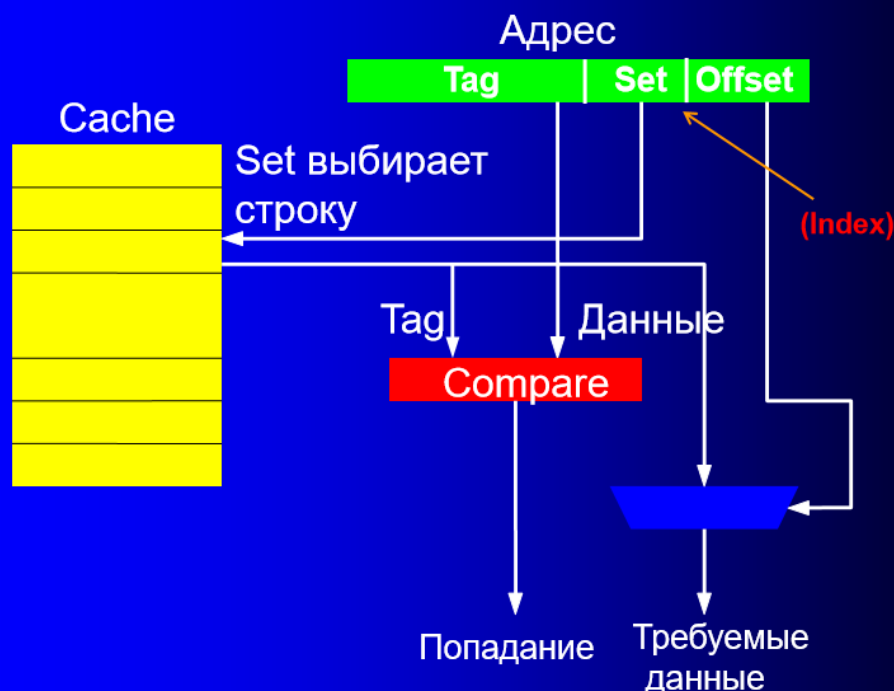
20 = 010100₂

44 = 101100₂

04 = 000100₂

Слайд 12

Direct-Mapped Cache



Слайд 13

Direct-Mapped vs. Fully-Associative

- Прямое отображение (Direct-Mapped)
 - Требует меньше места
 - Только один компаратор
 - Требуется меньше битов для тэгов
 - Быстрый: может возвращать данные процессору параллельно с определением попадания или промаха
 - Поочерёдное обращение к словам из разных блоков, размещаемых в одной строке кэш (Conflict misses) – снижение вероятности попадания
- Полностью ассоциативное отображение (Fully-Associative)
 - Нет конфликтов (conflict misses) → общая высокая вероятность попадания
 - Требуется дорогая ассоциативная память – отдельный компаратор для каждой строки кэш

Слайд 14

В памяти с прямым отображением (direct mapped cache), каждому блоку памяти поставлена в соответствие отдельная позиция (строка)-памяти. Таким образом, конкретный блок может храниться только в конкретной строке - памяти. В

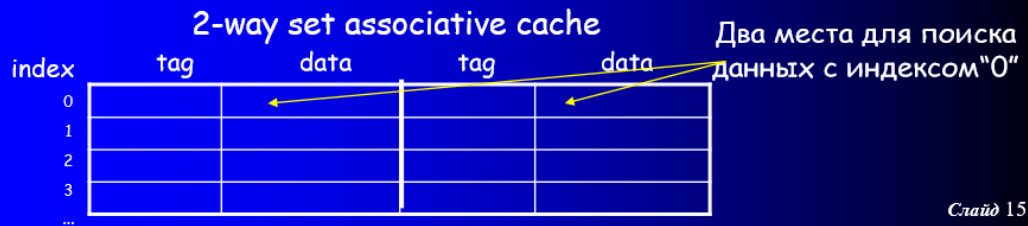
случае, если требуется записать блок данных из основной памяти в конкретную строку, то запись производится в любом случае – была данная строка занята или нет, несмотря на то, что соседние строки могут быть в это время свободными. Иными словами, замещение данных в памяти подобного типа и соответственно использование всей производится не самым рациональным способом.

Для того, чтобы определить какой блок памяти в какую строку записывать обычно используют несколько бит адреса для «задания» отображения блоков памяти. Например, биты 2 и 3 адреса (в зависимости от размера блока и строки) могут быть индексом, задающим номера строк.

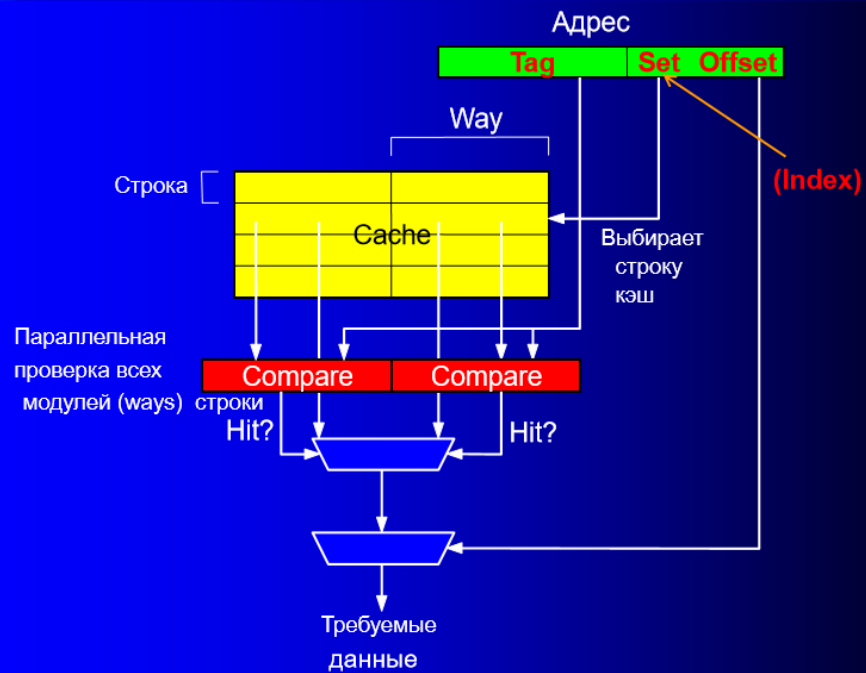
57. Кэш с множественно-ассоциативным отображением.

Множественно-ассоциативное отображение (k-way set associative cache)

- Direct mapped кэш проще:
 - Меньше аппаратуры; более быстр в потенциале
- Fully associative - меньше промахов и конфликтов.
- Кэш, совмещающий преимущества обоих способов:
 - Индекс вычисляется из адреса
 - В "k-way set associative cache" индекс определяет множество k модулей кэш, где могут храниться данные (т.е. k вариантов размещения для каждого блока данных ОЗУ).
 - k=1 – прямое отображение (direct mapped).
 - k= размеру кэша (в строках) – полностью ассоциативное.
 - Используется замещение LRU (или другое) для всего набора.



Set-Associative Caches



Слайд 17

2-way set associative cache in action

Последовательность ячеек: 24, 20, 04, 12, 20, 44, 04, 24, 44

index	tag	data	tag	data
0	24 - 27	data	-	-
1	20 - 23	data	-	-
0	24 - 27	data	-	-
1	20 - 23	data	04 - 07	data
0	24 - 27	data	-	-
1	12 - 15	data	04 - 07	data
0	24 - 27	data	-	-
1	12 - 15	data	04 - 07	data

24 = 011000₂ ; индекс- 0.

20 = 010100₂ ; индекс- 1.
(index is bit 2 of address)

04 = 000100₂ ; индекс- 1.
(-во 2-ой slot of "01" set)

12 = 001100₂ ; индекс- 1.
(kicks out older item in "01" set)

your turn ...

20 = 010100₂

44 = 101100₂

04 = 000100₂

Слайд 16

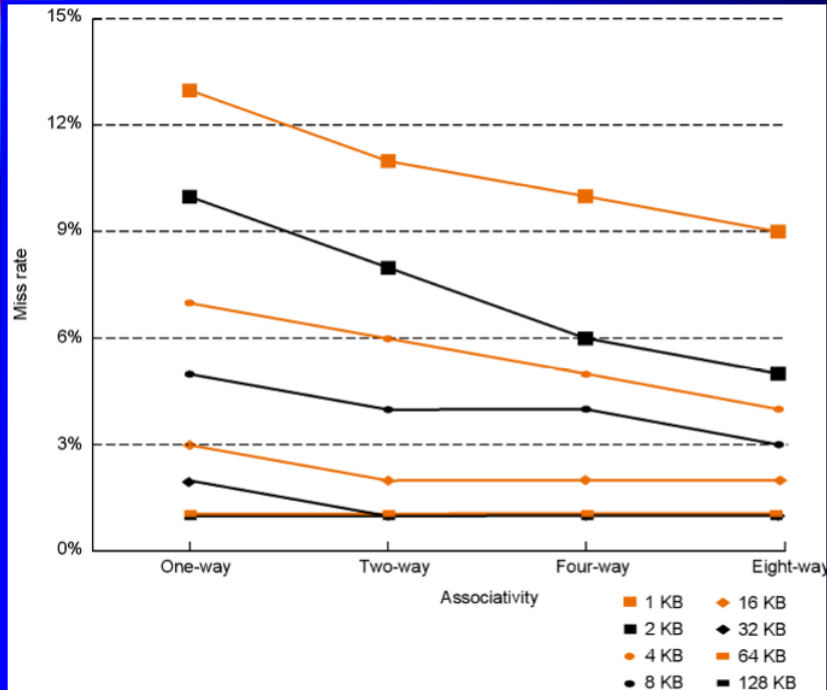
Для совмещения преимуществ обоих способов была разработана схема с множественно-ассоциативным отображением (k-way associative cache).

В "k-way set associative cache" индекс определяет множество наборов по k строк в каждом, где могут храниться данные. В зависимости от значения параметра k, данная организация может вырождаться в выше рассмотренные случаи:

- $k=1$ – полностью ассоциативное.
- k = количеству строк – прямое отображение (direct mapped).

58. Эффективность множественно-ассоциативного отображения. Размеры строки.

Эффективность множественно-ассоциативного отображения (cache associativity)



4-way cache – почти идентичная вероятность попадания как и у direct-mapped cache двойного размера

Слайд 18

Размеры блоков данных в кэш? (Cache Blocks)

tag	данные (большие блоки)

Плюсы:

- Большие блоки имеют преимущество в пространственной локальности.
- Меньше места требуется для тэгов (при заданном объёме кэш)

Минусы:

- Слишком большие блоки – неразумное использование пространства кэш (больше вероятность промаха).
- Большие блоки – больше времени для пересылки данных.

Слайд 19

Пример использования больших блоков в кэш

Последовательность ячеек: 24, 20, 04, 12, 20, 44, 04, 24, 44...

tag		8 Bytes of data	
index	0	-	-
	1	24 - 31	data

24 = 011000_2 ; индекс- 1.
(index is bit 3 of address)

index	0	16 - 23	data
	1	24 - 31	data

20 = 010100_2 ; индекс- 0.
(линия - Bytes 16-23 -
адреса линии кратны 8 байтам)

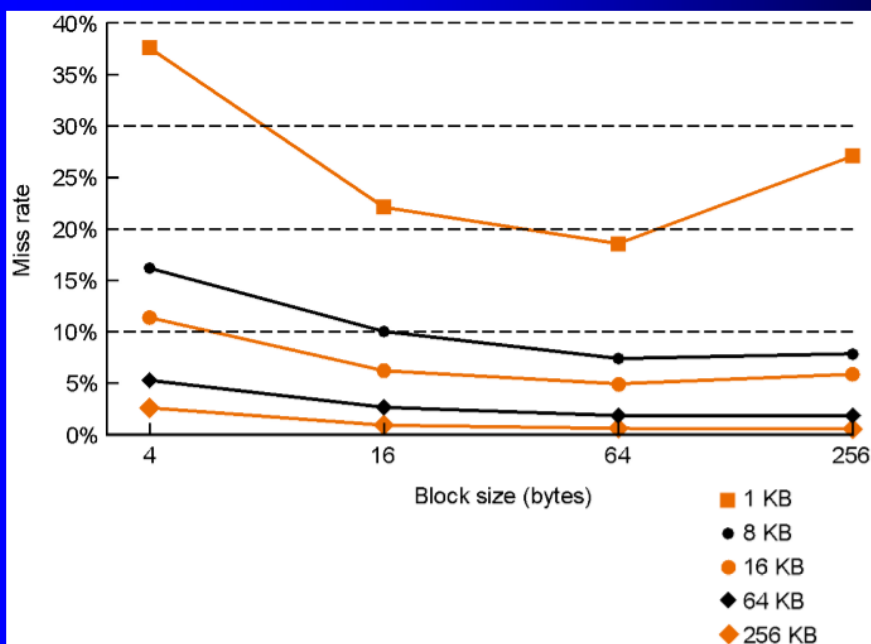
28 = 011100_2 ; индекс- 1.
Попадание - даже не смотря,
что до этого 28 не было!

index	0	16 - 23	data
	1	24 - 31	data

your turn ...
12 = 001100_2
08 = 001000_2
44 = 101100_2

Слайд 20

Block Size and Miss Rate

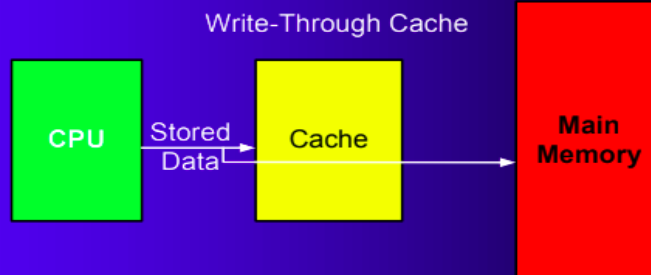


Rule of thumb: block size should be less than square root of cache size.

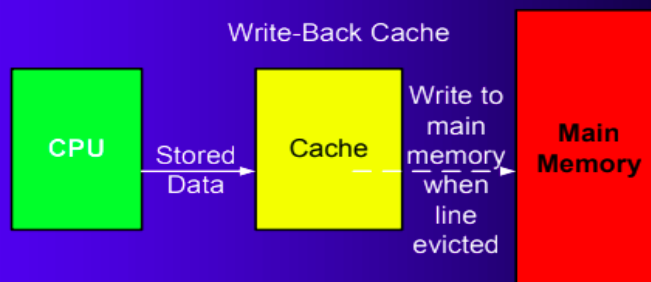
Слайд 21

Write-back vs. Write-Through Caches

Метод
сквозной
записи



Метод
обратной
записи



Слайд 10

Write-Through Cache

Write-through – прежде всего обновляется слово, хранящееся в основной памяти. Если в кэш есть копия слова, то она также обновляется, если копии нет – то из основной памяти в кэш пересылается слово (сквозная с отображением) либо нет (сквозная без отображения).

Главный плюс метода – **когда в строка кэш замещается другим блоком – не нужно обновлять память (нет задержки) – она уже обновлена. Прост в реализации.**

Но при записи будет потеря времени. Применялся в i80486.

Разновидность – буферизированная сквозная запись (**write behind**) – в кэш и FIFO, ЦП не ждёт пока FIFO пишет в основную память, т.е. ЦП с ОЗУ не работает вообще.

Слайд 11

Write-back Cache

Write-back – слово заносится только в кэш. Если строки в кэш нет, то сначала загрузка строки в кэш, потом её изменение. При замещении строки – её запись в основную память.

Для метода характерно, что **при каждом чтении (после определённого этапа) из ОЗУ – две пересылки между ОЗУ и кэш памятью. Какие?**

Разновидность – флаговая обратная запись – если в строке изменение, то флаг = 1. При замещении запись в ОЗУ только если флаг=1.

Применялся в МП класса 486 и Pentium от Cyrix.

Слайд 12

Сравнения

- Write-back cache в среднем на 10% эффективнее
 - Может записывать сразу всю строку (объединяя несколько операций записи в одну)
 - Использует свойство локальности ссылок
- Write-through cache проще в реализации
 - Не требует флага – была строка записана или же нет
 - Не требует ожидания (пока появится место) на запись строки в ОЗУ при её замещении в кэш
 - Не требует специальных интерфейсов с I/O
- Системы виртуальной памяти предпочитают работать с write-back схемой из-за очень больших накладок записи на диск
 - Имеется тенденция перехода кэш на запись **write-back** как на более производительную.

Слайд 13

61. Гарвардская архитектура организации. Преимущества.

Что и как хранить в кэш?

Если любой вид информации (без различия – команды либо данные программы) хранятся в кэш:

- Это совмещённый (unified) кэш.
- Принстонская архитектура.

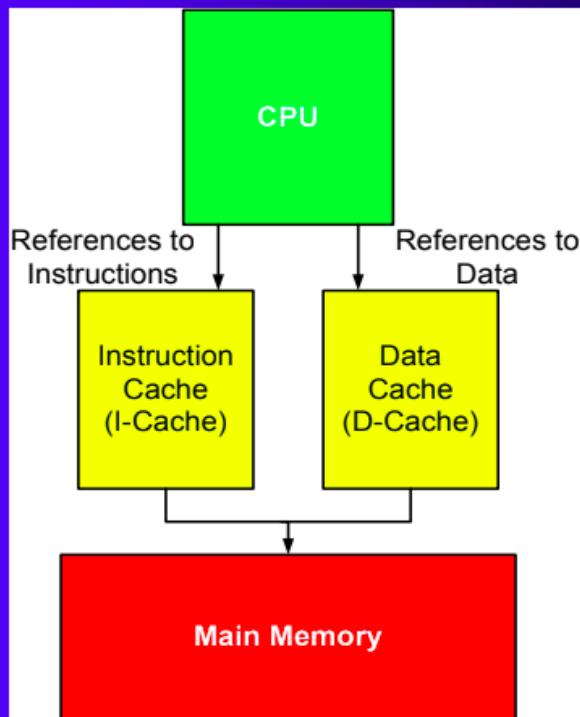
В последнее время чаще кэш разделяют – отдельно для команд и отдельно для данных

- Гарвардская архитектура

Совмещённая - проще, при заданной ёмкости – более вероятно попадание -> автоматический баланс между командами и данными, т.е. если в фрагменте программы больше команд, то и кэш будет насыщаться командами...

Слайд 16

Instruction and Data Caches



Слайд

Почему начинает преобладать гарвардская?

- Полоса пропускания: позволяет получать доступ параллельно к командам и данным
- Большинство программ не изменяют собственные команды
 - Самоизменяющиеся программы – слишком сложны, к тому же они значительно повышают нагрузку на оба кэша
- Кэш команд (I-Cache) может быть проще, чем кэш данных D-Cache, при условии, что команды никогда не перезаписываются (изменяются)
- Поток команд обладает высокой локальностью по ссылкам и может обладать более высоким коэф. попаданий на малом кэш
 - Data references never interfere with instruction references

Слайд 1

62. Выделенное (изолированное) адресное пространство СВВ, достоинства и недостатки.

Адресное пространство СВВ

В выделенном адресном пространстве для обращения к MBV применяются спец. команды и отдельная система адресов, что позволяет разделить шины для работы с памятью и шины в/вывода -> совмещать по времени обмен с памятью и MBV. Адресное прост-во ВМ может быть исп-но по прямому назначению.

Достоинства выделенного адресного пространства:

- адрес ПУ в команде в/в может быть коротким – кол-во ПУ намного меньше, чем ячеек ЗУ. Адрес короткий -> короткие команды ВВ -> простые дешифраторы.
- программы более наглядные – легче отладка. **IBM, Intel**
- разработка СВВ может проводиться отдельно от разраб-ки ОЗУ

Недостатки:

- ввод/вывод только через аккумулятор ЦП, для передачи из ПУ в РОН, если акк-р занят – четыре команды (сохр. акк-ра, ввод из ПУ, пересылка в РОН, восстановление аккумулятора).
- перед обработкой содержимого ПУ это содержимое надо переслать в ЦП (кэш не работает в принципе).

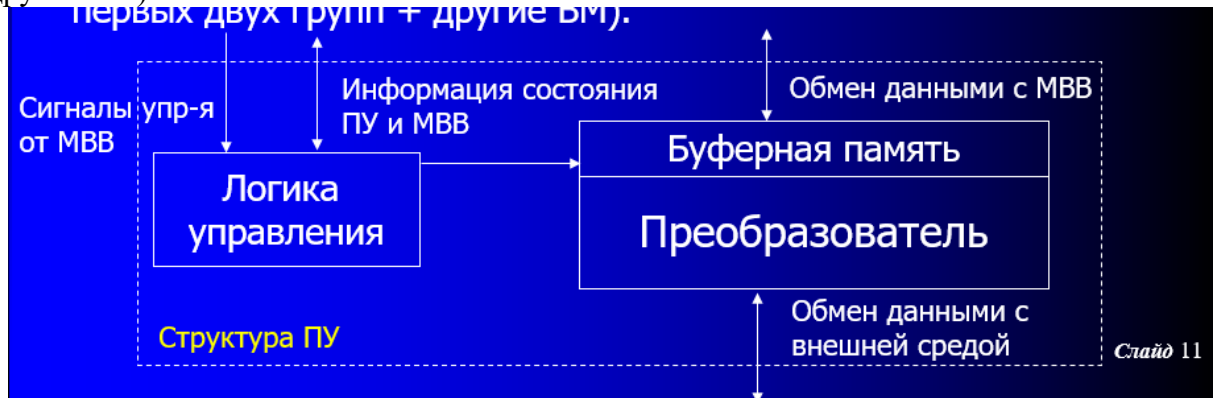
Слайд 10

63. Структура ПУ. Модули ввода-вывода.

Связь ВМ с внешним миром с пом. самых разнообразных ПУ. Каждое ПУ подключается к МВВ посредством индивидуальной шины. Интерфейс взаимодействия МВВ и ПУ называют «малым».

Всё множество ПУ – на три категории:

- для общения с пользователем (монитор, принтер, клавиатура...), для общения с ВМ (внешние ЗУ).
- стримеры, опт. диски, датчики, исполнительные механизмы)
- для связи с удалёнными устройствами (удалённые объекты первых двух групп + другие ВМ).



про модули ВВ читай в след. пункте.

64. Структура и функции модуля ввода/вывода.

Модуль ВВ отвечает за управление одним или несколькими ПУ и обмен данными между этими ПУ и ОЗУ с ЦП. Основные функции МВВ:

- Локализация данных
- Управление и синхронизация
- Обмен информацией
- Буферизация данных
- Обнаружение ошибок



Порт ввода вывода

Регистр ввода для приема входных данных от ПУ, регистр вывода для выдачи данных из ЦП в ПУ, регистр состояния ПУ, регистр управления (регистр режима) схема управления и селекции ПУ

Локализация данных – возм. обращения к одному из ПУ и адресации данных на нем.

Адрес – в адресной части команд ввода (in) вывода (out). СВВ может содержать несколько МВВ, каждый МВВ имеет свой диапазон адресов.

Старшие разряды – выбор МВВ в рамках системы ВВ. Младшие разряды – адреса регистров модуля либо подключенных к нему ПУ.

МВВ проверяет поступивший по ША адрес на попадание в «его» диапазон. Если попал – дешифрация адреса, перенаправление информации к/от адресуемому объекту.

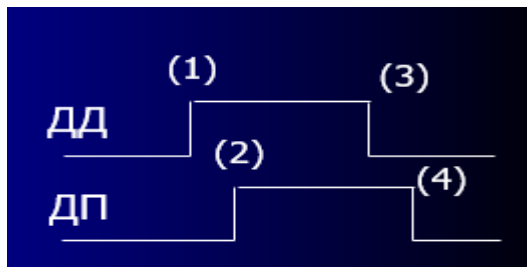
Для простейших устройств (клавиатура, принтер и т.п.) адрес ПУ однозначно определяет и расположение данных на этом устройстве. Для более сложных ВУ (например, внешние ЗУ) информация о нахождении данных требует детализации – например, номер записи (для ленты), либо номер цилиндра (для диска). Эта часть информации пересылается не в адресе, а в виде служебных сообщений по ШД. Обработка данной информации в модуле сложнее, чем выбор регистра или ПУ. Например, МВВ будет вынужден сам организовать поиск информации.

Управление и синхронизация – МВВ должен координировать транзакции данных между ВМ и ПУ. Следует учитывать след. факторы:

ЦП может взаимодействовать с несколькими ПУ, быстроедействие разных ПУ очень сильно отличается, если используются шины – то каждый обмен между ЦП и МВВ требует одну или несколько процедур арбитража шины.

Ввод/вывод – не синхронная операция => обмен сигналами подобно асинхронным шинам – «handshake». Для 2-сигн. системы – 4-ре шага:

1. ЦП с помощью сигнала ДД=1 (данные достоверны) сообщает о доступности данных, подлежащих выводу.
2. Приняв данные, устройство сообщает ЦП об их получении сигналом ДП=1.
3. Получив подтверждение ДП, ЦП обнуляет ДД и снимает данные с ШД.
4. Обнаружив, что ДД=0, устройство вывода снимает ДП, обрабатывает полученные данные и переходит в состояние готовности получения новой порции данных.



Обмен информацией – основная функция МВВ. Со стороны «большого» интерфейса – с ЦП, со стороны «малого» - с ВУ. Типовая последовательность операций ЦП при вводе/выводе:

1. Выбор требуемого периферийного устройства
2. Определение состояния МВВ и ПУ
3. Выдача указания МВВ на подключение нужного ПУ к процессору
4. Получение подтверждения на подключение ПУ
5. Распознавание сигнала готовности устр-ва к передаче очередного пакета инф.
6. Приём (передача) информации
7. Цикл п. 5 и 6 до завершения передачи информации в полном объёме
8. Логическое отсоединение ПУ от процессора.

Функция обмена информацией с ЦП включает в себя:

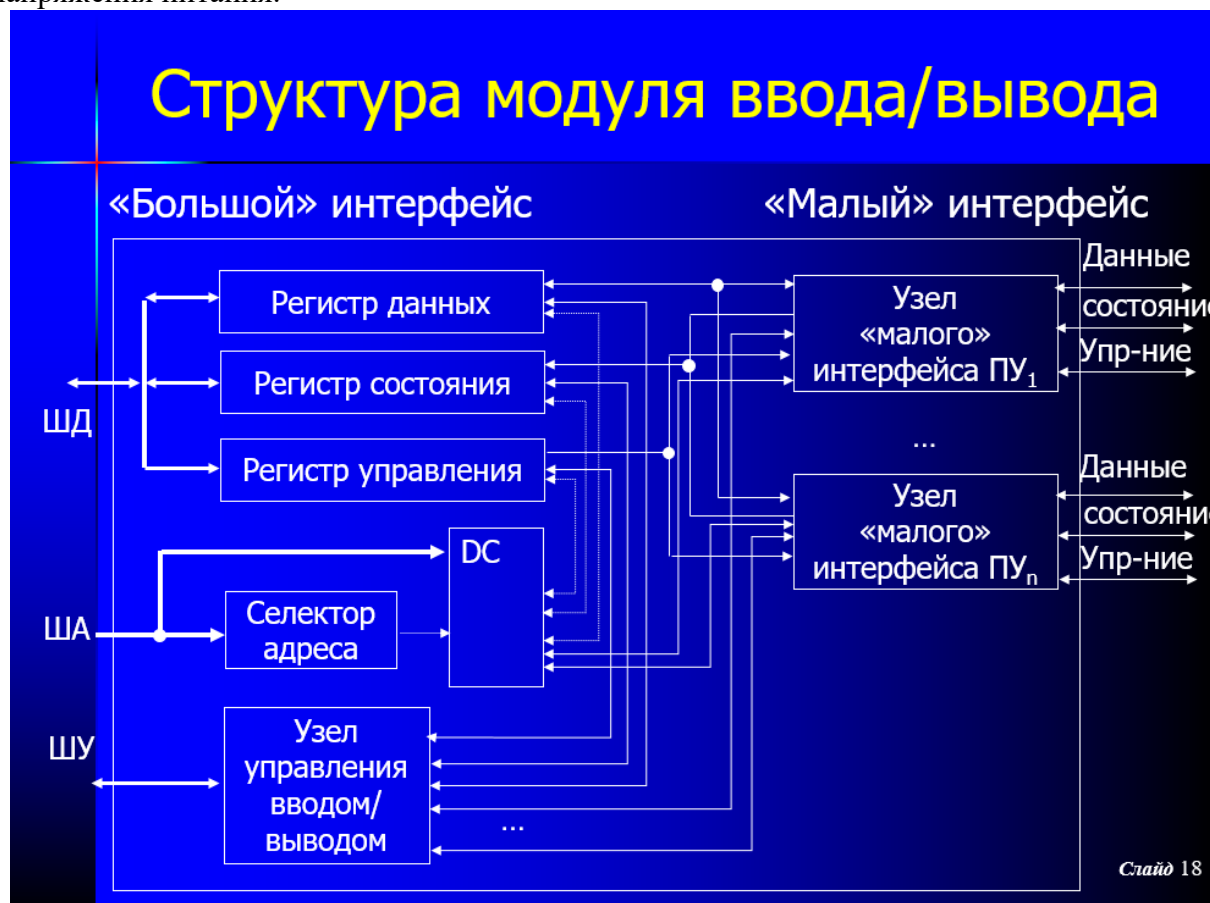
1. Дешифрацию команды (полученной по ШУ)
2. Пересылку данных между МВВ и ЦП по ШД
3. Извещение о состоянии ПУ
4. Распознавание адреса ПУ

Буферизация данных – различие в скорости обмена разных ПУ компенсируется буферизацией. MBV должен уметь работать со скоростью ОЗУ и самого медленного ПУ, к нему подключенного.

Обнаружение ошибок – одна из важнейших функций MBV. Модуль должен оповещать ЦП в каждом случае обнаружения ошибки. Причины возникновения ошибок:

- воздействие внешней среды, старение элементной базы (механика – износ, электроника – изменение сигналов), системное ПО, пользовательское ПО.

Самые частые причины – загрязнение и влага (оптика и механика), температура окружающей среды (электроника, механика), электромагнитное воздействие, скачки напряжения питания.



Пояснение к схеме:

Данные передаваемые в и из MBV – буферизируются в регистре данных (разрядность = ШД). Большинство ПУ ориентировано на побайтовый обмен информацией -> узел упаковки/распаковки. В случае нескольких ПУ – РД тоже может быть несколько.

Регистр управления – фиксирует поступившие из ЦП команды. В сложных MBV – РУ несколько – один для всего модуля и + для каждого из ПУ.

Регистр состояния – хранение битов состояния (готовность, занятость, offline) MBV и подключенных к нему ПУ. Может быть несколько регистров.

СВВ предусматривает работу с каждым регистром MBV или ПУ по отдельности – определяется системой адресации.

Узел управления – координация работы всех составляющих MBV и обеспечение взаимодействия с ЦП. Сигналы синхронизации, состояния, прерывания, сигналы арбитража и т.п.

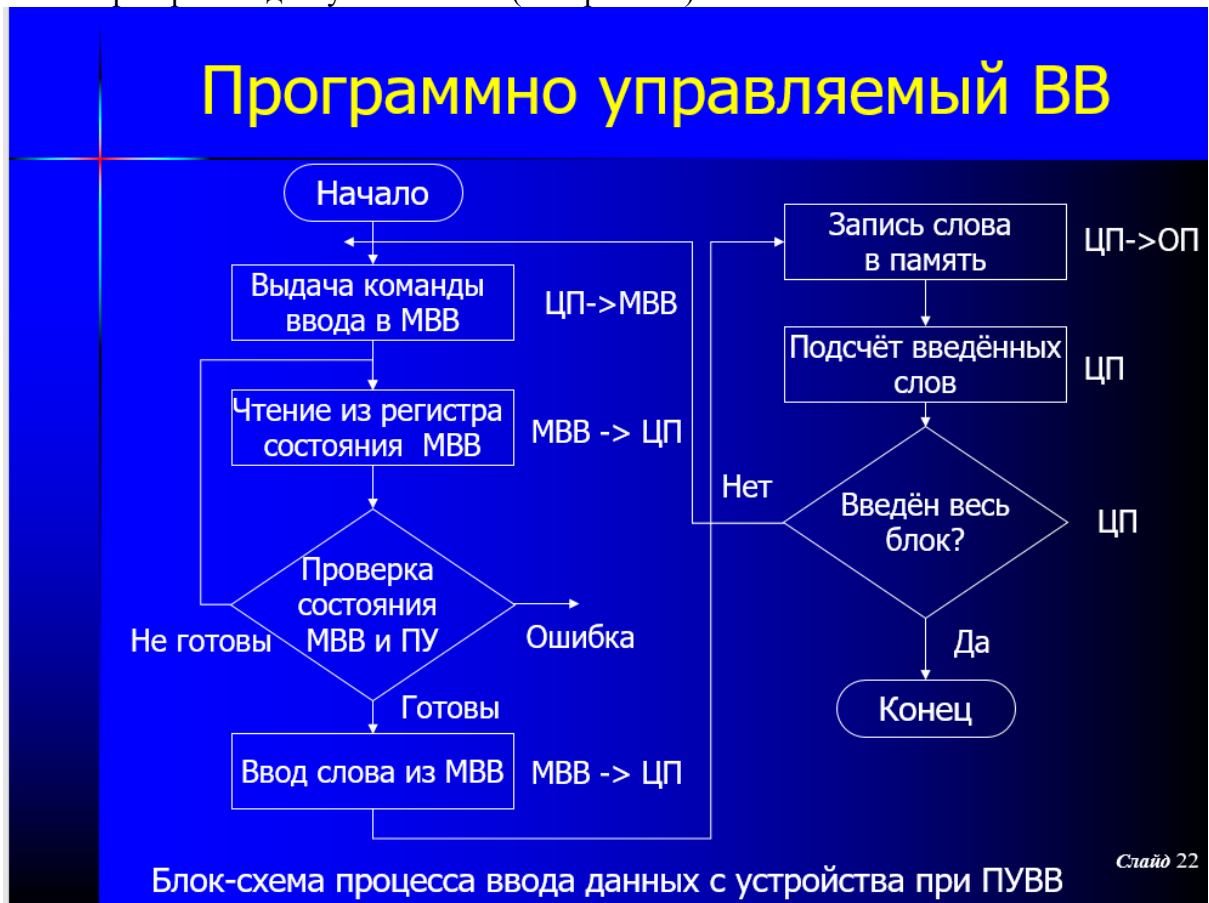
Малый интерфейс – подключение ПУ и взаимодействия с ними. ПУ подгоняются под один из стандартных протоколов. Каждое из ПУ обслуживается своим узлом малого интерфейса, который реализует принятый для данного ПУ стандартный протокол взаимодействия.

Основная задача MBV – освободить ЦП от необходимости знания деталей конкретных ПУ, с тем, чтобы ЦП мог управлять ПУ с помощью простых команд чтения/записи. MBV при этом берёт на себя задачи синхронизации, согласования форматов данных и т.п.

65. Программно-управляемый ВВ.

Способы обмена:

- программный (не форсированный)
- по прерываниям (форсированный)
- при прямом доступе к памяти (аппаратный)



Описание схемы:

Программный ввод-вывод:

Четыре типа команды, которые может получить MBV:

1. управление (активизация ПУ и указание требуемой операции – например, для стримера - перемотка или прокрутка вперед на одну запись)
2. проверка (включено ли, готово ли к работе устройство, завершена ли последняя операция, были ли ошибки и т.п.)
3. чтение (получить элемент данных из ПУ и занести его в регистр данных, а затем на ШД)
4. запись (принять элемент данных с ШД и переслать его в РД с последующей записью в ПУ).

Минусы способа: неэффективный простой процессора из-за ожидания готовности очередной порции информации. Пересылка даже одного слова требует несколько команд.

Плюсы способа: Простота, лёгкость изменения приоритета работы с ВУ (определяется процессором), подключение к СВВ новых ПУ и отключение старых – без сложностей.

66. Ввод/вывод по прерываниям.



Описание схемы:

Ввод/вывод по прерываниям: эффективнее программно управляемого ВВ – устраняет ненужные циклы ожидания, однако обработка прерывания требует дополнительного времени. Ещё один минус – пересылка всех данных идёт через процессор.

Два вопроса – идентификация устройства, которое инициировало прерывание, и арбитраж запросов.

Идентификация возможна тремя путями:

- множественные линии прерывания
- программная идентификация
- векторное прерывание.

Множественные линии прерывания – решение задачи «в лоб», простое, но не совсем рациональное – много управляющих линий.

Программная идентификация – после обнаружения запроса на прерывание, ЦП переходит к общей программе прерывания, опрашивающей MBV и выясняющей, кто был инициатором запроса. Может быть реализован с помощью отдельной линии опроса либо с помощью спец. регистров в MBV. Недостаток метода – потери времени на опрос

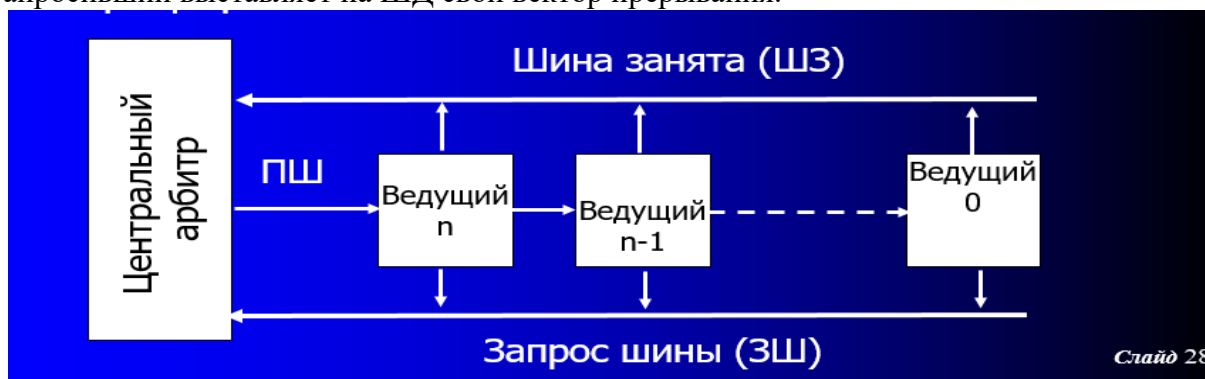
Наиболее эффективный вариант – аппаратная реализация векторного прерывания. Получив подтверждение прерывания от ЦП, MBV выставляет на ШД спец. слово – вектор прерывания.

Получив подтверждение прерывания от ЦП, MBV выставляет на ШД спец. слово – вектор прерывания.

Слово содержит либо адрес MBV либо другой уникальный идентификатор, который может быть указателем на соответствующую процедуру обработки прерывания. Реализация – с помощью таблицы векторов прерывания. Вход в таблицу – вектор прерывания. Начальный адрес таблицы задан неявно, т.е. под таблицу отводится определённая область памяти.

Два варианта реализации векторной идентификации источника прерывания – цепочечный опрос и арбитраж шины.

Цепочный опрос – для передачи запроса на прерывание используется общая линия. Линия подтверждения – последовательно проходит через все МВВ. Подтверждение прерывания посылается через ЦП, и последовательно доходит до инициатора прерывания. Запросивший выставляет на ШД свой вектор прерывания.



Слайд 28

Арбитраж шины – прежде чем выставит на ШД свой вектор, МВВ должен получить право на управление шиной -> в каждый момент времени только один МВВ может активизировать линию запроса прерывания.

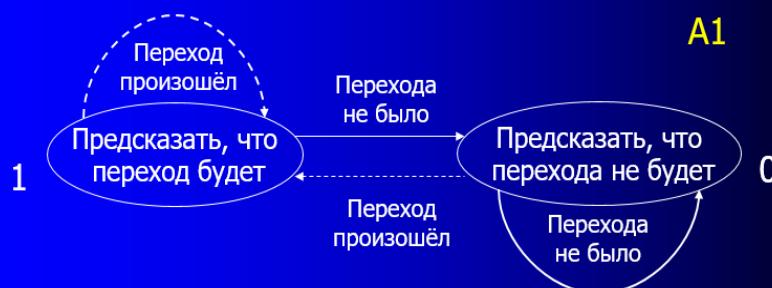
ЦП подтверждает запрос и МВВ выставляет вектор прерывания на шину данных.

Минусы ВВ по прерываниям: темп передачи ограничен скоростью, с которой ЦП в состоянии опросить и обслужить устройство, ЦП вовлечён в управление передачей, для каждой пересылки он должен выполнить определённое количество команд.

67. Динамические схемы предсказания: А1.

Динамическое предсказание – предполагает накопление информации об исходе предшествующих команд УП. История переходов – в виде таблицы, каждый элемент – m бит. Нужный элемент таблицы указывается с помощью k -разрядной двоичной комбинации – шаблона. Отсюда название – таблица истории для шаблонов (РНТ, Pattern History Table). Динамические схемы предсказания – один из видов автомата Мура, содержимое элементов РНТ – информация о текущем состоянии автомата. Три основных вида – А1, А2 и А3.

Динамическое предсказание переходов



А1 – два состояния, элемент РНТ – один бит ($m=1$), содержащий исход последнего выполнения команды УП.

Если команда завершилась переходом, то в соотв. элемент РНТ заносится 1, иначе – 0. Очередное предсказание совпадает с итогом предыдущего выполнения команды. После обработки очередной команды – содержимое элемента корректируется.

Слайд 43

68. Динамические схемы предсказания: A2.

Динамическое предсказание переходов

Динамическое предсказание – предполагает накопление информации об исходе предшествующих команд УП. История переходов – в виде таблицы, каждый элемент – m бит. Нужный элемент таблицы указывается с помощью k -разрядной двоичной комбинации – шаблона (pattern). Отсюда название – таблица истории для шаблонов (PHT, Pattern History Table).

Динамические схемы предсказания – один из видов автомата Мура, содержимое элементов PHT – информация о текущем состоянии автомата. Три основных вида – A1, A2 и A3.

Слайд 42

Динамическое предсказание переходов



A2 – пример из четырёх состояний, элемент PHT – два бита ($m=2$), содержащий исход последнего выполнения команды УП.

В A2 элементы PHT отражают результаты двух последних исходов команд УП (заполняются по схем регистра сдвига).

Слайд 44

69. Динамические схемы предсказания: А3.

Динамическое предсказание переходов

Динамическое предсказание – предполагает накопление информации об исходе предшествующих команд УП. История переходов – в виде таблицы, каждый элемент – m бит. Нужный элемент таблицы указывается с помощью k -разрядной двоичной комбинации – шаблона (pattern). Отсюда название – таблица истории для шаблонов (PHT, Pattern History Table).

Динамические схемы предсказания – один из видов автомата Мура, содержащее элементы PHT – информация о текущем состоянии автомата. Три основных вида – А1, А2 и А3.

Слайд 42

Динамическое предсказание переходов



Предсказание – старший разряд счётчика.

Элементы PHT в автомате А3 – реверсивные счётчики, работающие в режиме с насыщением.

Команда УП на конвейер – обращение к соотв. элементу PHT – если команда завершилась переходом +1, если нет -1. При насыщении переход за верхнюю 11 или нижнюю 00 границу не возможен.

Слайд 46

Динамическое предсказание переходов

Ожидания, что при увеличении m точность предсказания будет возрастать, на практике не подтверждаются. При $m > 3$ точность предсказания начинает снижаться.

Различия в точности при $m=2$ и $m=3$ незначительные (десятичные процента) -> в большинстве процессоров используются двухразрядные счётчики.

Логика предсказания для $m=2$ – алгоритм Смита (A3).

Четыре состояния:

00 – перехода не будет (сильное предсказание)

01 – перехода не будет (слабое предсказание)

10 – переход будет (слабое предсказание)

11 – переход будет (сильное предсказание)

Слайд 49

70. Адрес команды условного перехода в качестве шаблона доступа к таблице предыстории переходов (PHT).

Динамическое предсказание переходов

Динамическое предсказание – предполагает накопление информации об исходе предшествующих команд УП. История переходов – в виде таблицы, каждый элемент – m бит. Нужный элемент таблицы указывается с помощью k -разрядной двоичной комбинации – шаблона (pattern). Отсюда название – таблица истории для шаблонов (PHT, Pattern History Table).

Динамические схемы предсказания – один из видов автомата Мура, содержимое элементов PHT – информация о текущем состоянии автомата. Три основных вида – A1, A2 и A3.

Слайд 42

Динамическое предсказание переходов

Особенности использования таблицы (какая именно информация выступает в роли шаблона для доступа к РНТ и какого рода история фиксируется в элементах таблицы) составляют различия в алгоритмах динамического предсказания переходов:

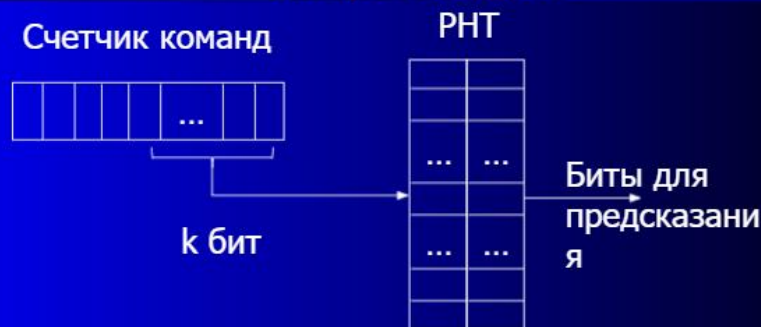
В качестве шаблонов для доступа к РНТ могут быть:

- адрес команды условного перехода
- регистр глобальной истории
- регистр локальной истории
- комбинация указанных вариантов.

1. При выборе адреса команды УП в качестве шаблона, можно учитывать поведение каждой конкретной команды УП. При многократном повторении большинства команд УП наблюдается повторяемость исхода – либо переход, либо нет. Данный способ позволяет отсортировать все команды и повысить точность.

Слайд 50

Динамическое предсказание переходов



При k-разрядном индексе таблица содержит 2^k элементов.

Схема обеспечивает высокую точность для многократно повторяющихся команд (циклов), но для многих команд (однократные либо редко выполняющиеся) – выполнение перехода коррелирует с поведением предшествующих команд. Данная схема не позволяет отслеживать подобные взаимосвязи, в отличие от схемы с регистром глобальной истории.

Слайд 51

71.Регистр глобальной истории в качестве шаблона доступа к таблице предыстории переходов (PHT).



P.S. Про саму РHT чуть выше

72.Регистр локальной истории в качестве шаблона доступа к таблице предыстории переходов (PHT).

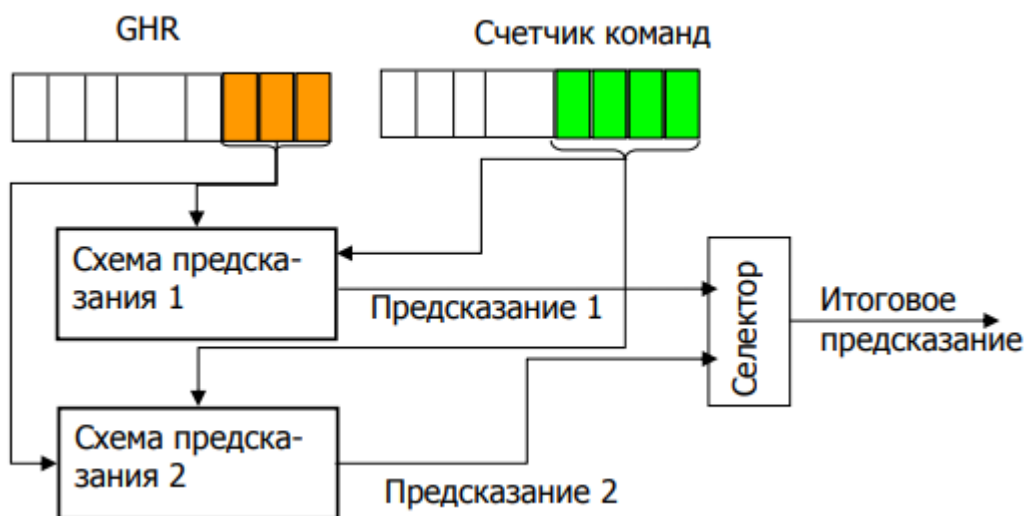
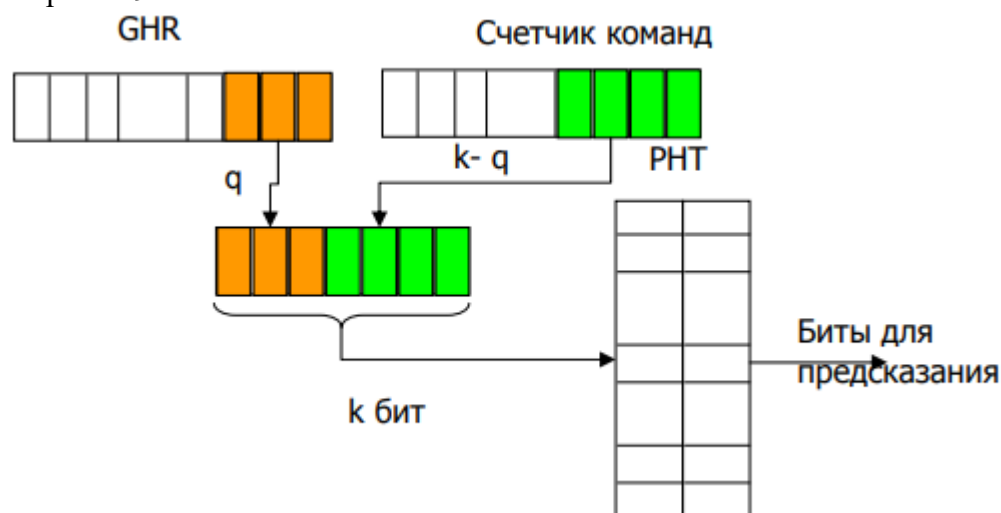


P.S. Про саму РHT чуть выше

73. Комбинационный шаблон доступа к таблице предыстории переходов (PHT).

Лучшие результаты по точности предсказания показывают схемы, комбинирующие оба приведённых выше подхода. На рис. 4.8 приведён пример, демонстрирующий подобную комбинацию.

При разработке реальных схем предсказания исходов условных переходов инженера сталкиваются с так называемой проблемой «холодного старта». Она заключается в том, что до тех пор, пока таблица истории шаблонов не будет заполнена реальными данными, схема не будет предсказывать переходы с необходимой точностью. Для решения данной проблемы были разработаны двух уровневые схемы предсказания, суть которых сводится к использованию нескольких принципиально отличающихся схем предсказания переходов, которые дают различную точность работы на этапе насыщения схем реальными данными и в режиме штатной работы. Решение о выборе предсказания конкретной схемы принимается селектором, который учитывает положительные результаты предсказания каждой из схем и выбирает ту, которая в последние N раз ошибалась меньше. Пример подобной схемы приведён на рис. 4.9.



74. Параллелизм. Уровни параллелизма.

Параллелизм на уровне команд (англ. *Instruction-level parallelism* — *ILP*) является мерой того, какое множество операций в компьютерной программе может выполняться одновременно. Потенциальное совмещение выполнения команд называется «параллелизмом на уровне команд».

Есть два подхода к выявлению параллелизма (*parallelism extraction*) на уровне команд:

- аппаратные средства — выявлением параллелизма в потоке операций занимаются специальные схемы процессора при исполнении кода программ;
- программное обеспечение — выявлением параллелизма занимается компилятор, который формирует исполняемый код программы под специальный процессор.

Уровень аппаратного обеспечения осуществляет динамический параллелизм, тогда как уровень программного обеспечения реализует статический параллелизм. Современные высокопроизводительные процессоры x86 (Intel Pentium 4, Intel Core) работают на динамической последовательности параллельного выполнения (Внеочередное исполнение команд). Статический параллелизм применялся в процессорах Itanium (однако в Itanium 2 применяется гибридный подход).

Рассмотрим следующую программу:

1. $e = a + b$
2. $f = c + d$
3. $m = e * f$

Операция 3 зависит от результатов операций 1 и 2, поэтому она не может быть вычислена, пока не будут завершены 1 и 2. Однако операции 1 и 2 не зависят от других операций, поэтому они могут быть вычислены одновременно. Если предположить, что каждая операция может быть завершена за одну единицу времени, то эти три инструкции могут быть завершены в общей сложности за две единицы времени, обеспечивая параллелизм равный 3/2.

Цель разработчиков компилятора и процессора заключается в выявлении параллелизма и получении от него максимального выигрыша. Обычные программы, как правило, написаны под последовательную модель исполнения, где команды выполняются одна за другой в порядке, установленном программистом. ILP позволяет компилятору и/или процессору распараллеливать выполнение нескольких инструкций или даже изменять порядок их выполнения.

Сколько ILP присутствует в программе сильно зависит от области её применения. В некоторых областях, таких как компьютерная графика и научные вычисления, число может быть очень большим. Тем не менее такие задачи как криптография могут демонстрировать гораздо меньше параллелизма.

Аппаратные методы выявления и использования параллелизма:

- Вычислительный конвейер, где выполнение нескольких инструкций может частично перекрываться;
- Суперскалярное выполнение операций, в которой несколько функциональных блоков используется для выполнения нескольких команд одновременно;
- Внеочередное исполнение, где инструкции выполняются в любом порядке, который не нарушает зависимости данных. Обратите внимание, что этот метод не зависит ни от конвейера, ни от суперскалярного выполнения. Текущие реализации внеочередного исполнения динамически (то есть в время выполнения программы и без помощи со стороны компилятора) получают ILP из обычных программ. Альтернативой является получение параллелизма во время компиляции и передача этой информации аппаратному обеспечению. Из-за сложности масштабирования техники внеочередного исполнения отрасль пересмотрела набор команд, которые явно кодируют несколько независимых операций в инструкции;
- Переименование регистров — метод, используемый для устранения ненужной сериализации (последовательного выполнения) операций, к которой приводит переиспользование регистров этими операциями, и используемый для внеочередного исполнения;

- Спекулятивное исполнение, которое позволяет выполнять инструкции целиком или частично до того, как станет ясно, нужно ли это выполнение. Часто используемой формой спекулятивного исполнения является спекулятивное исполнение потока управления, при котором инструкции (например, переход) за потоком управления выполняются до того, как определится та ветка программы, которую определяет инструкция перехода. Были предложены и используются несколько других форм спекулятивного исполнения, в том числе для спекулятивного исполнения, обусловленного предсказанием значения, предсказание зависимости по памяти и предсказание задержек;
- Предсказание переходов, которое используется, чтобы избежать простаивания для разрешения управления зависимостями. Предсказатель переходов используется со спекулятивным исполнением.

Потоковые архитектуры представляют собой другой класс архитектур, где ILP явно указаны, см., например, архитектуру TRIPS.

75. Классификация Флинна.

Классификация Флинна — общая классификация архитектур ЭВМ по признакам наличия параллелизма в потоках команд и данных

Всё разнообразие архитектур ЭВМ в этой классификации Флинна сводится к четырём классам:

- ОКОД — Вычислительная система с одиночным потоком команд и одиночным потоком данных (SISD, single instruction stream over a single data stream).
- ОКМД — Вычислительная система с одиночным потоком команд и множественным потоком данных (SIMD, single instruction, multiple data).
- МКОД — Вычислительная система со множественным потоком команд и одиночным потоком данных (MISD, multiple instruction, single data).
- МКМД — Вычислительная система со множественным потоком команд и множественным потоком данных (MIMD, multiple instruction, multiple data).

Так как в таксономии в качестве основного критерия используется параллелизм, то таксономия Флинна наиболее часто упоминается в технической литературе при классификации параллельных вычислительных систем. МКОД (MISD) — редко используемая архитектура, в основном с целью защиты от сбоев (например, для горячего резервирования вычислительных машин в лётных системах типа «Space Shuttle» или «Энергия — Буран», в SCADA, критичных к сбоям, и т. п.). Поскольку SISD-машина параллельной машиной не является, а MISD не является типичной параллельной архитектурой, все параллельные вычислительные системы попадают в класс либо SIMD, либо в MIMD.

С развитием технологий классы SIMD и MIMD стали охватывать слишком большой круг машин, кардинально отличных друг от друга. В связи с этим в технической литературе^{[9][10]} используется дополнительный критерий — способ работы с памятью с точки зрения программиста. По этому критерию системы делятся на «системы с общей памятью» (англ. *shared memory*, SM) и «системы с распределенной памятью» (англ. *distributed memory*, DM). Соответственно, каждый класс — SIMD и MIMD — делится на подклассы: SM-SIMD/DM-SIMD и SM-MIMD/DM-MIMD.

Следует обратить особое внимание на уточнение «с точки зрения программиста». Дело в том, что существуют вычислительные системы, где память физически распределена по узлам системы, но для всех процессоров системы она вся видна как общее единое глобальное адресное пространство.

76. Конвейеризация вычислений. Линейные и нелинейные конвейеры. Основные ступени конвейеров команд.

Конвейеризация Pipelining

- Необходимы отдельные стадии (задачи) *Requires separable jobs/stages*
- Необходимы отдельные ресурсы *Requires separate resources*
- Позволяет «параллелить» без размножения аппаратуры *Achieves parallelism without replication*
- Улучшает производительность *Improves throughput*
- Часто увеличивает задержку по отдельной задаче *Often increases single-task (e.g., instruction, laundry load) latency*
- Эффективность конвейера критична для производительности *Pipeline efficiency (keeping the pipeline full) critical to performance*

Слайд 7

Конвейеризация вычислений



На практике: равные задержки для всех $\Phi Б_i$ редки.

В архитектуре ВМ множество объектов, использующих конвейеризацию (например, АЛУ, память). Наиболее ощутимый результат – конвейеризация этапов машинного цикла.

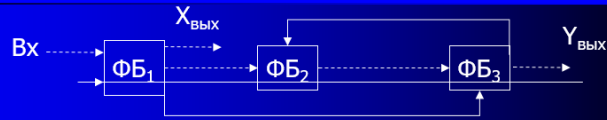
По способу синхронизации работы ступеней конвейера: синхронные и асинхронные.

Синхронные – ступени рядом, тракты распространения сигналов коротки – перекосов практически нет.

Асинхронные – связь м/у ступенями слабее, тракты длиннее. Пример – систолические массивы.

Слайд 8

Нелинейные конвейеры



В ряде ситуаций выгодно соединять блоки в соответствии с логикой обработки (пропуски, циклы и т.п.). В итоге можно вычислять более одной функции, но если они конфликтуют – то сложности с загрузкой конвейера.

Необходимо строить диаграммы однократной реализации функции и отслеживать моменты, не пересечения с второй функцией по обращению к одному блоку.

X	Y				X		X
	X		X				
		X		X		X	

ФБ ₁	X				X		X
ФБ ₂		X		X			
ФБ ₃			X		X		X

Y				Y			
		Y					
	Y		Y		Y		

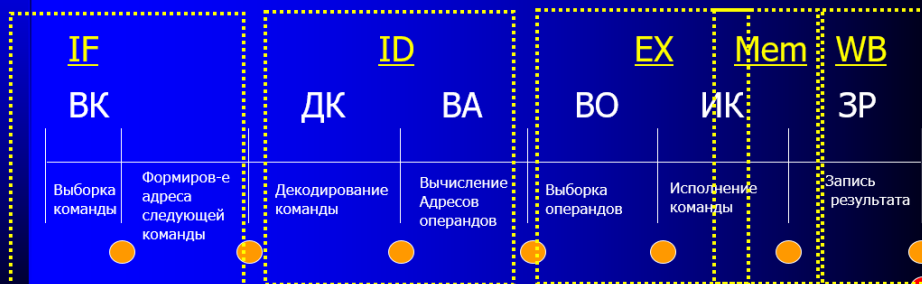
X		Y			X	Y	X
	X		X	Y			
		X	Y	X	Y	X	Y

Слайд 9

Конвейер команд

Ступени конвейера:

- IF: Instruction Fetch/Выборка команды
- ID: Instruction Decode (& register fetch & add PC+immed)/Декодирование команды
- EX: Execute/Выполнение команды
- Mem: Memory access/Работа с памятью
- WB: Write-Back into registers/Запись в регистр результата



Слайд 10

Пять ступеней

- Пять ступеней выполнения (для некоторых команд требуется меньше)

- IF: Instruction Fetch/Выборка команды
- ID: Instruction Decode (& register fetch & add PC+immed)/Декодирование команды
- EX: Execute/Выполнение команды
- Mem: Memory access/Работа с памятью
- WB: Write-Back into registers/Запись в регистр результата

	IF	ID	EX		Mem	WB	
	I cache	Decode, R-Read	ALU	PC update	D cache	R-Write	Total
R-type	1	1	.9	-	-	.8	3.7
Load	1	1	.9	-	1	.8	4.7
Store	1	1	.9	-	1	-	3.9
beq	1	1	.9	.1	-	-	3.0

Слайд 11

77. Программное и аппаратное устранение конфликтов по данным.

Риск по данным, в противоположность структурному риску — типичная и регулярная данным положим, что две команды в конвейере (i и j) предусматривают обращение к одной и той же переменной x, причем команда i предшествует команде j. В общем случае между i и j ожидаемы три типа конфликтов по данным:

- «Чтение после записи» (ЧПЗ): команда j читает x до того, как команда i успела записать новое значение x, то есть j ошибочно получит старое значение x вместо нового.
- «Запись после чтения» (ЗПЧ): команда j записывает новое значение x до того, как команда i успела прочитать x, то есть команда i ошибочно получит новое значение x вместо старого.
- «Запись после записи» (ЗПЗ): команда j записывает новое значение x прежде, чем команда i успела записать в качестве x свое значение, то есть x ошибочно содержит i-е значение x вместо j-го.

Возможен и четвертый случай, когда команда j читает x прежде команды i. Этот случай не вызывает никаких конфликтов, поскольку как i, так j получают верное значение x.

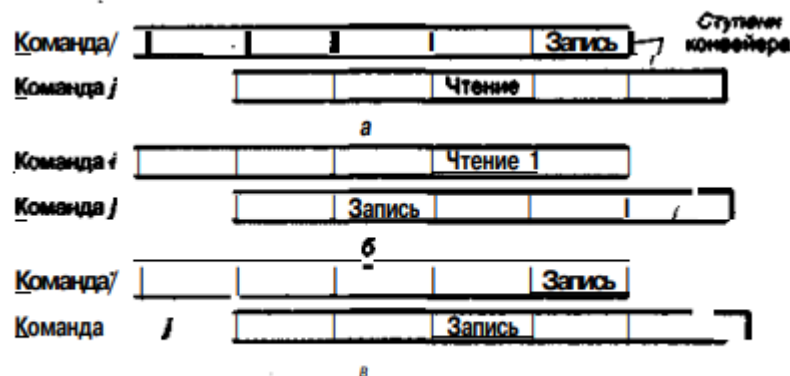


Рис- 9.4. Конфликты по данным: а - «Чтение после записи»; б - «Запись после чтения»; в - «Запись после записи»

Наиболее частый вид конфликтов по данным - ЧПЗ, поскольку операция чтения в цикле команды (этап ВО) предшествует операции записи (этап ЗР). По той же причине

конфликты типа ЗПЧ большой проблемы не представляют. Сложности появляются, только если структура конвейера допускает запись прежде чтения или если команды в конвейере обрабатываются в последовательности, отличной от предписанной программой. Такое возможно, если командам в конвейере разрешается «догонять» предшествующие им команды, приостановленные из-за какого-то конфликта. Конфликт типа ЗПЗ также не вызывает особых проблем в конвейерах, где команды следуют в порядке, определенном программой, и могут производить запись только на этапе ЗР. В худшем случае, когда одна команда догоняет другую из-за приостановки последней, имеет место конфликт по ресурсу — попытка одновременного доступа к одной и той же ячейке.

В борьбе с конфликтами по данным выделяют два аспекта: своевременное обнаружение потенциального конфликта и его устранение. Признаком возникновения конфликта по данным между двумя командами i и j служит невыполнение хотя бы одного из трех условий Бернштейна (Bernstein's Conditions):

- для ЧПЗ: $O_i \cap I(j) = \emptyset$;
- для ЗПЧ: $I(i) \cap O(j) = \emptyset$;
- для ЗПЗ: $I(i) \cap O(j) = \emptyset$.

Где $O(k)$ - множество ячеек, изменяемых командой k ; $I(l)$ - множество ячеек, читаемых командой l ; $\emptyset$ — пустое множество; $\cap$ — операция пересечения множеств.

Для борьбы с конфликтами по данным применяются как программные, так и аппаратные методы.

Программные методы ориентированы на устранение самой возможности конфликтов еще на стадии компиляции программы. Оптимизирующий компилятор пытается создать такой объектный код, чтобы между командами, склонными к конфликтам, находилась достаточное количество нейтральных в этом плане команд. Если такое не удастся, то между конфликтующими командами компилятор вставляет необходимое количество команд типа «Нет операции».

Фактическое разрешение конфликтов возлагается на аппаратные методы. Наиболее очевидным решением является остановка на несколько тактов с тем, чтобы команда i успела завершиться или, по крайней мере, миновать ступень конвейера, вызвавшую конфликт. Соответственно задерживаются и команды, следующие в конвейере за j -й командой. Данную ситуацию называют «пузырьком» в конвейере. Иногда приостанавливают только команду j , не задерживая следующие за ней команды. Это более эффективный прием, но его реализация усложняет конвейер.

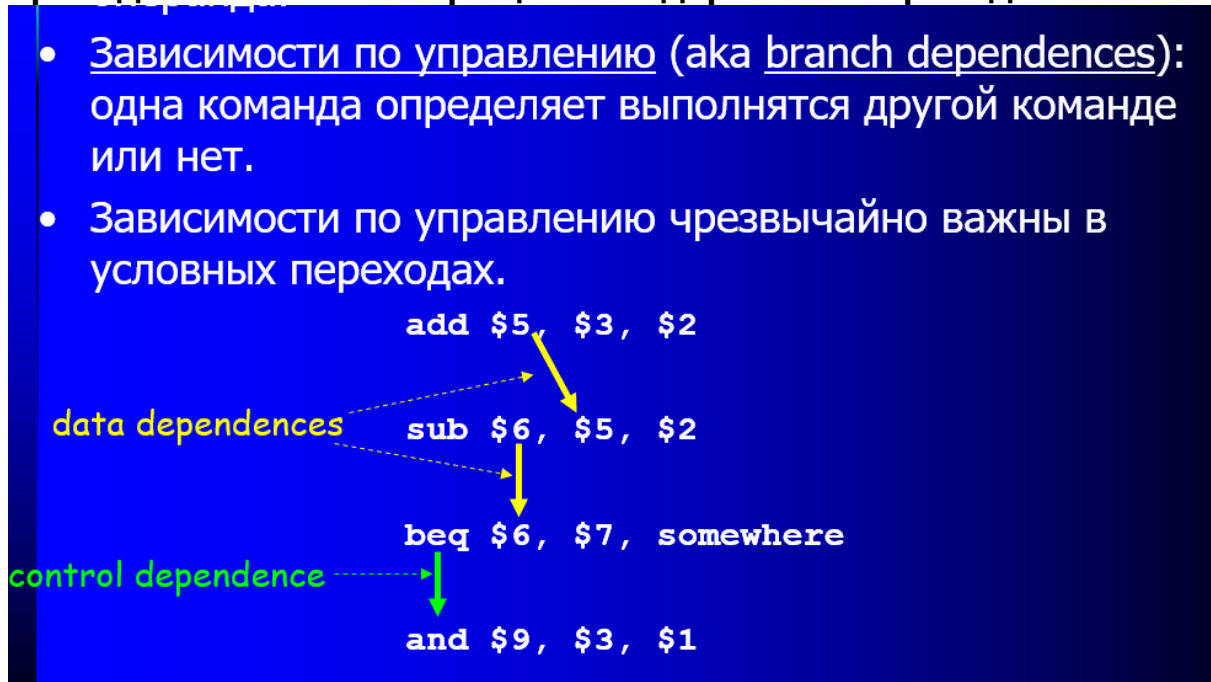
Понятно, что остановки конвейера снижают его эффективность и разработчики ВМ всячески стремятся сократить общее число остановок или хотя бы их длительность. Поскольку наиболее частые конфликты по данным - это ЧПЗ, основные усилия тратятся на противодействие именно этому типу конфликтов. Среди известных методов борьбы с ЧПЗ наибольшее распространение получил прием ускоренного продвижения информации (forwarding). Обычно между двумя соседними ступенями конвейера располагается буферный регистр, через который предшествующая ступень передает результат своей работы на последующую ступень, то есть передача информации возможна лишь между соседними ступенями конвейера. При ускоренном продвижении, когда для выполнения команды требуется операнд, уже вычисленный предыдущей командой, этот операнд может быть получен непосредственно из соответствующего буферного регистра, минуя все промежуточные ступени конвейера. С данной целью в конвейере предусматриваются дополнительные тракты пересылки информации (тракты опережения, тракты обхода), снабженные средствами мультиплексирования.

Конфликты типа RAW (Read After Write): команда j пытается прочитать операнд прежде, чем команда i запишет на это место свой результат. При этом команда j может получить некорректное старое значение операнда.

Устранение конфликтов по данным типов WAR и WAW достигается путем отказа от неупорядоченного исполнения команд, но чаще всего путем введения буфера восстановления последовательности команд.

Как отмечалось выше, наличие конфликтов приводит к значительному снижению производительности микропроцессора. Определенные типы конфликтов требуют приостановки конвейера. При этом останавливается выполнение всех команд, находящихся на различных стадиях обработки (до 20-ти команд в Pentium-4). Другие конфликты, например, при неверном предсказанном направлении перехода, ведут к необходимости полной перезагрузки конвейера. Потери будут тем больше, чем более длинный конвейер используется в микропроцессоре. Такая ситуация явилась одной из причин сокращения числа ступеней в микропроцессорах последних моделей.

78. Риски по управлению. Потери производительности на переходах. Способы сокращения издержек на переходы.



Зависимости по управлению могут «вылиться» в риски по управлению (aka control hazards) команд, которые сложно корректно обработать на конвейере.

Два фактора:

- реализация перехода (для любой команды перехода) выливается в потерю минимум двух тактов на вычисление нового адреса команды.
- издержки перехода для команд условного перехода – могут быть большими, чем 2 такта.

1. Выборка команды (ВК)	Издержки – потеря 4 тактов. В случае большего количества стадий конвейера – большие потери.
2. Декодирование команды (ДК)	
3. Вычисление адресов операндов (ВА)	
4. Выборка операндов (ВО)	
5. Исполнение команды (ИК)	
6. Запись результата (ЗР)	

Сокращение издержек на выборку команды из точки перехода:

1. Вычисление исполнительного адреса перехода на ступени декодирования команды;
2. Использование буфера адресов перехода;
3. Использование памяти для хранения команд, расположенных в точке перехода;

4. Использование буфера цикла.

1. При декодировании выясняется не только тип команды (переход или нет), но и способ адресации и ВЫЧИСЛЯЕТСЯ адрес точки перехода. Экономия одного такта при реализации перехода. Требуется дополнительных сумматоров, которые и вычисляют исполнительный адрес точки перехода.

2. Использование буфера адресов перехода;

Буфер адресов перехода (ВТА, Branch Target Buffer) – память небольшой ёмкости, в которой хранятся исполнительные адреса точек перехода нескольких последних команд, для которых переход имел место быть. В качестве тэгов – адреса соответствующих команд.

Перед выборкой команды – счётчик команд сравнивается с содержимым. Если в ней есть подобный адрес – то можно не вычислять исполнительный адрес команды перехода. Экономия одного такта.

Если в нет требуемого адреса, то - исполнение команды обычным образом. Если это оказалась команда перехода – занесение её и адреса точки перехода в (принцип замещения LRU).

Метод ВТА хорош для циклов. Обычно используется в составе более сложных схем по устранению конфликтов по управлению.

3. Использование памяти для хранения команд, расположенных в точке перехода;

-память команд, расположенных в точке перехода (ВТИ, Branch Target Instruction Cache) – усовершенствованный вариант ВТА. Память помимо исполнительного адреса перехода содержит ещё и код команды на которую совершается переход.

К экономии одного цикла как в ВТА прибавляется экономия одного цикла на выборку команды из памяти.

Максимальный эффект – при реализации программных циклов.

4. Использование буфера цикла.

Буфер цикла – маленькая быстродействующая память, входит в состав первой ступени конвейера (ВК). В буфере хранятся коды n последних команд в той последовательности, в которой они выбирались. Если имеет место переход, то сперва проверяется нет ли нужной команды в буфере, и если да – то команда извлекается из буфера.

Наиболее эффективен при реализации циклов и итераций. Если буфер достаточно велик, чтобы вместить всё тело цикла, то выборка команд – только в первый раз, остальные обращения – только к буферу.

По принципу – буфер похож на ВТИ, но хранит ещё и последовательность команд, меньше по ёмкости и дешевле.

79. Способы решения проблемы условного перехода

Нарушение размерности подачи команд на вход конвейера – память занята, либо команда состоит из нескольких слов. Для обеспечения ритмичности подачи – вводят буфер предвыборки (FIFO) между ступенью ВК и остальным конвейером. Буфер можно рассматривать как дополнительные ступени конвейера. Удлинение конвейера не сказывается на его производительности, увеличивается лишь время прохождения команды по конвейеру. Типичный буфер – на 2-8 команд.

Для решения проблемы условных переходов в конвейер включаются два таких буфера.

1. Буферы предвыборки

Каждая извлечённая из памяти и помещённая в основной буфер команда анализируется блоком перехода. При обнаружении условного перехода, блок вычисляет исполнительный адрес точки перехода и параллельно с продолжением работы основного буфера загружает команды начиная с точки перехода в дополнительный буфер. Если происходит переход, то основной буфер сбрасывается и подключается дополнительный.

Минусы буферов предвыборки:

- дублирование оборудования;
- если команды УП близко расположены в коде, то двух буферов будет недостаточно.

2. Множественные потоки – дублирование начальных ступеней конвейера и создание таким образом двух параллельных потоков команд.

В одной ветви – последовательность выполнения команд в случае неосуществления перехода, во втором – в случае осуществления.

Оба потока сходятся в точке, где итог проверки условия будет очевиден.

Дальнейшее продвижение по конвейеру – только «правильный» поток.

Основной минус – проблемы при близко расположенных в коде УП. Очередной УП поступает на ВК до момента, принятия окончательного решения по текущей команде перехода. Каждая подобная команда в итоге будет требовать отдельного потока.

3. Задержанный переход

Продолжение выполнения команд, следующих за командной УП, вне зависимости от её исхода.

Смысл имеется только в тех случаях, когда «доведённые до конца» команды являются «полезными», т. е. вычисляемыми в любом случае, вне зависимости от результата выполнения команды перехода.

Реализация стратегии – с помощью компилятора и оптимизатора.

Компилятор после каждой команды перехода вставляет NOP нужное количество раз, а оптимизатор – «перемешивает» команды таким образом, чтобы как можно больше NOP были бы заменены «полезными командами».

В оптимизированной программе удаётся заменить более 20% NOP.

80. Переименование регистров.

Основная проблема - устранение зависимостей команд по данным (ЧПЗ и ЗПЗ).

Решение - переименование регистров.

Упорядоченные выдача и завершение команд -> все регистры содержат нужные значения.

Неупорядоченные выдача и завершение -> неупорядоченная запись в регистры.

Пример:

I1: MUL R2, R0, R1 {R2<- R0*R1}

I2: ADD R0, R1, R2 {R0<- R1+R2}

I3: SUB R2, R0, R1 {R2<- R0-R1}

Неупорядоченная выдача/завершение провоцирует ситуации:

а) I2 исполнена до того, как I1 успела записать в R2 свой результат

б) I3 исполнена раньше, чем I1 -> в результате неверный результат будет получен в

I2, а по завершению в R2 результат I1, а не I3

Введём доп. регистры R0a и R2a

Новая последовательность:

I1: MUL R2, R0, R1 {R2<- R0*R1}

I2: ADD R0a, R1, R2 {R0a<- R1+R2}

I3: SUB R2a, R0a, R1 {R2<- R0a-R1}

Неупорядоченная выдача/завершение:

Конфликт б) устранён, а) - остался.

а) I2 исполнена до того, как I1 успела записать в R2 свой результат

б) I3 исполнена раньше, чем I1 -> в результате неверный результат будет получен в

I2, а по завершению в R2 результат I1, а не I3

Основная идея - каждый новый результат записывается в один из свободных в текущий момент доп. регистров, ссылки на заменяемый регистр во всех посл. командах соответствующим образом корректируются.

Два типа – логические регистры (видные программисту) и физически присутствующие в ЦП (АРФ).

Если физических регистров больше логических, то в определённый момент времени результат, записываемый в лог. регистр R_i будет сохранён в физ. регистре R_j . Все ссылки на R_i должны быть заменены на R_j .

Номера логических регистров динамически отображаются на номера физических – посредством таблиц подстановки (lookup table), обновляющиеся после декодирования каждой команды.

Если число логических и физических регистров одинаково, то используется буфер переименования (БП).

Буфер переименования – асс-е ЗУ

	ВЗ	Рг	Значение	ЗД	ПП
0					
1					
2					
...					
N-1					

ВЗ – вход занят. Рг – номер переименованного регистра.
ЗД – значение достоверно. ПП- последнее переименование

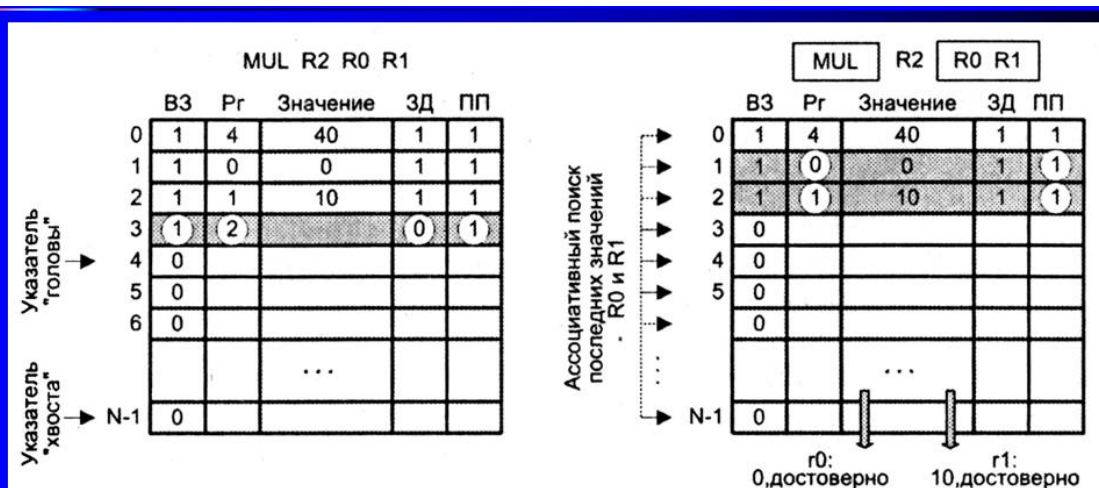
Аппаратный регистровый файл (АРФ)

	Значение	ЗД
0		
1		
...		
M-1		

	ВЗ	Рг	Значение	ЗД	ПП
0	1	4	40	1	1
1	1	0	0	1	1
2	1	1	10	1	1
3	0				
4	0				
5	0				
6	0				
...					
N-1	0				

	ВЗ	Рг	Значение	ЗД	ПП
0	1	4	40	1	1
1	1	0	0	1	1
2	1	1	10	1	0
3	1	1	15	1	1
4	0				
5	0				
6	0				
...					
N-1	0				

Пример многократного переименования регистра R1



Циклическое заполнение буфера – указатели головы и хвоста

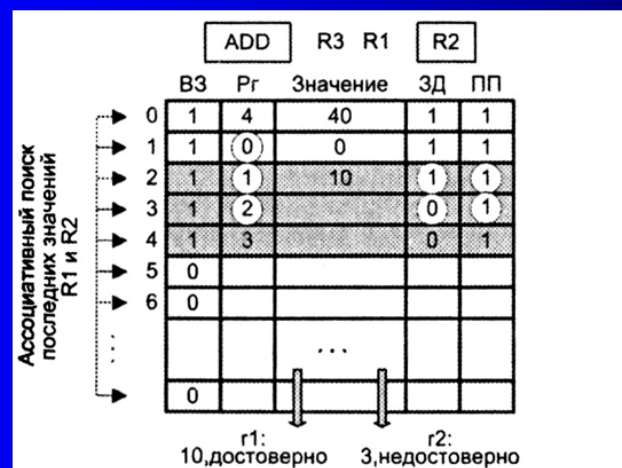
Выделение входа – V3 и ПП =1, Pr = номер регистра, ЗД=0

Пример (слева) – выделение входа для регистра R2

Пример справа – выборка значений регистров R0 и R1

Поиск по БП если операнды отсутствуют в АКФ.

Слайд 9



Первое, что делается при выполнении команды – выделение входа для результата команды.

Если значение регистра ещё не вычислено – передаётся номер входа и пометка (ЗД). R1- всё нормально, R2 (номер входа) – передаётся на исполнительную часть ЦП.

Слайд 10

SUB R2 R0 R1					
	ВЗ	Рг	Значение	ЗД	ПП
0	1	4	40	1	1
1	1	0	0	1	1
2	1	1	10	1	1
3	1	2		0	0
4	0	3		0	1
5	1	2		0	1
6	0				
			...		
N-1	0				

В БП несколько входов ссылаются на один регистр R2. SUB выдана до завершения команды MUL, – под R2 выделен вход 5, при этом поле ПП входа 3 изменено на 0 -> все остальные команды, использующие R2, будут брать значение из входа 5.

	ВЗ	Рг	Значение	ЗД	ПП
0	1	4	40	1	1
1	1	0	0	1	1
2	1	1	10	1	1
3	1	2	0	1	0
4	1	3		0	1
5	1	2		0	1
6	0				
			...		
0					

Результат 0
Вход 3 →

	ВЗ	Рг	Значение	ЗД	ПП
0	0	4	40	1	1
1	1	0	0	1	1
2	1	1	10	1	1
3	0			0	0
4	0	3		0	1
5	1	2		0	1
6	0				
			...		
N-1	0				

Освобождение
входа 0 →

По завершении MUL – модификация входа 3 (значения R2) – бит ЗД = 1.

Последний момент – освобождение входа буфера преобразования.

81. Идентификация устройств при вводе/выводе по прерываниям.

Идентификация возможна тремя путями:

- множественные линии прерывания
- программная идентификация
- векторное прерывание.

Множественные линии прерывания – решение задачи «в лоб», простое, но не совсем рациональное – много управляющих линий.

Программная идентификация – после обнаружения запроса на прерывание, ЦП переходит к общей программе прерывания, опрашивающей MBV и выясняющей, кто был

инициатором запроса. Может быть реализован с помощью отдельной линии опроса либо с помощью спец. регистров в МВВ. Недостаток метода – потери времени на опрос.

Наиболее эффективный вариант – аппаратная реализация векторного прерывания. Получив подтверждение прерывания от ЦП, МВВ выставляет на ШД спец. слово – вектор прерывания. Слово содержит либо адрес МВВ либо другой уникальный идентификатор, который может быть указателем на соответствующую процедуру обработки прерывания. Реализация – с помощью таблицы векторов прерывания. Вход в таблицу – вектор прерывания. Начальный адрес таблицы задан неявно, т.е. под таблицу отводится определённая область памяти.

82. Прямой доступ к памяти. Понятие и задачи КПДП.

Прямой доступ к памяти требует наличия на системной шине дополнительного модуля – КПДП, способного брать на себя функции ЦП по управлению системной шиной и обеспечивать передачу информации напрямую между ОП и ПУ.

Если ЦП желает прочитать или записать блок данных, то он помещает в КПДП инициализирующую информацию:

- вид запроса (чтение/запись)
- адреса устройства ввода/вывода
- адреса начальной ячейки блока памяти, откуда читать или куда писать
- количество слов, подлежащих чтению или записи.

Отправная точка – ОП, поэтому чтение – это считывание из ОП и запись в ПУ, запись – приём данных от ПУ и запись их в ОП.

Чтение (ОП->ПУ). КПДП выставляет на ША содержимое РА и формирует сигнал ЧтЗУ. Считанное из ОП слово помещается на ШД. КПДП выставляет на ША адрес ПУ и формирует сигнал Выв.

При записи (ПУ->ОП). КПДП сначала выставляет на ШД адрес устройства и формирует сигнал Вв, по которому введённые данные поступают на ШД. После чего, помещает на ША адрес ячейки ОП и выдаёт сигнал ЗпЗУ. По этому сигналу данные записываются в ОП.

В обоих случаях – буферизация пересылаемого слова в РД контроллера (необходимо для компенсации скоростей работы ОП и ПУ). Сигналы Выв и Вв формируются контроллером лишь при получении от ПУ сигнала готовности.

По завершении пересылки КПДП снимает «Запрос ПДП», в ответ ЦП снимает сигнал «Подтверждение ПДП».

Эффективность ПДП зависит от способа распределения системной шины между ЦП и КПДП в процесс пересылки блока:

Блочная пересылка (КПДП полностью захватывает СШ, на весь период ЦП не имеет доступа к шине)

Пропуск цикла (после передачи каждого слова на один цикл КПДП освобождает системную шину)

Прозрачный режим (КПДП имеет доступ к системной шине, только в тех циклах, когда ЦП в ней не нуждается).

Прозрачный режим – наиболее эффективен для ЦП, но существенно замедляет пересылку блока данных.

КПДП вправе захватить шину в пределах цикла команды (в отличие от прерывания ЦП не нужно запоминать контекст задачи)

83. Переупорядочивание команд с использованием технологии «шелвинга»

После декодирования команд и переименования регистров команды передаются на исполнение. Поскольку порядок выполнения команд может отличаться от предписанного

программой, необходимо обеспечить корректность их операндов (частично решается путем переименования регистров) и правильную последовательность занесения результатов в регистры АРФ. Одним из наиболее распространенных приемов решения этих проблем служит *переупорядочивание команд*. В его основе лежат использование *окна команд* — буферной памяти, куда помещаются все команды, прошедшие декодирование, и переименование регистров (последняя операция выполняется только с теми командами, которые записывают свой результат в регистры). Окно команд обеспечивает отсрочку передачи команд на исполнение до момента готовности операндов, а также нужную очередность завершения команд и загрузки их результатов в регистры АРФ. Эта техника известна также под названием **шелвинг** (shelving). Существует два варианта окна команд - **централизованное** и **распределенное**.

Централизованное.

Данное окно реализуется в виде так называемого *табло* (score-board).

Табло (иногда слово Scoreboard переводят как табельная доска) представляет собой буферное запоминающее устройство, в котором хранится некоторое количество последних извлеченных из памяти и декодированных команд, а также текущая информация о доступности ресурсов, привлекаемых для их исполнения. Функциями табло являются оперативное выявление команд, для исполнения которых уже доступны все необходимые операнды и ресурсы, и выдача таких команд на исполнение в соответствующие функциональные блоки. Табло можно рассматривать как систему предварительной диспетчеризации команд, однако оно осуществляет контроль выполнения команд и после их выдачи.

Все извлеченные из памяти команды сразу же после их декодирования и, если это необходимо, переименования регистров заносятся в табло, причем с соблюдением порядка их следования в программе. Физически табло реализуется на основе ассоциативной памяти. Каждой команде выделяется одна ячейка, состоящая из нескольких полей:

- поля операции, где хранится дешифрованный код операции;
- двух полей операндов, размещающих значения операндов, если они известны, либо информацию о том, откуда эти операнды должны быть получены;
- поля результата, указывающего регистру, куда должен быть помещен результат выполнения данной команды;
- поля битов достоверности.

Содержимое табло

	ОП	Поле результата	Поле операнда 1	Поле операнда 2	Биты достоверности	
					ЗД1	ЗД2
1	ОП1	ИД	ЗН	ИД	1	0
2	ОП2	ИД	ЗН	ИД	1	0
3	ОП3	ИД	ИД	ЗН	0	1
4	ОП4	ИД	ЗН	ЗН	1	1
5	ОП5	ИД	ИД	ЗН	0	1
...						

ИД – идентификатор регистра.
ЗН – значение операнда

При занесении команд в табло, делается попытка заполнить поля операндов. Сперва поиск в регистровом файле – если бит 3Д регистра операнда в АРФ установлен в 0, значит операндом является результат предыдущей операции и дальше следует его искать в буфере переименования.

Если значение операнда ещё не вычислено – в поле заносится номер входа буфера переименования где находится последняя ссылка на искомый регистр.

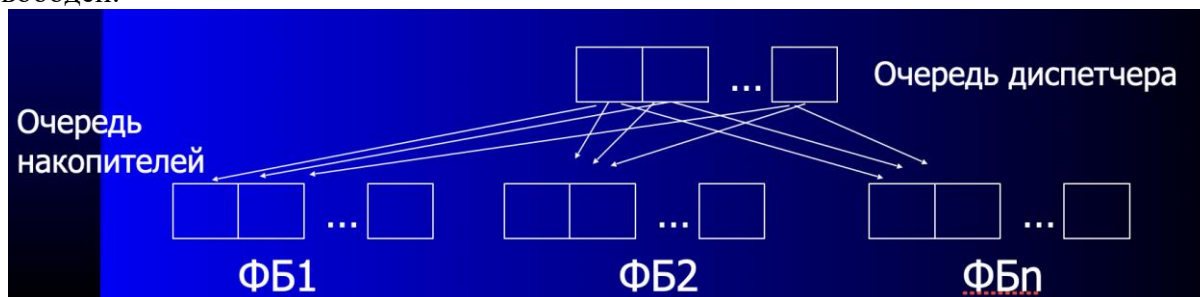
После завершения команды в функциональном блоке результат записывается в указанную ячейку буфера переименования, с одновременным ассоциативным поиском по всем строкам табло и заменой соотв. указателей входов на вычисленное значение.

Удаление команды из табло – основание для записи регистра в АРФ и удалении соотв. записи из буфера переименования.

Распределенное.

Вместо одного централизованного табло – для каждого ФБ свой буфер декодированных команд (- накопитель команд или схема резервирования).

Логика работы каждого накопителя аналогична централизованному окну команд. Выдача команд на выполнение только если все необходимые операнды - в наличие и ФБ свободен.



84. Типы прерываний. Контроллеры прерываний ПК.

Прерывание означает временное прекращение основного процесса вычислений для выполнения некоторых запланированных или незапланированных действий, вызываемых работой аппаратуры или программы.

Классификация прерываний

В зависимости от источника, прерывания делятся на

- аппаратные - возникают как реакция микропроцессора на физический сигнал от некоторого устройства (клавиатура, системные часы, клавиатура, жесткий диск и т.д.), по времени возникновения эти прерывания асинхронны, т.е. происходят в случайные моменты времени;
- программные - вызываются искусственно с помощью соответствующей команды из программы (int), предназначены для выполнения некоторых действий операционной системы, являются синхронными;
- исключения - являются реакцией микропроцессора на нестандартную ситуацию, возникшую внутри микропроцессора во время выполнения некоторой команды программы (деление на ноль, прерывание по флагу TF (трассировка)).

Общая классификация прерываний

- **внешние** - вызываются внешними по отношению к микропроцессору событиями (по существу - это группа аппаратных прерываний) Вложенных прерываний нет!
- **внутренние** - возникают внутри микропроцессора во время вычислительного процесса (по существу - это исключительные ситуации и программные прерывания).

Внешние прерывания возникают по сигналу какого-нибудь внешнего устройства. Внешние прерывания подразделяются на **не-маскируемые** и **маскируемые**.

Для того чтобы связать адрес обработчика прерывания с номером прерывания, используется таблица векторов прерываний, занимающая первый килобайт оперативной памяти - адреса от 0000:0000 до 0000:03FF. Таблица состоит из 256 элементов - FAR-адресов обработчиков прерываний. Эти элементы называются векторами прерываний. В первом слове элемента таблицы записано смещение, а во втором - адрес сегмента обработчика прерывания.

Программируемый контроллер прерываний (Programmable Interrupt Controller, PIC) отвечает за приём запросов прерываний от различных устройств, их хранение в ожидании обработки, выделение наиболее приоритетного из одновременно присутствующих запросов и выдачу его вектора в процессор, когда последний пожелает обработать прерывание. Слово «программируемый» в названии контроллера означает, что режимы его работы устанавливаются программно, а не являются жёстко «защитными».

Традиционная система прерываний ПК включает 16 линий запросов прерываний, из которых одна (IRQ2) используется для каскадного соединения контроллеров и поэтому никогда не вызывает.

85. Типы регенерации динамических ЗУ.

Простота схемы DRAM позволяет достичь высокой плотности размещения, в итоге, снизить стоимость. Главный недостаток - что накапливаемый на конденсаторе заряд со временем теряется. Среднее время утечки заряда DRAM составляет сотни(десятки) миллисекунд, поэтому, заряд следует регенерировать. Регенерация производится каждые 2-8 мс.

В различных типах ИМС динамической памяти нашли применение три основных метода регенерации:

Одним сигналом RAS (ROR - RAS Only Refresh);

Сигналом CAS, предваряющим сигнал RAS (CBR - CAS Before RAS);

Автоматическая регенерация (SR - Self Refresh).

Регенерация одним RAS использовалась еще в первых микросхемах DRAM. На шину адреса выдается адрес регенерируемой строки, сопровождаемый сигналом RAS. При этом выбирается строка ячеек и хранящиеся там данные поступают на внутренние цепи микросхемы, после чего записываются обратно. Так как сигнал CAS не появляется, цикл чтения/записи не начинается. В следующий раз на шину адреса подается адрес следующей строки и т. д., пока не восстановятся все ячейки, после чего цикл повторяется. К недостаткам метода можно отнести занятость шины адреса в момент регенерации, когда доступ к другим устройствам ВМ блокирован.

Особенность **метода CBR** в том, что если в обычном цикле чтения/записи сигнал RAS всегда предшествует сигналу CAS, то при появлении сигнала CAS первым начинается специальный цикл регенерации. В этом случае адрес строки не передается, а микросхема использует свой внутренний счетчик, содержимое которого увеличивается на единицу при каждом очередном CBR-цикле. Режим позволяет регенерировать память, не занимая шину адреса, то есть более эффективен.

Автоматическая регенерация памяти связана с энергосбережением, когда система переходит в режим «сна» и тактовый генератор перестает работать. При отсутствии внешних сигналов RAS и CAS обновление содержимого памяти методами ROR или CBR невозможно, и микросхема производит регенерацию самостоятельно, запуская собственный генератор, который тактирует внутренние цепи регенерации.

86. Микросхемы DDR SDRAM.

Синхронные DRAM – обмен информацией синхронизируется только тактовыми импульсами => улучшается КПД шины «ЦП-память». Время ожидания данных процессор может использовать для других действий, не связанных с обращением к памяти.

В синхронных DRAM вместо продолжительности цикла доступа оперируют минимально допустимым периодом тактовой частоты – время порядка 8-10 нс.

Микросхемы SDRAM. Аббревиатура SDRAM (Synchronous DRAM - Синхронная DRAM) используется для обозначения микросхем "обычных" синхронных динамических ОЗУ. Кардинальные отличия SDRAM от асинхронных динамических ОЗУ можно свести к четырем положениям:

- синхронный метод передачи данных на шину;
- применение нескольких (двух или четырех) внутренних банков памяти;
- конвейерный механизм пересылки пакета;
- передача части функций контроллера памяти логике самой микросхемы.

Синхронность памяти позволяет контроллеру памяти "знать" моменты готовности данных, за счет чего снижаются издержки циклов ожидания и поиска данных.

В отличие от BEDO (Burst Extended Data Output) конвейер позволяет передавать данные пакета по тактам, благодаря чему ОЗУ может работать бесперебойно на более высоких частотах, чем асинхронные ОЗУ.

Так как данные появляются на выходе микросхемы одновременно с тактовыми импульсами, упрощается взаимодействие памяти с другими устройствами ЭВМ.

87. Суперконвейерные процессоры.

Эффективность конвейера находится в прямой зависимости от того, с какой частотой на его вход подаются объекты обработки. Добиться n -кратного увеличения темпа работы конвейера можно двумя путями:

- разбиением каждой ступени конвейера на n «подступеней» при одновременном повышении тактовой частоты внутри конвейера также в n раз;
- включением в состав процессора n конвейеров, работающих с перекрытием.

Каждая из ступеней стандартного конвейера разбита на две более простые подступени, обозначенные индексами 1 и 2. Выполнение операции в подступенях занимает половину тактового периода. Тактирование операций внутри конвейера производится с частотой, вдвое превышающей частоту «внешнего» тактирования конвейера, благодаря чему на каждой ступени конвейера можно в пределах одного «внешнего» тактового периода выполнить две команды.



В сущности, суперконвейеризация сводится к увеличению количества ступеней конвейера, как за счет добавления новых ступеней, так и путем дробления имеющихся ступеней на несколько простых подступеней. Главное требование — возможность реализации операции в каждой подступени наиболее простыми техническими средствами, а значит, с минимальными затратами времени. Вторым, не менее важным, условием является одинаковость задержки во всех подступенях.

Критерием для причисления процессора к суперконвейерным служит число ступеней в конвейере команд. К суперконвейерным относят процессоры, где таких ступеней больше шести.

Суперконвейеризация здесь стала следствием разбиения этапов выборки команды и выборки операнда, а также введения в конвейер дополнительного этапа проверки тега, появление которой обусловлено архитектурой системы команд машины.

Удлинение конвейера ведет не только к усугублению проблем, характерных для любого конвейера, но и возникновению дополнительных сложностей. В длинном конвейере возрастает вероятность конфликтов. Дороже встает ошибка предсказания перехода — приходится очищать большее число ступеней конвейера, на что требуется больше времени. Усложняется логика взаимодействия ступеней конвейера.

88. Суперскалярные процессоры.

Суперскалярный процессор (англ. *superscalar processor*) — процессор, поддерживающий так называемый параллелизм на уровне инструкций (то есть, процессор, способный выполнять несколько инструкций одновременно) за счёт включения в состав его вычислительного ядра нескольких одинаковых функциональных узлов (таких как АЛУ, FPU, умножитель (*integer multiplier*), сдвигающее устройство (*integer shifter*) и другие устройства). Планирование исполнения потока инструкций осуществляется динамически вычислительным ядром (не статически компилятором). Способы увеличения производительности, которые могут использоваться совместно:

- Использование конвейера (англ. *pipelining*)
- увеличение количества функциональных узлов процессора (суперскалярность)
- увеличение количества ядер (многоядерность)
- увеличение количества процессоров (многопроцессорность)

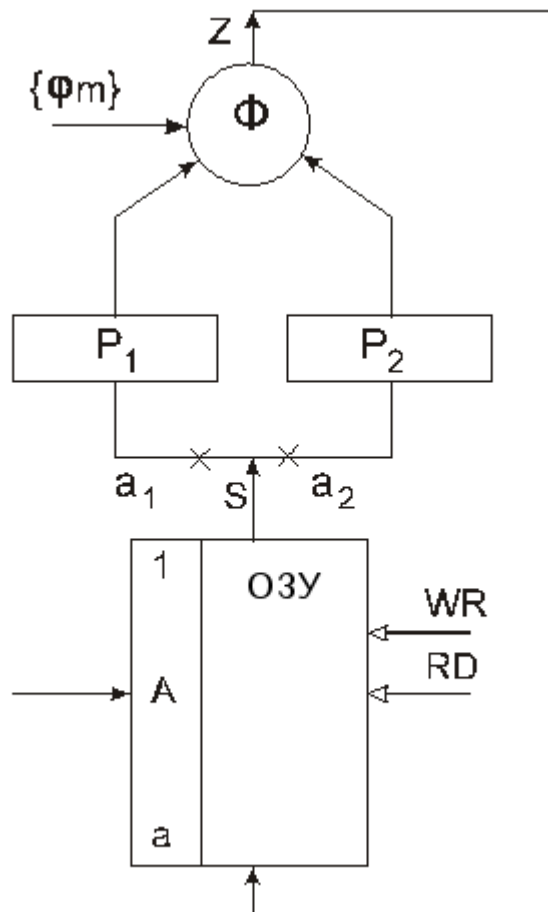
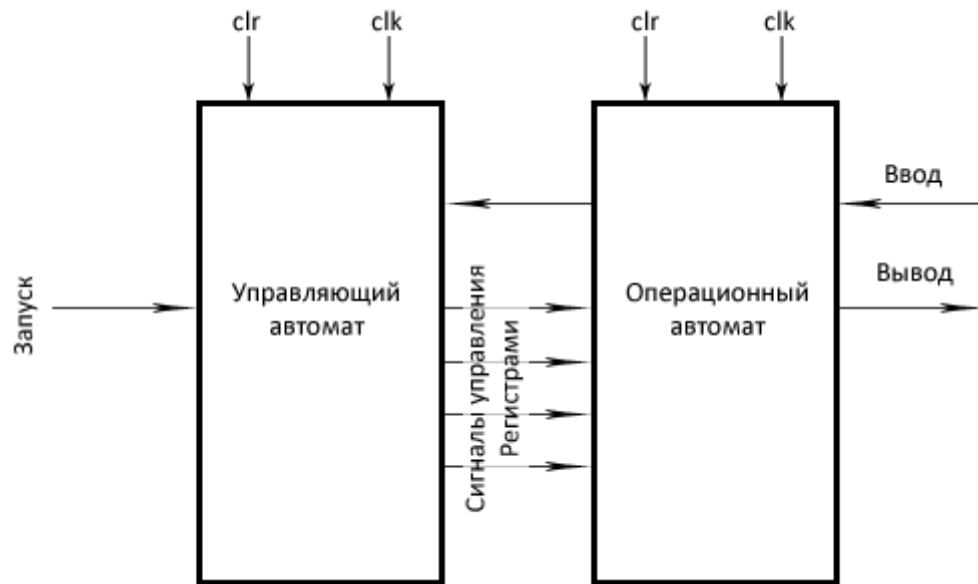
При использовании конвейера количество узлов остаётся прежним; увеличение производительности достигается за счёт одновременной работы узлов, ответственных за разные стадии обработки инструкций одного потока. При использовании суперскалярности увеличение производительности достигается за счёт одновременной работы большего количества одинаковых узлов, независимо обрабатывающих инструкции одного потока (в том числе, и большего количества конвейеров). При использовании нескольких ядер каждое ядро выполняет инструкции отдельного потока, причем каждое из них может быть суперскалярным и/или конвейерным. При использовании нескольких процессоров каждый процессор может быть многоядерным. В суперскалярном процессоре инструкция извлекается из потока инструкций (находящегося в памяти), определяется наличие или отсутствие зависимости инструкции по данным от других инструкций, затем инструкция выполняется. Одновременно, в течение одного такта, может выполняться несколько независимых инструкций. По классификации Флинна одноядерные суперскалярные процессоры относят к группе процессоров SISD (англ. *single instruction stream, single data stream* — один поток инструкций, один поток данных). Подобные процессоры, поддерживающие инструкции для работы с короткими векторами, могут быть отнесены к группе SIMD (англ. *single instruction stream, multiple data streams* — один поток инструкций, несколько потоков данных). Многоядерные суперскалярные процессоры относят к группе MIMD (англ. *multiple instruction streams, multiple data streams* — несколько потоков инструкций, несколько потоков данных).

89. Операционные автоматы. Структура М- автомата и S-автоматов.

Операционный автомат — набор функциональных элементов, таких как АЛУ и логические операции, которые выполняют обработку данных. Операционные устройства, такие как микропроцессоры, состоят из операционного автомата и управляющего автомата, большую часть которых занимает управляющий автомат, регулирующий передачу данных между операционным автоматом и памятью.

В некоторых операционных устройствах (ОУ) МО выполняются над большим числом внутренних слов, от десятков до сотен (например, специализированные процессоры ввода-вывода).

Для уменьшения стоимости таких устройств стандартную регистровую память целесообразно заменить оперативным запоминающим устройством (ОЗУ). ОА, память которого состоит из ОЗУ, называется S-автоматом.



Заданная микрооперация m реализуется комбинационной схемой F . Схема вычисляет значение слова $Z := m(P_1, P_2)$. Значение Z загружается в ОЗУ по адресу $A = A_3$. Такт S-автомата подразделяется на следующие действия:

1. чтение первого операнда из памяти по адресу A_1 : $P_1 := 3Y[A_1]$;
2. чтение второго операнда из ОЗУ по адресу A_2 : $P_2 := 3Y[A_2]$;

3. выполнение микрооперации $Z := {}_m(P_1, P_2)$;
4. запись результата в память ОЗУ $[A_3] := Z$.

Операционные автоматы, синтезированные на основе принципа обобщения комбинационных схем, называются М-автоматами.

Структурная организация М-автомата показана на рис.6.19.

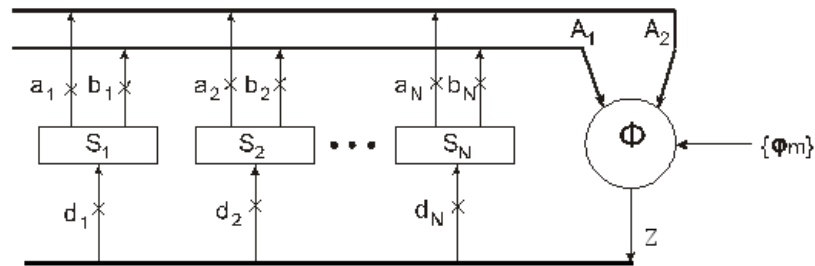


Рисунок 6.19 – Структурная организация М-автомата

В структуре М-автомата регистры $S_1, \dots, S_N$ используются для хранения слов. Обобщённая комбинационная схема используется для вычисления значений ${}_m(S_{p1}, \dots, S_{pk})$ выполняемых микрооперацией. Данные (операнды) поступают на входы схемы через шины A_1, A_2 . Для выборки слова на шину A_1 используются сигналы $a_1, \dots, a_n$, для выборки слова на шину A_2 используются сигналы $b_1, \dots, b_n$. Сигнал a_i инициирует присвоение $A_1 := S_i$, сигнал b_i инициирует присвоение $A_2 := S_j$. Схема настраивается на выполнение микрооперации $Z := {}_m(A_1, A_2)$ с помощью управляющего сигнала ${}_m$. Загрузка результата Z в любой регистр S_k инициируется управляющим сигналом d_k . Как ясно из рис. 6.19, М-автомат выполняет одну микрооперацию за такт. Таким образом, производительность М-автомата минимальна. Быстродействие М-автомата незначительно отличается от быстродействия I-автомата. Длительность такта немного увеличивается за счёт введения в схему мультиплексоров A_1, A_2 . Затраты оборудования М-автомата минимальны, поскольку комбинационная схема используется для выполнения всех эквивалентных микроопераций из множества Y .

90. Операционные автоматы. Структура I-автомата.

Проектируемый операционный автомат должен обеспечивать заданную производительность. Требуемая производительность может быть обеспечена только в том случае, если синтезируемая структура не вносит ограничений на совместимость микроопераций, т.е. обеспечивает возможность одновременного выполнения всех функционально совместимых микроопераций.

Для минимизации аппаратных затрат необходимо обобщать комбинационные схемы, выполняющие микрооперации из множества $Y = \{S_n := {}_j m(S)\}$, $n = 1, \dots, N$ и вычисляющие осведомительные сигналы из множества $X = \{x_e = {}_\psi e(S_{b1}, \dots, S_{bk})\}$.

Операционные автоматы, структура которых обеспечивает одновременного выполнения всех эквивалентных и функционально совместимых микроопераций при использовании минимально возможного числа комбинационных схем, относятся к классу I-автоматов.

Структура I-автомата синтезируется следующим образом:

1. Множество микроопераций $Y = \{y_1, \dots, y_m\}$ разбивается на подмножества $Y_1, \dots, Y_N$, соответствующие внутренними регистрами $S_1, \dots, S_N$.
2. На подмножествах Y_n , $n=1, \dots, N$ выделяются классы эквивалентных микроопераций K_{nj} , $j = 1, \dots, J_n$.

Параллельные вычисления. Структура организации и синтеза ИМ-автоматов. Суперскалярные процессоры.

Обобщённая структура I- автомата – класса устройств, в котором количество регистров = количество АЛУ.

Три части: 1) регистры памяти, 2) комбинац. схемы, реализующие АЛУ, 3) формирование признаков результата.

The diagram illustrates a super-scalar processor architecture. It features a horizontal bus at the top. Below the bus, there are multiple parallel processing units. Each unit consists of a register (RG) and an ALU. The registers are labeled $RG_1, RG_2, \dots, RG_k$. Each register has two inputs: $x_1, x_2, \dots, x_k$ and $a_{11}, \dots, a_{1k}$. The ALU units are labeled $АЛУ_1, АЛУ_2, \dots, АЛУ_k$. Each ALU has two inputs: A_1 and A_k . The ALU units are connected to the registers via data paths $d_1, d_2, \dots, d_k$. The ALU units also receive control signals $\Phi_1, \Phi_2, \dots, \Phi_k$ from the registers. The ALU units are connected to the registers via data paths $D_1, D_2, \dots, D_k$. The ALU units are connected to the registers via data paths $X_{АЛУ_1}, X_{АЛУ_2}, \dots, X_{АЛУ_k}$. The ALU units are connected to the registers via data paths $ШД_1 - ШД_k$.

[illegible]

91. Управляющие автоматы с жёсткой логикой. Автоматы Мили и Мура.

Управляющие автоматы

Согласно теории автоматов цифровым автоматом называется последовательностная схема, которая имеет набор состояний, обозначаемых обычно $A_1, A_2, \dots, A_N$.

В моменты прихода тактовых импульсов автомат переходит из одного состояния в другое, определяемое как текущим состоянием, так и набором входных (осведомительных) сигналов $X_1, X_2, \dots, X_N$, при этом формируется последовательность наборов выходных (управляющих) сигналов $Y_1, Y_2, \dots, Y_N$.

По способу формирования выходных сигналов автоматы подразделяют на автоматы Мили и автоматы Мура.

Слайд 17

Управляющие автоматы

- Для автомата Мили определяется следующий закон функционирования
- $A(t+1) = [A(t), X(t)];$
- $Y(t) = [A(t), X(t)],$
- Для автомата Мили, как видно из выражений, выходные сигналы $Y(t)$ зависят как от состояния автомата $A(t)$ в текущий момент времени t , так и от входных сигналов $X(t)$.
- Аналогично для автомата Мура
- $A(t+1) = [A(t), X(t)];$
- $Y(t) = [A(t)],$
- Для него, как видно из выражений, выходные сигналы $Y(t)$ зависят только от состояния автомата $A(t)$ в текущий момент времени t .

Слайд 18

Управляющие автоматы

- При практической реализации цифровых автоматов они могут быть реализованы тремя разными способами:
- На элементах с жесткой логикой
- На постоянном запоминающем устройстве (микропрограммные автоматы)
- На микропроцессорах или микроконтроллерах (программные автоматы).

Слайд 19

Управляющие автоматы



В управляющих автоматах на элементах с жесткой логикой – для хранения информации о состоянии используется набор триггеров, на их тактовый вход подается тактирующий сигнал, входные сигналы X подаются на комбинационные устройства, вырабатывающие сигналы управления для (функции возбуждения) триггеров. Выходные сигналы Y формируются при помощи других комбинационных схем из выходных сигналов триггеров А (для автомата Мили) или из выходных сигналов триггеров Аи входных сигналов Y (для автомата Мура, на рисунке).

Слайд 20

Управляющие автоматы

- выбор типа запоминающих и логических элементов;
- кодирование состояний автомата;
- синтез комбинационной схемы, формирующей сигналы возбуждения и выходные сигналы.
- В. М. Глушковым разработан общий конструктивный прием, называемый каноническим методом структурного синтеза автомата. Этот метод позволяет свести задачу синтеза автомата к задаче синтеза комбинационной схемы путем построения системы булевых функций, выражающих зависимость выходных сигналов и сигналов возбуждения от входных сигналов и состояний автомата. Полученные булевы функции минимизируются (например, методом карт Карно) и используются в качестве формы для построения схемы автомата.

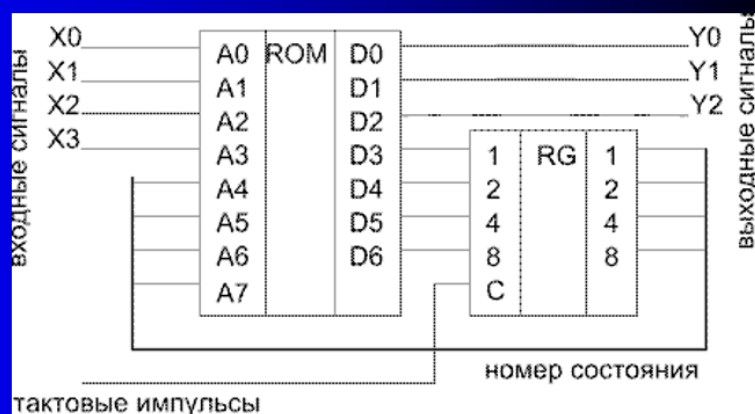
Слайд 21

Управляющие автоматы

- **Достоинства и недостатки автоматов с жесткой логикой.**
- Схемы с жесткой логикой, как правило, позволяют обеспечить **наибольшее быстродействие** из всех возможных методов построения цифрового автомата, однако при возрастании сложности реализуемых алгоритмов схемы автоматов с жесткой логикой очень быстро становятся **более сложными**, чем схемы автоматов с микропрограммным или программным управлением. Поэтому схемы автоматов с жесткой логикой в настоящее время используют только в том случае, когда требуется максимальное быстродействие.

Слайд 22

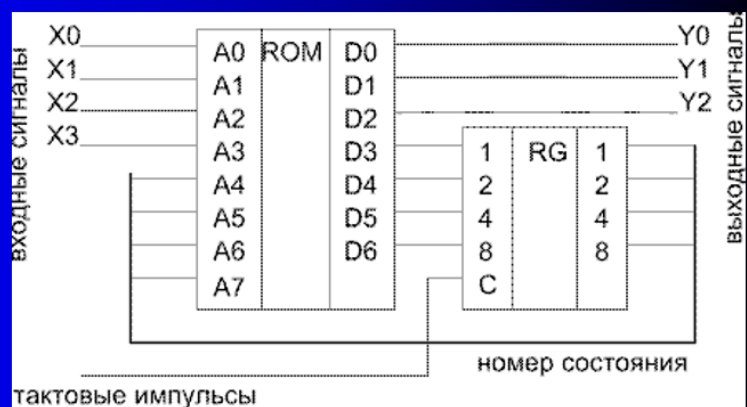
Управляющие автоматы



При построении управляющих автоматов с микропрограммной логикой его основой является ПЗУ. Входные (адресные) сигналы для ПЗУ формируются из номера текущего состояния, хранимого в регистре (A4-A7) и комбинации входных сигналов (X0 - X3, подаваемые на входы A0 - A3).

Слайд 23

Управляющие автоматы



В ячейке памяти с данным адресом в младших разрядах (D0 - D2), хранится комбинация выходных сигналов (Y0 - Y2), в старших - адрес перехода к новому состоянию.

(Схема на рисунке изображает микропрограммную реализацию автомата Мили - выходные являются функцией входных).

Слайд 24

Управляющие автоматы

- **Достоинства и недостатки автоматов с микропрограммной логикой.**
- Достоинством автоматов с микропрограммной логикой является простота реализации и особенно простота проектирования.
- Среди недостатков следует отметить то, что, поскольку далеко не все комбинации входных сигналов и номеров состояний (адресные сигналы ПЗУ) реализуются на практике, то не все ячейки ПЗУ используются. Также и с выходными сигналами - зачастую количество их используемых комбинаций существенно меньше возможного (2^n), где n -количество линий, на которых формируются сигналы.

Слайд 25

Управляющие автоматы

- **Достоинства и недостатки автоматов с микропрограммной логикой.**
- Все это приводит к тому, что, либо приходится ставить на вход и выход автомата преобразователи кодов, а это ухудшает быстродействие схемы, либо использовать ПЗУ большого объема. Все это приводит к тому, что большую часть площади структуры современного микропроцессора занимает ПЗУ, в которой хранятся микропрограммы работы его блоков.
- Автоматы с микропрограммной логикой следует использовать тогда, когда алгоритм работы слишком сложен для устройств с жесткой логикой, но требования к быстродействию не допускают применения программных автоматов.

Слайд 26

Управляющие автоматы

- **Построение схем автоматов с программной логикой.**
- Автоматы с программной логикой являются микропроцессорными системами, в которых алгоритм функционирования реализован программным путем. Алгоритм программы для такого автомата строится следующим образом ? каждая вершина графа автомата заменяется группой блоков, в которых формируются выходные сигналы и анализируются входящие, стрелки, соединяющие вершины графа автомата заменяются командами условного или безусловного перехода.
- **Достоинства и недостатки автоматов с программной логикой.**
- По сравнению с автоматами на элементах с жесткой логикой или микропрограммных , программные автоматы обладают наиболее низким быстродействием.
- В то же время в случае достаточно сложных алгоритмов работы программные автоматы оказываются наиболее простыми с точки зрения схемотехники. Особенно это ощутимо в случае использования микроконтроллеров.

Слайд 27

93. Кодирование микрокоманд. Горизонтальный, вертикальный способы кодирования.

6. Микропрограммирование. Горизонтальный, вертикальный и смешанный способы кодирования команд.

- **Определение:** элементарная функциональная операция, выполняемая в компьютере за один тактовый интервал времени и приводимая в действие одним управляющим сигналом, называется микрооперацией. Соответственно совокупность микроопераций, выполняемых параллельно во времени, называют микрокомандой.
- Выполнение всех действий в процессоре осуществляется в виде отработки множества команд (процессоры с фиксированной архитектурой) или микрокоманд (в процессорах с разрядно-модульной организацией связей). В частности, микропроцессорная секция *K1804BC1* функционирует под действием микрокоманд, хранящихся в управляющей памяти. При этом всю последовательность микрокоманд, предназначенных для реализации некоторого преобразования, называют микропрограммой.

Слайд 12

6. Микропрограммирование. Горизонтальный, вертикальный и смешанный способы кодирования команд.

- Стандартная структура микрокоманды обычно включает в свой состав две основные части: операционную и адресную. Адресная часть используется при формировании адреса следующей микрокоманды. При этом в состав компьютера включается специальный блок (*БМУ*), который позволяет выполнять различные действия над адресами. Управление процессом преобразования данных осуществляется операционной частью микрокоманды. В этом случае микрооперации кодируются двоичными кодами, для формирования которых используют три основных способа:

- 1. *Вертикальное кодирование или микропрограммирование.* Данный метод предполагает, что всей совокупности микроопераций присваиваются коды, образующие ряд целых чисел без знака. При отработке микрокоманд данного типа дешифратор преобразует код микрокоманды в один управляющий сигнал, который и является сигналом микрооперации.

Слайд 13

6. Микропрограммирование. Горизонтальный, вертикальный и смешанный способы кодирования команд.

- 1. *Вертикальное кодирование или микропрограммирование.* Недостатки способа заключаются:
 - 1) в необходимости построения достаточно сложного дешифратора;
 - 2) в увеличении длины микропрограммы (в одной микрокоманде содержится только одна микрооперация);
 - 3) в отсутствии наглядности функционального назначения микрокоманды.
- *Горизонтальное микропрограммирование.* При данном способе каждому разряду операционной части микрокоманды ставится в соответствие определенная микрооперация. Наличие в разряде единицы говорит о том, что некоторая микрооперация выполняется, причем выполнение действий под управлением i разряда не зависит от состояния других j разрядов. Длина операционной части при горизонтальном микропрограммировании составляет число m , т.е. соответствует количеству микроопераций.

Слайд 14

6. Микропрограммирование. Горизонтальный, вертикальный и смешанный способы кодирования команд.

Горизонтальное микропрограммирование. Достоинства метода:

- 1) возможность выполнения одновременно любого числа функционально совместимых микроопераций;
- 2) отсутствие схем дешифрации.

Недостатки:

- 1) большая длина микрокоманды, достигающая в отдельных случаях сотен бит;
- 2) большая разрядность $ЗУ$ микрокоманд или большая длительность выборки управляющих слов при малой разрядности $ЗУ$ $Мк$.

94. Кодирование микрокоманд. Смешанный способ кодирования команд.

Смешанный способ микропрограммирования. В данном случае сочетаются горизонтальный и вертикальный способы кодирования. При этом множество микроопераций m разбивается на k подмножеств, а в составе микрокоманды каждому подмножеству выделяется свое поле:

$$m = \bigcup_{i=1}^k m_i \quad \text{Слайд 15}$$

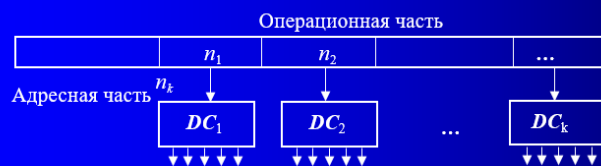
6. Микропрограммирование. Горизонтальный, вертикальный и смешанный способы кодирования команд.

Смешанный способ микропрограммирования.

Соответственно каждое поле имеет свое функциональное назначение, а микрооперации внутри подмножества (поля) кодируются вертикально.

Длина операционной части микрокоманды при смешанном микропрограммировании определяется величиной

$$n_{o.ч} = \sum_{i=1}^k \text{int} \log_2 m_i$$



где k – число функциональных групп;

m_i – число микроопераций в группе.

Смешанный способ микропрограммирования применяется наиболее часто. Однако в данном случае требуется решать задачу разбиения множества всех микроопераций на непересекающиеся подмножества, т.е. микрооперации подмножества m_i не должны включаться в подмножество m_j и наоборот.

Слайд 16

95. Последовательность выполнения микропрограмм.



Рис. 6.11. Организация управляющей памяти

На рис. 6.11 показано возможное размещение микрокоманд в памяти микропро-грамм. Содержимое ПМП определяет последовательность микроопераций, которые должны выполняться на каждом этапе цикла команды, а также последовательность этапов. Каждый этап представлен соответствующей микропрограммой. Микро-программы завершаются микрокомандой перехода, определяющей последую-щие действия. В управляющей памяти имеется также специальная микропрогра-ма-переключатель: в зависимости от текущего кода операции она указывает, этап исполнения какой команды должен быть выполнен.

Большей частью микрокоманды в микропрограмме выполняются последовательно, однако в общем случае очередность микроопераций не является фиксиро-ванной. По этой причине в УУ необходимо предусмотреть эффективную систему реализации переходов. Переходы, как безусловные, так и условные, являются неотъемлемой частью любой микропрограммы.

96. Организация памяти микропрограмм.

Функциональные возможности и структура микропрограммных автоматов в значительной степени зависят от организации памяти для хранения микропрограмм. Основные способы организации памяти микропрограмм можно свести к следующим вариантам [3].

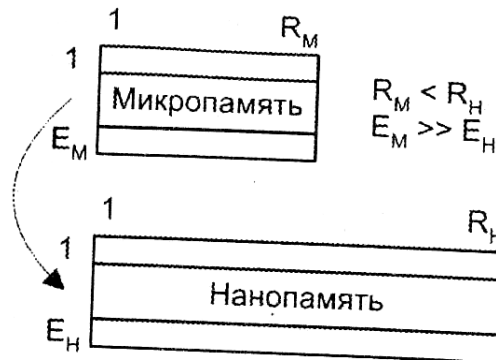
1. Каждое слово ПМП содержит одну микрокоманду. Это наиболее простая организация ПМП. Основной недостаток — в каждом такте работы МГЛА требуется обращение к памяти микропрограмм, что приводит к снижению быстродействия МПА.

2. Одно слово ПМП содержит несколько микрокоманд. В результате осуществляется одновременное считывание из ПМП нескольких МК, что позволяет повысить быстродействие УУ.

3. Сегментация ПМП, при которой память разделяется на сегменты, состоящие из 2^q соседних слов, при этом адрес слова АМК разделяется на два поля: S и A . Поле S определяет адрес сегмента, а поле A — адрес слова в сегменте. Адрес S устанавливается специальной микрокомандой. В последующих микрокомандах указывается только адрес слова A в сегменте. Таким образом, разрядность адресной части МК уменьшается.

4. Двухуровневая память. Первый уровень — микропамять, хранящая микрокоманды. Второй уровень — нанопамять, содержащая нанокоманды.

МК выбирается из микропамяти и служит для адресации слова нанопамати, которое включает в себя необходимые сигналы управления. После выполнения микроопераций из микропамяти выбирается следующая МК. Иначе говоря, нанопамать реализует функцию шифратора управляющих сигналов. Двухуровневая память рассматривается как способ для уменьшения необходимой емкости ПМП, ее использование целесообразно только при многократном повторении микрокоманд в микропрограмме.



97. Закон Амдала.

Закон Амдала - иллюстрирует ограничение роста производительности вычислительной системы с увеличением количества вычислителей. Джин Амдал сформулировал закон в 1967 году, обнаружив простое по существу, но непреодолимое по содержанию ограничение на рост производительности при распараллеливании вычислений: «В случае, когда задача разделяется на несколько частей, суммарное время ее выполнения на параллельной системе не может быть меньше времени выполнения самого медленного фрагмента». Согласно этому закону, ускорение выполнения программы за счет распараллеливания её инструкций на множестве вычислителей ограничено временем, необходимым для выполнения ее последовательных инструкций.

98. Закон Густафсона.

Закон Густафсона - оценка максимально достижимого ускорения выполнения параллельной программы, в зависимости от количества одновременно выполняемых потоков вычислений («процессоров») и доли последовательных расчётов.

При оценке ускорения параллельного выполнения закон Амдала предполагает, что объем задачи остается постоянным. Величина ускорения по закону Амдала показывает, во сколько раз меньше времени потребуется параллельной программе для выполнения. Однако величину ускорения можно рассматривать и как увеличение объема выполненной задачи за постоянный промежуток времени. Закон Густафсона появился именно из этого предположения.

99. Организация памяти вычислительных систем.

В зависимости от того, каким образом организована память многопроцессорных (многомашинных) систем, различают вычислительные системы с общей памятью (shared memory) и ВС с распределенной памятью (distributed memory).

В системах с общей памятью (ее часто называют также совместно используемой или разделяемой памятью) память ВС рассматривается как общий ресурс, и каждый из процессоров имеет полный доступ ко всему адресному пространству. Системы с общей памятью называют сильно связанными (closely coupled systems). Подобное построение вычислительных систем имеет место как в классе SIMD, так и в классе MIMD. Иногда, чтобы подчеркнуть это обстоятельство, вводят специальные подклассы, используя для их обозначения аббревиатуры SM-SIMD (Shared Memory SIMD) и SM-MIMD (Shared Memory MIMD).

В варианте с распределенной памятью каждому из процессоров придается собственная память. Процессоры объединяются в сеть и могут при необходимости обмениваться данными, хранящимися в их памяти, передавая друг другу так называемые сообщения. Такой вид ВС называют слабо связанными (loosely coupled systems). Слабо связанные системы также встречаются как в классе SIMD, так и в классе MIMD, и иной раз, чтобы подчеркнуть данную особенность, вводят подклассы DM-SIMD (Distributed Memory SIMD) и DM-MIMD (Distributed Memory MIMD).

В некоторых случаях вычислительные системы с общей памятью называют мультипроцессорами, а системы с распределенной памятью - мультикомпьютерами.

Различие между общей и распределенной памятью — это разница в структуре виртуальной памяти, то есть в том, как память выглядит со стороны процессора. Физически почти каждая система памяти разделена на автономные компоненты доступ к которым может производиться независимо. Общую память от распределенной отличает то, каким образом подсистема памяти интерпретирует поступивший от процессора адрес ячейки. Для примера положим, что процессор выполняет команду Load R0, i, означающую «Загрузить регистр R0 содержимым ячейки i». В случае общей памяти i — это глобальный адрес, и для любого процессора указывает на одну и ту же ячейку. В распределенной системе памяти i — это локальный адрес. Если два процессора выполняют команду Load R0, i, то каждый из них обращается к i-й ячейке в своей локальной памяти то есть к разным ячейкам, и в регистры R0 могут быть загружены неодинаковые значения.

Различие между двумя системами памяти должно учитываться программистом, поскольку оно определяет способ взаимодействия частей распараллеленной программы. В варианте с общей памятью достаточно создать в памяти структуру данных и передавать в параллельно используемые подпрограммы ссылки на эту структуру. В системе с распределенной памятью необходимо в каждой локальной памяти иметь копию совместно используемых данных. Эти копии создаются путем вкладывания разделяемых данных в сообщения, посылаемые другим процессорам.

100. Модели архитектур распределённой памяти.

В системе с распределенной памятью каждый процессор обладает собственной памятью и способен адресоваться только к ней. Некоторые авторы называют этот тип систем многомашиными ВС или мульткомпьютерами, подчеркивая тот факт, что блоки, из которых строится система, сами по себе являются небольшими вычислительными системами с процессором и памятью. Модели архитектур с распределенной памятью принято обозначать как архитектуры без прямого доступа к удаленной памяти (NORMA, No Remote Memory Access). Такое название следует из того факта, что каждый процессор имеет доступ только к своей локальной памяти. Доступ к удаленной памяти (локальной памяти другого процессора) возможен только путем обмена сообщениями с процессором, которому принадлежит адресуемая память.

Подобная организация характеризуется рядом достоинств. Во-первых, при доступе к данным не возникает конкуренции за шину или коммутаторы — каждый процессор может полностью использовать полосу пропускания тракта связи с собственной локальной памятью. Во-вторых, отсутствие общей шины означает, что нет и связанных с этим ограничений количества процессоров: размер системы ограничивает только сеть, объединяющая процессоры. В-третьих, снимается проблема когерентности памяти. Каждый процессор вправе самостоятельно менять свои данные, не заботясь о согласовании копий данных в собственной локальной памяти.

Основной недостаток ВС с распределенной памятью заключается в сложности обмена информацией между процессорами. Если какой-то из процессоров нуждается в данных из памяти другого процессора, он должен обменяться с этим процессором сообщениями. Это приводит к двум видам издержек:

- требуется время для того, чтобы сформировать и переслать сообщение от одного процессора к другому;
- для обеспечения реакции на сообщения от других процессоров принимающий процессор должен получить запрос прерывания и выполнить процедуру обработки этого прерывания.

Структура системы с распределенной памятью приведена на рис. 11.4. В левой части (рис. 11.4, а) показан один процессорный элемент (ПЭ). Он включает в себя собственно процессор (Р), локальную память (М) и два контроллера ввода/вывода (К_о и К_и). В правой части (рис. 11.4, б) показана четырехпроцессорная система, иллюстрирующая, каким образом сообщения пересылаются от одного процессора к другому. По отношению к каждому ПЭ все остальные процессорные элементы можно рассматривать просто как устройства ввода/вывода. Для послышки сообщения в другой ПЭ процессор формирует блок данных в своей памяти и извещает свой локальный контроллер о необходимости передачи информации на внешнее устройство. По сети межсоединений это сообщение пересылается на приемный контроллер ввода/вывода, принимающего ПЭ. Последний находит место для сообщения в собственной локальной памяти и уведомляет процессор-источник о получении сообщения.

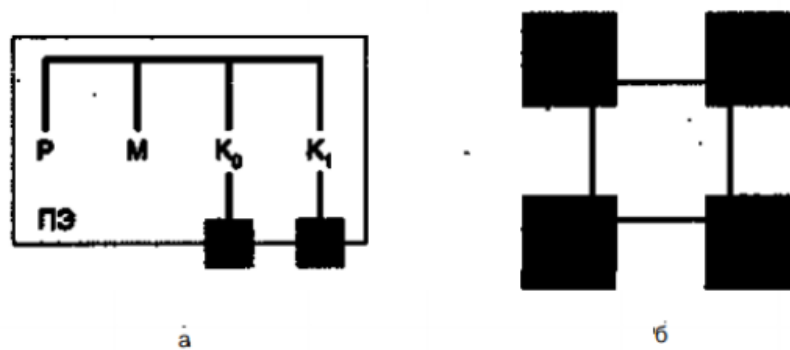


Рис. 11.4. Вычислительная система с распределенной памятью: а - процессорный элемент; б - объединение процессорных элементов

Интересный вариант системы с распределенной памятью представляет собой модель распределенной совместно используемой памяти (DSM, Distribute Shared Memory), известной также и под другим названием архитектуры с неоднородным доступом к памяти и программным обеспечением когерентности (SC-NUMA, Software-Coherent Non-Uniform Memory Architecture). Идея этой модели состоит в том, что ВС, физически будучи системой с распределенной памятью, благодаря операционной системе представляется пользователю как система с общей памятью. Это означает, что операционная система предлагает пользователю единое адресное пространство, несмотря на то что фактическое обращение к памяти "чужого" компьютера ВС по-прежнему обеспечивается путем обмена сообщениями.

101. Когерентность памяти.

В простейшей форме проблему когерентности -памяти можно пояснить следующим образом (рис. 2.6). Пусть два процессора P_1 и P_2 связаны с общей памятью посредством шины. Сначала оба процессора читают переменную x . Копии блоков, содержащих эту переменную, пересылаются из основной памяти в локальные и обоих процессоров (рис. 2.6, а). Далее процессор P_1 выполняет операцию увеличения значения переменной x на единицу. Так как копия пере-менной уже находится в -памяти данного процессора, произойдет -попадание и значение x будет изменено только в -памяти 1. Если теперь процессор P_2 вновь выполнит операцию чтения x , то также произойдет -попадание и P_2 получит хранящееся в его -памяти «старое» значение x (рис. 2.6, б).

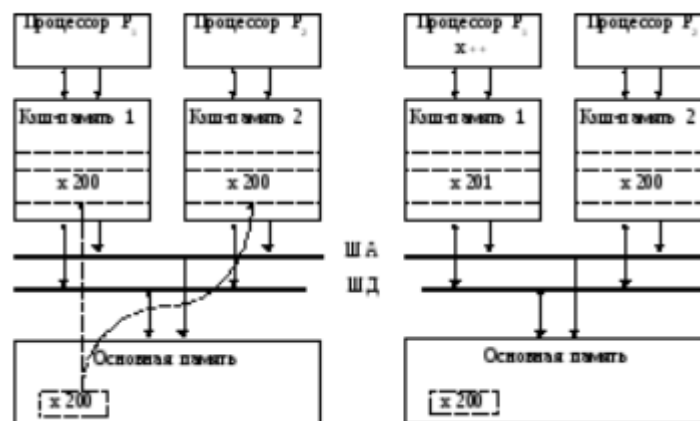


Рис. 2.6. Иллюстрация проблемы когерентности памяти: а – содержимое памяти до изменения значения x ; б – после изменения.

Поддержание согласованности требует, чтобы при изменении элемента данных одним из процессоров соответствующие изменения были проведены в -памяти остальных процессоров, где есть копия измененного элемента данных, а также в общей памяти.

Аналогичная проблема возникает и в однопроцессорных системах, где присутствует несколько уровней -памяти. Здесь требуется согласовать содержимое ей разных уровней.

В решении проблемы когерентности выделяются два подхода: программный и аппаратный.

102. Протокол MESI.

Было придумано множество протоколов когерентности, отличающиеся алгоритмами, количеством состояний и, как следствие скоростью работы и масштабируемостью. Большинство современных протоколов когерентности представляют вариации протокола MESI.

В данной схеме каждая линия а может находиться в одном из четырех состояний:

Модифицированная (M) англ. modified. Таким флагом может быть помечена только линия в одном е. Данное состояние означает, что данная линия была изменена, но до памяти эти изменения еще не дошли. Хозяин такой линии может спокойно читать и писать в нее без опроса остальных.

Эксклюзивная (E) англ. exclusive. Обозначенная таким флагом линия, так же, как и М-линия может находиться только в одном е. Содержащиеся в ней данные полностью идентичны данным в оперативной памяти. Записывать и читать из нее можно без внешних запросов, так как она хранится только в одном е. После записи такая строка должна быть помечена, как модифицированная.

Разделяемая (S) англ. shared. Линия может одновременно содержаться в ах нескольких устройств и использоваться совместно. Запросы на запись в такую линию всегда идут на общую шину, что приводит к тому, что все линии с таких адресов в остальных ах помечаются как недействительные. При этом содержание основной памяти также обновляется. Чтение же из такой линии, не требует ни каких внешних запросов.

Недействительная (I) англ. invalid. Такая линия считается невалидной и попытка прочитать приведет к -промаху. Линия помечается недействительной в случае если она пуста или содержит устаревшую информацию.

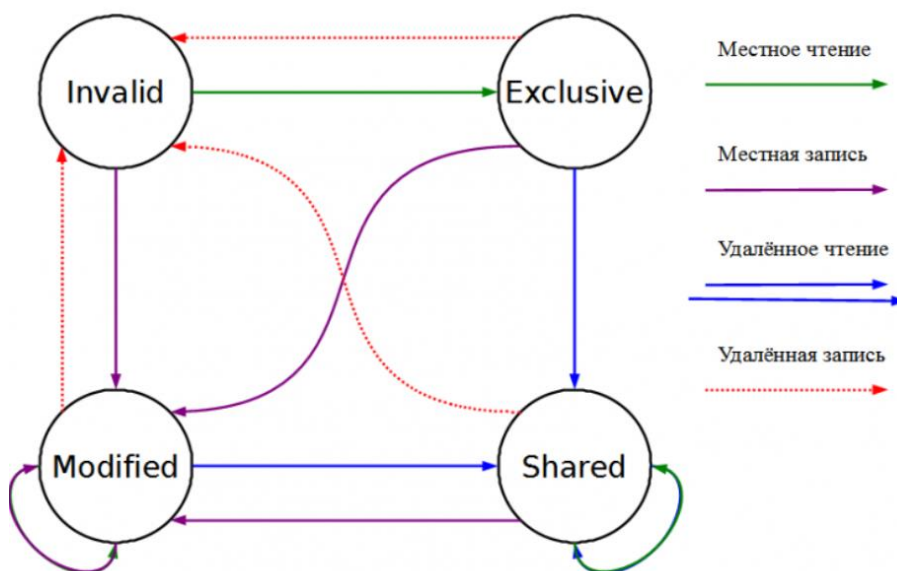


Диаграмма переходов протокола MESI. Доступ является местным, если он был инициирован процессором данного а, удаленным — если его вызвал любой другой.

Также хочу рассмотреть одну из оптимизаций протокола MESI
MOESI

Для данного протокола флаги состояния были расширены еще одним значением «Владелец» (O) англ. owner. Данное состояние является комбинацией состояний «Модифицированная» и «Разделяемая». Такое состояние позволяет избежать

необходимости записи измененной линии в память, тем самым снижая трафик, направленный в память. Линия в таком состоянии содержит наиболее свежие данные.

103. Организация виртуальной памяти.

Виртуальная память — метод управления памятью компьютера, позволяющий выполнять программы, требующие больше оперативной памяти, чем имеется в компьютере, путем автоматического перемещения частей программы между основной памятью и вторичным хранилищем (например, жёстким диском). В системе с виртуальной памятью используемые программами адреса, называемые виртуальными адресами, транслируются в физические адреса в памяти компьютера. Трансляцию виртуальных адресов в физические выполняет аппаратное обеспечение, называемое блоком управления памятью. Для программы основная память выглядит как доступное и непрерывное адресное пространство либо как набор непрерывных сегментов, вне зависимости от наличия у компьютера соответствующего объёма оперативной памяти. Управление виртуальными адресными пространствами, соотнесение физической и виртуальной памяти, а также перемещение фрагментов памяти между основным и вторичным хранилищами выполняет операционная система.

2 варианта организации виртуальной памяти:

- 1) Страничная организация виртуальной памяти
- 2) Сегментная организация виртуальной памяти

Страничная:

В большинстве современных операционных систем виртуальная память организуется с помощью страничной адресации. Оперативная память делится на страницы: области памяти фиксированной длины (например, 4096 байт), которые являются минимальной единицей выделяемой памяти. Исполняемый процессором пользовательский поток обращается к памяти с помощью адреса виртуальной памяти, который делится на номер страницы и смещение внутри страницы. Процессор преобразует номер виртуальной страницы в адрес соответствующей ей физической страницы. Если ему не удалось это сделать, то требуется дозаполнение буфера путём обращения к таблице страниц. Если страница была выгружена из оперативной памяти, то операционная система подкачивает страницу с жёсткого диска. При запросе на выделение памяти операционная система может «сбросить» на жёсткий диск страницы, к которым давно не было обращений. Критические данные обычно находятся в оперативной памяти.

Сегментная:

Механизм организации виртуальной памяти, при котором виртуальное пространство делится на части произвольного размера — сегменты. Этот механизм позволяет, к примеру, разбить данные процесса на логические блоки. Для каждого сегмента, как и для страницы, могут быть назначены права доступа к нему пользователя и его процессов. При загрузке процесса часть сегментов помещается в оперативную память, а часть сегментов размещается в дисковой памяти. Сегменты одной программы могут занимать в оперативной памяти несмежные участки. Во время загрузки система создает таблицу сегментов процесса, в которой для каждого сегмента указывается начальный физический адрес сегмента в оперативной памяти, размер сегмента, правила доступа, признак модификации, признак обращения к данному сегменту за последний интервал времени и некоторая другая информация. Если виртуальные адресные пространства нескольких процессов включают один и тот же сегмент, то в таблицах сегментов этих процессов делаются ссылки на один и тот же участок оперативной памяти, в который данный сегмент загружается в единственном экземпляре. Система с сегментной организацией функционирует аналогично системе со страничной организацией: время от времени происходят прерывания, связанные с отсутствием нужных сегментов в памяти, при необходимости освобождения памяти некоторые сегменты выгружаются, при каждом обращении к оперативной памяти выполняется преобразование виртуального адреса в физический. Виртуальный адрес при

сегментной организации памяти может быть представлен парой (g, s) , где g — номер сегмента, а s — смещение в сегменте. Физический адрес получается путём сложения начального физического адреса сегмента, найденного в таблице сегментов по номеру g , и смещения s . Недостатком данного метода распределения памяти является фрагментация на уровне сегментов и более медленное по сравнению со страничной организацией преобразование адреса.

104. Построение управляющих автоматов.

Управляющий автомат — часть микропроцессора, выполняющая управляющие функции над данными. Представляет собой конечный автомат. Согласно теории автоматов, цифровым автоматом называется последовательность схем, которая имеет набор состояний, обозначаемых обычно $A_1, A_2, \dots, A_N$. В моменты прихода тактовых импульсов автомат переходит из одного состояния в другое, определяемое как текущим состоянием, так и набором входных (осведомительных) сигналов $X_1, X_2, \dots, X_N$, при этом формируется последовательность наборов выходных (управляющих) сигналов $Y_1, Y_2, \dots, Y_N$. Для каждого автомата задается закон функционирования или алгоритм переходов из одного состояния в другое под действием разных комбинаций входных сигналов с описанием комбинаций формируемых при этом выходных сигналов. Такой алгоритм может быть задан либо в виде графа, либо в виде таблицы переходов. Когда автомат не работает, он находится в начальном состоянии A_1 . При запуске автомат сохраняет состояние A_1 в течение одного такта, за время которого формируются соответствующие значения входных сигналов X_1 . По окончании первого такта автомат переключается в очередное состояние A_2 предписанное законом функционирования, и в ОА начинается выполнение следующий набор микроопераций. Момент окончания микропрограммы отмечается возвратом автомата в начальное состояние a_1 .

105. Способы кодирования микропрограмм.

Информация о том, какие сигналы управления должны быть сформированы в процессе выполнения текущей МК, в закодированном виде содержится в микрооперационной части (МО) микрокоманды. Способ кодирования микроопераций во многом определяет сложность аппаратных средств устройства управления и его скоростные характеристики. Применяемые в микрокомандах варианты кодирования сигналов управления можно свести к трем группам: горизонтальное, вертикальное, смешанное.

При горизонтальном микропрограммировании под каждый сигнал управления в микрооперационной части микрокоманды выделен один разряд. Это позволяет в рамках одной микрокоманды формировать любые сочетания сигналов управления, чем обеспечивается максимальный параллелизм выполнения микроопераций. Отсутствует необходимость в декодировании МО и выходы регистра микрокоманды могут быть непосредственно подключены к операционному устройству системы. Большие затраты и низкая эффективность.

При максимальном (вертикальном) кодировании каждой микрооперации присваивается определенный код, например, ее порядковый номер в полном списке возможных микроопераций. Этот код и заносится в МО. Минимальные аппаратные затраты на хранение микрокоманд, однако возникает необходимость в дешифраторе, который должен преобразовать код микрооперации в соответствующий сигнал управления. При большом количестве сигналов управления дешифратор вносит значительную временную задержку и в каждой МК указывается лишь один сигнал управления, инициирующий только одну микрооперацию, за счет чего увеличиваются длина микропрограммы и время ее реализации. Проблема устраняется при подключении к выходам дешифратора шифратора, что приводит к увеличению количества сигналов управления, формируемых одновременно.

В зависимости от принципа разбиения микроопераций на группы различают горизонтально-вертикальное и вертикально-горизонтальное кодирование. В горизонтально-вертикальном методе в каждую группу включаются взаимно несовместимые сигналы управления (микрооперации), т.е. сигналы управления, которые никогда не встречаются вместе в одной микрокоманде. При этом сигналы, обычно формируемые в одном и том же такте, оказываются в разных группах. Внутри каждой группы сигналы управления кодируются максимальным (вертикальным) способом, а группы – минимальным (горизонтальным) способом.

При вертикально-горизонтальном способе все множество сигналов управления (микроопераций) также делится на несколько групп, однако в группу включаются сигналы управления (микрооперации), наиболее часто встречающиеся вместе в одном такте. Поле МО делится на две части: МО1 и МО2. Поле МО1, длина которого равна максимальному количеству сигналов управления (микроопераций) в группе, кодируется горизонтально, а поле МО2, указывающее на принадлежность к определенной группе, – вертикально. Со сменой группы меняются и сигналы управления поля МО1.

При горизонтальном, вертикальном и горизонтально-вертикальном способах кодирования микроопераций каждое поле микрокоманды несет фиксированные функции, т.е. имеет место прямое кодирование. При косвенном кодировании одно из полей отводится для интерпретации других полей. Примером косвенного кодирования микроопераций может служить вертикально-горизонтальное кодирование.

В случае прямого кодирования микрокоманд каждое поле микрокоманд имеет фиксированные функции. Косвенное кодирование характеризуется наличием дополнительных полей, содержимое которых меняет смысл основных полей микрокоманды. Таким образом, интерпретация полей, формирующих управляющие сигналы, зависит от бит дополнительных полей. Косвенное кодирование сокращает объем памяти микропрограмм, так как позволяет уменьшать длину микрокоманд. Однако косвенное кодирование в некоторой степени нарушает стройность микропрограммного управления, вызывает усложнение дешифраторов и приводит к снижению скорости работы из-за потерь времени на операционную часть адресная часть дополнительных полей микрокоманды.

106. Команды арифметической и логической обработки.

В данную группу входят команды, обеспечивающие арифметическую и логическую обработку информации в различных формах ее представления. Для каждой формы представления чисел в АСК (архитектура системы команд) обычно предусматривается некий стандартный набор операций. Помимо вычисления результата выполнение арифметических и логических операций сопровождается формированием в АЛУ признаков (флагов), характеризующих этот результат. Наиболее часто фиксируются такие признаки, как:

- Z (Zero) — нулевой результат;
- N (Negative) — отрицательный результат;
- V (oVerflow) — переполнение разрядной сетки;
- C (Carry) — наличие переноса.

К стандартному набору операций над целыми числами, представленными в форме с фиксированной запятой, следует отнести:

- двухместные арифметические операции (операции с двумя операндами): сложение, вычитание, умножение и деление;
- одноместные арифметические операции (операции с одним операндом): вычисление абсолютного значения (модуля) операнда, изменение знака операнда;

- операции сравнения, обеспечивающие сравнение двух целых чисел и выработку признаков, характеризующих соотношение между сопоставляемыми величинами ($=$, $<$, $>$, $<=$, $>=$).

Часто этот перечень дополняют такими операциями, как вычисление остатка от целочисленного деления, сложение с учетом переноса, вычитание с учетом заема, увеличение значения операнда на единицу (инкремент), уменьшение значения операнда на единицу (декремент). Отметим, что выполнение арифметических команд может дополнительно сопровождаться перемещением данных из устройства ввода в АЛУ или из АЛУ на устройство вывода.

Для работы с числами, представленными в форме с плавающей запятой, в АСК большинства машин предусмотрены:

- основные арифметические операции;
- операции сравнения, обеспечивающие сравнение двух вещественных чисел с выработкой признаков;
- операции преобразования: формы представления (между фиксированной и плавающей запятой), формата представления (с одинарной и двойной точностью).

Стандартная система команд ЭВМ содержит команды для выполнения различных логических операций над отдельными битами слов или других адресуемых единиц. Такие команды предназначены для обработки символьных и логических данных. Минимальный набор поддерживаемых логических операций — это «НЕ», «И», «ИЛИ» и сложение по модулю 2. В дополнение к побитовым логическим операциям, практически во всех АСК предусмотрены команды для реализации операций логического, арифметического и циклического сдвигов. При логическом сдвиге влево или вправо сдвигаются все разряды слова. Биты, вышедшие за пределы разрядной сетки, теряются, а освободившиеся позиции заполняются нулями. При арифметическом сдвиге данные трактуются как целые числа со знаком, причем бит знака не изменяет положения. При сдвиге вправо освободившиеся позиции заполняются значением знакового разряда, а при сдвиге влево — нулями. Арифметические сдвиги позволяют ускорить выполнение некоторых арифметических операций. Так, если числа представлены двоичным дополнительным кодом, то сдвиги влево и вправо эквивалентны соответственно умножению и делению на 2. При циклическом сдвиге смещаются все разряды слова, причем значение разряда, выходящего за пределы слова, заносится в позицию, освободившуюся с противоположной стороны, то есть потери информации не происходит. Одно из возможных применений циклических сдвигов — это перемещение интересующего бита в крайнюю левую (знаковую) позицию, где он может быть проанализирован как знак числа.

Для работы со строками в АСК обычно предусматриваются команды, обеспечивающие перемещение, сравнение и поиск строк. В большинстве машин перечисленные операции просто имитируются за счет других команд.

Команды преобразования осуществляют изменение формата представления данных. Примером может служить преобразование из десятичной системы счисления в двоичную или перевод 8-разрядного кода символа из кодировки ASCII в кодировку EBCDIC, и наоборот.

107. Команды управления потоком команд.

Концепция фон-неймановской вычислительной машины предполагает, что команды программы, как правило, выполняются в порядке их расположения в памяти. Для получения адреса очередной команды достаточно увеличить содержимое счетчика команд на длину текущей команды. В то же время основные преимущества ВМ заключаются именно в возможности изменения хода вычислений в зависимости от возникающих в процессе счета результатов. С этой целью в АСК (архитектура системы команд) вычислительной машины включаются команды, позволяющие нарушить естественный порядок следования и передать управление в иную точку программы. В адресной части

таких команд содержится адрес точки перехода (адрес той команды, которая должна быть выполнена следующей). Переход реализуется путем загрузки адреса точки перехода в счетчик команд (вместо увеличения содержимого этого счетчика длину команды).

В системе команд ВМ можно выделить три типа команд, способных изменить последовательность вычислений:

- безусловные переходы;
- условные переходы (ветвления);
- вызовы процедур и возвраты из процедур.

Согласно приведенным данным, среди команд рассматриваемой группы доминируют условные переходы. Несмотря на то что присутствие в программе большого числа команд безусловного перехода считается признаком плохого стиля программирования, такие команды обязательно входят в АСК любой ВМ. Для их обозначения в языке ассемблера обычно используется английское слово `jmp` (прыжок). Команда безусловного перехода обеспечивает переход по заданному адресу без проверки каких-либо условий.

Условный переход происходит только при соблюдении определенного условия, в противном случае выполняется следующая по порядку команда программы. Большинство производителей ВМ в своих ассемблерах обозначают подобные команды словом `branch` (ветвление). Условием, на основании которого осуществляется переход, чаще всего выступают признаки результата предшествующей арифметической или логической операции. Каждый из признаков фиксируется в своем разряде регистра флагов процессора. Возможен и иной подход, когда решение о переходе принимается в зависимости от состояния одного из регистров общего назначения, куда предварительно помещается результат операции сравнения. Третий вариант — это объединение операций сравнения и перехода в одной команде. В системе команд ВМ для каждого признака результата предусматривается своя команда ветвления (иногда — две: переход при наличии признака и переход при его отсутствии). Большая часть условных переходов связана с проверкой взаимного соотношения двух величин или с равенством (неравенством) некоторой величины нулю. Последний вид проверок используется в программах наиболее интенсивно.

Одной из форм команд условного перехода являются команды пропуска. В них адрес перехода отсутствует, а при выполнении условия происходит пропуск следующей команды, то есть предполагается, что отсутствующий в команде адрес следующей команды эквивалентен адресу текущей команды, увеличенному на длину пропускаемой команды. Такой прием позволяет сократить длину команд передачи управления.

Для всех языков программирования характерно интенсивное использование механизма процедур. Процедура может быть вызвана в любой точке программы. Для ВМ такой вызов означает, что в этой точке необходимо выполнить процедуру, после чего вернуться в точку, непосредственно следующую за местом вызова. Процедурный механизм базируется на командах вызова процедуры, обеспечивающих переход из текущей точки программы к начальной команде процедуры, и командах возврата из процедуры, для возврата в точку, непосредственно расположенную за командой вызова. Такой режим предполагает наличие средств для сохранения текущего состояния содержимого счетчика команд в момент вызова (запоминание адреса точки возврата) и его восстановления при выходе из процедуры.

Форматы команд:

Типовая команда, в общем случае, должна указывать:

- подлежащую выполнению операцию;
- адреса исходных данных (операндов), над которыми выполняется операция;
- адрес, по которому должен быть помещен результат операции.

В соответствии с этим команда состоит из двух частей: операционной и адресной

Формат команды определяет ее структуру, то есть количество двоичных разрядов, отводимых под всю команду, а также количество и расположение отдельных полей

команды. Полем называется совокупность двоичных разрядов, кодирующих составную часть команды. При создании ВМ выбор формата команды влияет на многие характеристики будущей машины. Оценивая возможные форматы, нужно учитывать следующие факторы:

- общее число различных команд;
- общую длину команды;
- тип полей команды (фиксированной или переменной длины) и их длина;
- простоту декодирования;
- адресуемость и способы адресации;
- стоимость оборудования для декодирования и исполнения команд.

Длина команды

Это важнейшее обстоятельство, влияющее на организацию и емкость памяти, структуру шин, сложность и быстродействие ЦП. С одной стороны, удобно иметь в распоряжении мощный набор команд, то есть как можно больше кодов операций, операндов, способов адресации, и максимальное адресное пространство. Однако все это требует выделения большего количества разрядов под каждое поле команды, что приводит к увеличению ее длины. Вместе с тем, для ускорения выборки из памяти желательно, чтобы команда была как можно короче, а ее длина была равна или кратна ширине шины данных. Для упрощения аппаратуры и повышения быстродействия ВМ длину команды обычно выбирают кратной байту, поскольку в большинстве ВМ основная память организована в виде 8-битовых ячеек. В рамках системы команд одной ВМ могут использоваться разные форматы команд. Обычно это связано с применением различных способов адресации. В таком случае в состав кода команды вводится поле для задания способа адресации (СА).

108. В чём состоит особенность SIMD-команд и в каком формате должны быть представлены операнды? Что такое «арифметика с насыщением» и где она применяется?

SIMD (Single Instruction Multiple Data) - это единый поток инструкций и множественный поток данных. Проще говоря, одна инструкция может обрабатывать несколько данных одновременно.

В отличие от обычных команд, оперирующих двумя операндами, SIMD-команды обрабатывают сразу две группы операндов. Операнды таких команд обычно представлены в одном из упакованных форматов.

Арифметика с насыщением - это версия арифметики, в которой все операции, такие как сложение и умножение, ограничены фиксированным диапазоном между минимальным и максимальным значением. Если результат операции больше максимального, он устанавливается на максимум; если он ниже минимума, он ограничивается минимумом.

Арифметика насыщения позволяет использовать эффективные алгоритмы для решения многих задач, особенно при цифровой обработке сигналов. Например, регулировка уровня громкости звукового сигнала может привести к переполнению.

109. Особенности архитектуры ARM процессоров.

Чипы архитектуры ARM применяются с набором команд RISC (Reduced Instruction Set Computer). Его преимущество в изначально небольшом наборе простых команд, которые обрабатываются с минимальными затратами.

Архитектура ARM «выезжает» за счет низкого энергопотребления и в целом-то достаточной производительности, например, прорисовки веб-страниц и воспроизведения медиаконтента.

Режим Thumb - в этом режиме процессор выполняет альтернативный набор 16-битных команд, позволяющий размещать большее количество инструкции в меньший объем памяти по сравнению с 32-битным набором команд.

Расширение команд SIMD используется для увеличения скорости обработки аудио и видео в приложениях, работающих на маломощных платформах.

Технология TrustZone - это аппаратная технология, предназначена для защиты памяти устройства от критических сбоев.

Последние версии микропроцессоров ARM стали многоядерными. Технологический процесс создания чипов ARM освоен на уровне 28-нанометрового техпроцесса.

110. Организация виртуальной памяти. Многоуровневые таблицы трансляции адресов.

Виртуальная память — это концепция, которая позволяет уйти от использования физических адресов, используя вместо них виртуальные, и это даёт ряд преимуществ:

- Расширение реального адресного пространства.

- Создание изолированных адресных пространств для различных процессов, что повышает безопасность системы.

- Задание различных свойств для разных участков памяти.

- При этом вся виртуальная память делится на участки памяти постоянного размера, называемые страницами.

- Использовании нескольких уровней таблицы страниц.

Они могут представлять огромный разреженный каталог страниц, где неиспользуемые регионы не требуют выделения памяти. Следовательно, представление намного более компактное и есть возможность для многих процессов в памяти иметь каталоги страниц без существенного влияния на производительность.

Сегодня наиболее сложные структуры таблиц страниц содержат 4 уровня.

Чтобы определить физический адрес, соответствующий виртуальному адресу, процессор сначала определяет адрес наивысшего уровня в каталоге. И потом идет по подуровням, прибавляя каждый раз к нач. адресу.

111. Архитектура ARM Cortex процессоров

Благодаря Гарвардской архитектуре операции выполняются параллельно.

Благодаря методу bit banding осуществляется эффективный доступ к регистрам и флагам устройств ввода-вывода, расположенных в SRAM. Благодаря данному методу можно сэкономить размер и время выполнения программы.

- 4 гигабайтное адресное пространство Cortex.

- Доступ к SRAM осуществляется напрямую через арбитр.

- Архитектура с двухмостовой схемой APB (Advanced Peripheral Buses)

- У ЦПУ Cortex имеется регистровый файл, состоящий из 16 32-битных регистров, отдельный регистр, который называется регистром статуса программы.

В ядро Cortex входит 24-битный вычитающий счетчик с функциями автоматической перезагрузки и генерации прерывания. Он называется таймером SysTick и предназначен для использования в качестве стандартного таймера во всех Cortex-микроконтроллерах.

- Полностью 32-битная архитектура: все регистры 32-битные.

- Большое количество (от 16) регистров общего назначения, характерное для архитектуры RISC.

- Контроллер прерываний, который поддерживает до 240 прерываний на все случаи жизни, до 256 их приоритетов, и обеспечивает быструю реакцию на прерывания.

- Контроллер DMA. Позволяет периферии читать/писать данные в RAM без участия МК.